

# Natural Language Processing

Guida all'esame — scritto e orale

La teoria del corso, dalle slide del docente, spiegata da zero,

con le risposte alle domande d'esame

Corso di Laurea Triennale in Informatica — Università di Catania

Prof. Misael Mongiovì, a.a. 2026–2027

Questa guida prepara la parte di **teoria** dell'esame di Natural Language Processing: la **prova scritta**, a domande aperte sugli argomenti del corso, e la parte dell'**orale** che verifica le conoscenze. È scritta per chi parte quasi da zero: ogni argomento comincia dall'idea, spiegata a parole, e solo dopo arriva alle formule. Il progetto pratico e le esercitazioni in Python non sono trattati.

**Da dove vengono i contenuti.** Dalle slide del docente, con la pagina indicata fra parentesi quadre accanto a ogni definizione, formula e procedimento. Dove una slide è solo un titolo o una figura, il passo mancante viene dal libro di testo del corso (Jurafsky e Martin, *Speech and Language Processing*, edizione di gennaio 2026), alle pagine indicate nel programma, ed è segnato come tale. Il programma ufficiale del corso dà anche le modalità d'esame e sette esempi di domande: sono tutte risolte nell'ultimo capitolo.

**Come sono controllati i conti.** Ogni esempio numerico è ricalcolato da un programma (cartella **verifiche/**).

# Indice

|                                                                                            |           |
|--------------------------------------------------------------------------------------------|-----------|
| <b>Come usare questa guida</b>                                                             | <b>1</b>  |
| <b>1 Il Natural Language Processing e il testo: espressioni regolari e normalizzazione</b> | <b>4</b>  |
| 1.1 Che cos'è il Natural Language Processing . . . . .                                     | 4         |
| 1.2 Le espressioni regolari . . . . .                                                      | 4         |
| 1.3 Parole, token e type; i corpora . . . . .                                              | 5         |
| 1.4 La pipeline di normalizzazione . . . . .                                               | 6         |
| 1.4.1 Tokenization . . . . .                                                               | 6         |
| 1.4.2 Byte-Pair Encoding (BPE) . . . . .                                                   | 7         |
| 1.4.3 Word normalization: case folding, lemmatization, stemming . . . . .                  | 7         |
| 1.4.4 Sentence segmentation . . . . .                                                      | 8         |
| <b>2 Modelli di linguaggio n-gram</b>                                                      | <b>9</b>  |
| 2.1 Dalla chain rule all'assunzione di Markov . . . . .                                    | 9         |
| 2.2 L'esempio del Berkeley Restaurant Project . . . . .                                    | 10        |
| 2.3 Valutare un modello di linguaggio: la perplexity . . . . .                             | 10        |
| 2.4 Generalizzazione, zeri e smoothing . . . . .                                           | 11        |
| 2.4.1 Add-one (Laplace) . . . . .                                                          | 12        |
| 2.4.2 Backoff e interpolazione . . . . .                                                   | 12        |
| 2.4.3 n-gram web-scale e stupid backoff . . . . .                                          | 13        |
| 2.4.4 Absolute discounting . . . . .                                                       | 13        |
| 2.4.5 Kneser-Ney . . . . .                                                                 | 13        |
| 2.4.6 Advanced language modeling . . . . .                                                 | 14        |
| <b>3 Classificazione di testi: Naive Bayes e valutazione</b>                               | <b>16</b> |
| 3.1 Il problema della classificazione . . . . .                                            | 16        |
| 3.2 Naive Bayes . . . . .                                                                  | 16        |
| 3.2.1 Esempio svolto: sentiment con add-1 . . . . .                                        | 18        |
| 3.2.2 Varianti per il sentiment e osservazioni . . . . .                                   | 18        |
| 3.3 Valutare un classificatore . . . . .                                                   | 19        |
| <b>4 Classificazione di testo con la regressione logistica</b>                             | <b>22</b> |
| 4.1 Classificatori generativi e discriminativi . . . . .                                   | 22        |
| 4.2 Il modello: feature, punteggio, sigmoide, decisione . . . . .                          | 23        |
| 4.3 Apprendimento: la cross-entropy loss . . . . .                                         | 24        |
| 4.4 Gradient descent e stochastic gradient descent . . . . .                               | 24        |
| 4.5 Overfitting e regolarizzazione . . . . .                                               | 25        |
| 4.6 Regressione logistica multinomiale e softmax . . . . .                                 | 26        |
| <b>5 Semantica vettoriale e word embedding</b>                                             | <b>28</b> |
| 5.1 Dal significato alla distribuzione . . . . .                                           | 28        |
| 5.2 Vettori sparsi: le matrici di co-occorrenza . . . . .                                  | 29        |
| 5.3 Misurare la similarità: prodotto scalare e coseno . . . . .                            | 29        |

|           |                                                                                   |           |
|-----------|-----------------------------------------------------------------------------------|-----------|
| 5.4       | tf-idf: pesare i conteggi . . . . .                                               | 30        |
| 5.5       | Embedding densi: word2vec . . . . .                                               | 31        |
| 5.6       | Proprietà degli embedding, analogie e bias . . . . .                              | 33        |
| <b>6</b>  | <b>Reti neurali per il linguaggio</b>                                             | <b>35</b> |
| 6.1       | L'unità neurale . . . . .                                                         | 35        |
| 6.2       | Perché serve un hidden layer: il problema XOR . . . . .                           | 35        |
| 6.3       | Reti feed-forward per la classificazione del testo . . . . .                      | 36        |
| 6.4       | Il modello di linguaggio neurale feed-forward . . . . .                           | 37        |
| 6.5       | Come si addestra: loss, gradiente, backpropagation . . . . .                      | 38        |
| 6.6       | Oltre la finestra fissa: RNN e LSTM, in breve . . . . .                           | 39        |
| <b>7</b>  | <b>Transformer, BERT ed encoder-decoder</b>                                       | <b>41</b> |
| 7.1       | Dall'attenzione al self-attention layer . . . . .                                 | 41        |
| 7.2       | Il transformer block . . . . .                                                    | 42        |
| 7.3       | Il transformer come modello di linguaggio . . . . .                               | 43        |
| 7.4       | BERT: un encoder bidirezionale pre-addestrato . . . . .                           | 45        |
| 7.5       | Architetture encoder-decoder . . . . .                                            | 47        |
| <b>8</b>  | <b>POS tagging, Named Entity Recognition ed entity linking</b>                    | <b>49</b> |
| 8.1       | Parti del discorso e POS tagging . . . . .                                        | 49        |
| 8.2       | Named Entity Recognition e schema BIO . . . . .                                   | 50        |
| 8.3       | Entity linking: BLINK, bi-encoder e cross-encoder . . . . .                       | 51        |
| <b>9</b>  | <b>Parser sintattici: costituenti, CKY e dipendenze</b>                           | <b>53</b> |
| 9.1       | Perché la sintassi: grammatiche a costituenti . . . . .                           | 53        |
| 9.2       | Forma normale di Chomsky e algoritmo CKY . . . . .                                | 53        |
| 9.3       | Dependency parsing . . . . .                                                      | 54        |
| 9.4       | Transition-based parsing . . . . .                                                | 55        |
| <b>10</b> | <b>Parser semantici e rappresentazioni del significato</b>                        | <b>58</b> |
| 10.1      | Rappresentare il significato . . . . .                                            | 58        |
| 10.2      | Semantica model-theoretic e logica del primo ordine . . . . .                     | 58        |
| 10.3      | Ruoli semantici: PropBank e FrameNet . . . . .                                    | 59        |
| 10.4      | Rappresentazioni a grafo: FRED e AMR . . . . .                                    | 60        |
| 10.5      | SPRING: da testo ad AMR con un transformer . . . . .                              | 61        |
| <b>11</b> | <b>GPT e Large Language Models</b>                                                | <b>63</b> |
| 11.1      | GPT: un decoder pre-addestrato, poi adattato al compito . . . . .                 | 63        |
| 11.2      | GPT-2 e GPT-3: la scala, zero-shot e in-context learning . . . . .                | 64        |
| 11.3      | InstructGPT: allineare il modello con il feedback umano . . . . .                 | 65        |
| 11.4      | Usare un LLM: prompting e campionamento . . . . .                                 | 66        |
| 11.5      | GPT contro BERT . . . . .                                                         | 66        |
| <b>12</b> | <b>Applicazioni: question answering e RAG, machine translation, summarization</b> | <b>68</b> |
| 12.1      | Question answering e RAG: cercare, poi leggere (o generare) . . . . .             | 68        |
| 12.2      | Machine translation e text summarization . . . . .                                | 70        |
| <b>13</b> | <b>Le domande d'esame</b>                                                         | <b>73</b> |
| 13.1      | Come si risponde a una domanda aperta . . . . .                                   | 73        |
| 13.2      | Le sette domande del programma, risolte . . . . .                                 | 73        |
| 13.3      | Altre domande probabili, per argomento . . . . .                                  | 76        |

## Come usare questa guida

---

### Com'è fatto l'esame

Il programma ufficiale del corso dice: «Le conoscenze saranno valutate mediante una prova scritta, seguita da una prova orale che valuterà un progetto precedentemente concordato, la capacità di applicare le tecniche apprese a problemi pratici, e la padronanza del linguaggio tecnico del settore» [programma del corso]. Nelle slide del docente lo scritto è «domande aperte sugli argomenti teorici trattati nel corso» [slide Intro p. 16].

- **Lo scritto:** domande aperte di teoria. Il programma ne elenca sette come esempio (sotto): sono del docente, quindi la guida le tratta per intero nel capitolo 13, e ogni capitolo contiene i riquadri «Come rispondere» per le domande del suo argomento.
- **L'orale:** discussione del progetto e verifica delle conoscenze. «Ha valore integrativo rispetto alla prova scritta»: non serve a recuperare punti, è parte del voto. Il progetto in Python va concordato prima con il docente e non è trattato in questa guida.
- **Il voto** premia, oltre alla correttezza, «le capacità di esposizione e di collegamento dei contenuti» e, per il 28–30, lo «spirito critico» [programma del corso]. Per questo ogni risposta modello finisce con i limiti del metodo e un collegamento a un altro argomento.

#### Le sette domande d'esempio del programma

1. Descrivere le tecniche di smoothing dei modelli di linguaggio basati su N-grams, evidenziando vantaggi e limiti dei vari metodi.
2. Discutere l'importanza dei word embeddings nella comprensione del linguaggio naturale. Illustrare il funzionamento del modello Word2Vec.
3. Illustrare la rappresentazione delle parole mediante vettori sparsi basata su tf-idf e cosine similarity. Discuterne brevemente i limiti.
4. Descrivere il modello BERT illustrandone la strategia di pre-training.
5. Descrivere l'utilizzo di una feed-forward network con un hidden layer per la classificazione di testo.
6. Esplorare il ruolo dei Transformer nell'ambito dei modelli neurali di linguaggio. Illustra il funzionamento del self-attention layer in un blocco transformer.
7. Confrontare i modelli generativi GPT con quelli bidirezionali come BERT, evidenziando i diversi ambiti di applicazione.

Il programma avverte che sono indicative: «le domande effettivamente proposte in sede d'esame potranno divergere, anche in modo significativo». Servono a capire il tipo di domanda, non a prevedere il compito.

## Che cosa c'è nella guida

- **Capitoli 1–12:** la teoria del corso, nell'ordine del programma, dalle slide del docente: normalizzazione del testo, modelli n-gram, classificazione (Naive Bayes, regressione logistica), semantica vettoriale, reti neurali, transformer e BERT, POS tagging e NER, parser sintattici e semantici, GPT e Large Language Models, applicazioni.
- **Capitolo 13:** come si risponde a una domanda aperta, le sette domande del programma risolte, e altre domande probabili per ogni argomento.
- **Non ci sono:** il progetto, le esercitazioni in Python e il materiale sui cluster HPC. La guida è solo teoria.

Dentro ogni capitolo, ogni argomento segue lo stesso ordine: prima **l'idea** a parole, poi le definizioni e le formule, poi un **esempio** piccolo con i numeri, poi il riquadro «**Come rispondere**», con la scaletta della risposta da dare allo scritto, e infine gli **errori tipici**.

## Come si studia, se si parte da zero

1. **Prima l'idea, poi la formula.** Leggi il riquadro «L'idea» e prova a ripeterlo a voce, senza guardare. Se non ci riesci, la formula che segue sarà solo un simbolo da ricordare.
2. **Leggi i simboli.** In ogni formula chiediti che cosa sono i simboli e da dove vengono i numeri: la guida lo dice sempre al primo uso.
3. **Rifai gli esempi su carta.** I conti sono piccoli apposta (un bigram, tre token, una matrice  $2 \times 2$ ). Chi li sa rifare sa spiegare il metodo.
4. **Rispondi per iscritto.** Per ogni riquadro «Come rispondere», scrivi la risposta in dieci righe a libro chiuso, poi confronta. È l'esercizio più vicino allo scritto.
5. **Collega.** Alla fine di ogni argomento chiediti a che cos'altro si lega (Naive Bayes e regressione logistica, tf-idf e word2vec, BERT e GPT). Il collegamento è un criterio di voto dichiarato.

## Da dove viene ogni cosa

- Le fonti sono le **slide del docente**, citate con il nome del deck e la pagina, per esempio [slide N-gram p. 17] o [slide Transformer p. 9]. I deck: Intro, Normalizzazione, N-gram, Naive Bayes, Regressione logistica, Semantica vettoriale, Reti neurali, Transformer, POS-NER, Parser sintattici, Parser semantici, Applicazioni, GPT.
- Il **libro di testo** del corso (Jurafsky e Martin, *Speech and Language Processing*, edizione di gennaio 2026, pagine del PDF come nel programma) entra solo dove una slide è un titolo o una figura senza il passo che serve, o dove il programma elenca un argomento che le slide nominano soltanto: RAG, HMM per il POS tagging, RNN e LSTM, instruction tuning. Questi passi sono segnati [dal libro, p. N; non nelle slide].
- Ciò che serve a capire ma non sta né nelle slide né nel libro è segnato [non nelle fonti del corso]. Le spiegazioni a parole dei riquadri «L'idea» sono della guida.
- **Ogni numero è ricalcolato da un programma** (cartella **verifiche**/), compresi quelli delle slide: dove un numero del docente non torna, la guida riporta quello giusto.

**Convenzioni usate in tutta la guida**

- I termini tecnici restano in inglese, come nelle slide e nelle domande del programma (word embedding, pre-training, fine-tuning, self-attention layer; invariabili al plurale), con la spiegazione italiana al primo uso.
- La notazione è quella delle slide:  $P(w_n | w_{n-1})$  per le probabilità condizionate,  $\sigma(z)$  per la sigmoide, **Q**, **K**, **V** per query, key e value. Dove conta la base del logaritmo, è detta.
- Il separatore decimale è il punto (0.25).

**I riquadri**

---

|                |                                              |
|----------------|----------------------------------------------|
| <b>Indaco</b>  | l'idea, a parole, prima delle formule        |
| <b>Blu</b>     | definizioni, formule e algoritmi delle slide |
| <b>Viola</b>   | come rispondere a una domanda d'esame        |
| <b>Verde</b>   | che cosa chiede l'esame                      |
| <b>Rosso</b>   | errori e confusioni tipiche                  |
| <b>Arancio</b> | regole e convenzioni da ricordare            |
| <b>Grigio</b>  | note                                         |

---

# Capitolo 1

## Il Natural Language Processing e il testo: espressioni regolari e normalizzazione

### 1.1 Che cos'è il Natural Language Processing

#### L'idea

Per un calcolatore un testo è solo una sequenza di caratteri: non sa dove finisce una parola, né quale significato di «pesca» è quello giusto. Il NLP è l'insieme di tecniche che da quei caratteri risale al significato e, all'inverso, produce testo: il corso costruisce questa catena dai token di questo capitolo ai Large Language Model.

**Definizione e applicazioni.** Il **Natural Language Processing** (NLP, elaborazione del linguaggio naturale) è il punto d'incontro fra linguaggio umano e sistemi computazionali: la tecnologia che permette alle macchine di *comprendere* il linguaggio scritto o parlato, *interpretarne* sfumature, contesto e significati, *generare* testi coerenti e pertinenti e *interagire* in linguaggio naturale. [slide Intro p. 5] Lo usiamo ogni giorno: motori di ricerca che capiscono query conversazionali, machine translation (traduzione automatica), correttori grammaticali, filtri anti-spam, sistemi di raccomandazione, assistenti virtuali per il customer support. [slide Intro p. 8]

**Evoluzione e sfide.** Dal NLP tradizionale (regole predefinite e analisi statistica per task specifici) al machine learning (algoritmi che apprendono pattern dai dati), al deep learning (reti neurali profonde), fino ai modelli generativi come GPT, che producono contenuti originali e contestuali; [slide Intro p. 9] oggi il linguaggio unifica la *multimodalità*: testo, visione (capire e descrivere immagini), audio (riconoscimento e sintesi del parlato), azione (istruzioni per sistemi autonomi). [slide Intro p. 10] Sfide tecniche: *ambiguità* (più significati), *contesto* (sfumature situazionali), *sarcasmo e ironia* (significati impliciti), *diversità delle lingue*; etiche: *bias* nei dati di training, *privacy*, *misinformation* (contenuti fuorvianti), *trasparenza* delle decisioni. [slide Intro p. 11]

#### «Che cos'è il NLP, quali applicazioni e quali sfide»

1. *Definizione*: incontro fra linguaggio umano e calcolo; la macchina comprende, interpreta, genera, interagisce. *Applicazioni*: ricerca, machine translation, correttori, anti-spam, raccomandazione, assistenti virtuali; sui testi: estrarre significato, riassumere, classificare, sentiment analysis [slide Intro p. 7].
2. *Evoluzione*: regole e statistica → machine learning → deep learning → modelli generativi (GPT); multimodalità con il linguaggio come coordinatore.
3. *Sfide*: ambiguità, contesto, ironia, diversità delle lingue; bias, privacy, misinformation, trasparenza; già tokenization e normalizzazione (sez. 1.4) dipendono da lingua e task.

### 1.2 Le espressioni regolari

#### L'idea

Cercare «woodchuck» con una ricerca esatta perde *Woodchuck* e *woodchucks*. Un'espressione regolare (*regular expression*, regex) descrive in una riga un *insieme* di stringhe e un motore di ricerca la confronta con il testo: `[Ww]oodchucks?` trova tutte e quattro le varianti.

## Espressione regolare

Un **linguaggio formale per specificare stringhe di testo**: un pattern che individua le stringhe che vi corrispondono (*match*). [slide Intro p. 23] Le regole essenziali sono nella tabella 1.1.

| pattern          | significato                                  | esempio                                              |
|------------------|----------------------------------------------|------------------------------------------------------|
| [wW]oodchuck     | classe di caratteri; [0-9] è un intervallo   | Woodchuck, woodchuck [slide Intro p. 24]             |
| [^Ss]            | negazione, se ^ è il primo in [ ]            | [^A-Z] = non una maiuscola [slide Intro p. 25]       |
| a b c            | disgiunzione di alternative, = [abc]         | groundhog woodchuck [slide Intro p. 26]              |
| colou?r          | carattere precedente opzionale               | color, colour [slide Intro p. 27]                    |
| oo*h!, o+h!      | 0 o più / 1 o più ripetizioni (Kleene *, +)  | oh!, ooh!, ooh! [slide Intro p. 27]                  |
| beg.n            | un carattere qualsiasi                       | begin, begun, beg3n [slide Intro p. 27]              |
| \. \* \\\; \n \t | escape: carattere alla lettera; a capo, tab  | \.\$ = punto a fine riga [slide Intro pp. 28–29]     |
| ^[A-Z], .\$      | ancore: inizio e fine riga (posizioni)       | ^[A-Z] trova «Palo Alto» [slide Intro p. 29]         |
| ([0-9]+) ... \1  | cattura in un registro; (? :a b) non cattura | s/([0-9]+)/<\1>/: 35 → <35> [slide Intro pp. 35, 37] |
| (?=p), (?!p)     | lookahead: p segue (o no), non consumato     | ^(?!Volcano)[A-Za-z]+ [slide Intro p. 38]            |

Tabella 1.1: Le regole essenziali delle espressioni regolari.

**Precisione, sostituzioni, capture group.** Per la parola *the*, [tT]he trova anche *other* e *theology*; [^a-zA-Z][tT]he[^a-zA-Z] richiede un non-lettera prima e dopo: un compromesso fra falsi match e varianti perse. [slide Intro pp. 30–33] In *sed* la sostituzione è *s/regexp/sostituzione/*, con i flag *g* (tutte le occorrenze) e *i* (ignora maiuscole/minuscole); le parentesi tonde *catturano* il match in un registro riusabile con \1, \2, ... (tabella 1.1). [slide Intro pp. 34–36]

**ELIZA.** ELIZA (Weizenbaum, MIT, 1966) simula uno psicoterapeuta rogersiano senza alcuna semantica: la frase dell'utente, in maiuscolo [dal libro, p. 33; non nelle slide], passa per una cascata di sostituzioni (con il flag *i* [non nelle fonti del corso]) e la prima che corrisponde produce la risposta, riuscendo con \1 un pezzo della frase: *s/. \* I'M (depressed|sad) .\*/I AM SORRY TO HEAR YOU ARE \1/* dà, per «He says I'm depressed much of the time», I AM SORRY TO HEAR YOU ARE DEPRESSED; *s/. \* all .\*/IN WHAT WAY?/*. [slide Intro pp. 22, 39–41]

### 1.3 Parole, token e type; i corpora

## L'idea

Quante parole ci sono in «She sang until she could sing no more.»? Dipende da che cosa si conta: le occorrenze o le parole distinte; se il punto conta; se *She* e *she*, o *sang* e *sing* (stesso lemma), sono la stessa parola. Prima di contare bisogna fissare queste convenzioni.

## Token, type, lemma, wordform

**Word token:** ogni occorrenza di una parola nel testo. **Word type:** elemento distinto del vocabolario. [slide Normalizzazione p. 5] **Lemma:** insieme di forme con la stessa radice, part of speech (categoria grammaticale) e significato di massima, cioè la forma del vocabolario (*cat* e *cats*: stesso lemma); **wordform:** la forma flessa come appare nel testo (*cats*). [slide Normalizzazione p. 4]  $N$  = numero di token del corpus,  $V$  = vocabolario,  $|V|$  la sua dimensione. [slide Normalizzazione p. 6] *Esempio:* la frase sopra ha 9 token (8 senza il punto); 8 type senza distinguere le maiuscole, 7 se si contano i lemmi (*sang* = *sing*), 6 se si toglie anche la punteggiatura. [slide Normalizzazione p. 5]

**Legge di Heaps (o di Herdan)**

$$|V| = k N^\beta, \quad \beta \approx 0.67\text{--}0.75,$$

con  $k, \beta$  costanti positive: il vocabolario cresce poco più della radice quadrata del numero di token [slide Normalizzazione p. 6] e non smette mai di crescere: compariranno sempre parole mai viste (*unknown words*) [dal libro, p. 15; non nelle slide]; da qui la tokenization subword (sez. 1.4.2).

| corpus         | token $N$ | type $ V $ |
|----------------|-----------|------------|
| Shakespeare    | 884 000   | 31 000     |
| Switchboard    | 2.4 mln   | 20 000     |
| COCA           | 440 mln   | 2 mln      |
| Google N-grams | $10^{12}$ | 13+ mln    |

[slide Normalizzazione p. 6]

**Corpora.** Un *corpus* (raccolta strutturata di testi) varia per lingua (7 097 nel mondo), varietà, *code switching* (più lingue insieme) e genere, e va documentato: motivazione, situazione, raccolta, annotazione. [slide Normalizzazione pp. 3, 7–9]

## 1.4 La pipeline di normalizzazione

Ogni task di NLP richiede una normalizzazione sistematica del testo in tre passi (figura 1.1): **word tokenization**, **format normalization**, **sentence segmentation**. [slide Normalizzazione p. 10]

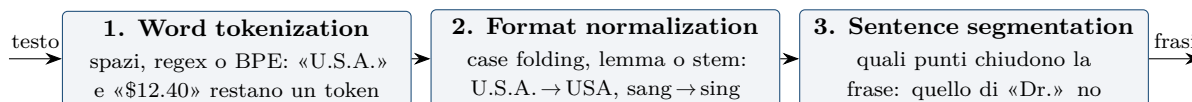


Figura 1.1: La pipeline di normalizzazione. Fonte: slide Normalizzazione p. 10; esempi dalle pp. 5, 17, 38, 46.

### 1.4.1 Tokenization

Per le scritture latina, cirillica, greca e araba il metodo più semplice è separare sugli spazi (in Unix: `tr, sort | uniq -c`). [slide Normalizzazione pp. 11–15] Ma lo spazio non basta: abbreviazioni (m.p.h., Ph.D., AT&T, cap'n), prezzi e date (\$45.55, 01/02/06), URL e hashtag (#nlproc), clitici (*we're*, *j'ai*, *l'honneur*), espressioni di più parole (*New York*, *rock 'n' roll*). [slide Normalizzazione p. 16] Si usano allora tokenizer a regole, regex che elencano i casi: in NLTK `regex_tokenize`, `word_tokenize` e `TreebankWordTokenizer`. [slide Normalizzazione pp. 17–18]

**Lingue senza spazi.** Cinese, giapponese e thai non separano le parole. In cinese una parola ha in media 2.4 caratteri (*hanzi*, ciascuno un morfema), ma non c'è accordo su cosa sia una parola: «Yao Ming reaches the finals» si segmenta in 3 parole o in 5, oppure in 7 caratteri; in pratica il token è il singolo carattere. [slide Normalizzazione pp. 19–25]

**Tokenization data-driven e subword.** Invece di regole fissate a priori decidono i dati, e i token possono essere parole intere o *parti* di parole (*subword*). Tre algoritmi, **Byte-Pair Encoding**, **Unigram Language Modeling** e **WordPiece**, tutti con un **token learner**, che dal corpus di addestramento induce un vocabolario, e un **token segmenter**, che tokenizza una frase nuova secondo quel vocabolario. [slide Normalizzazione pp. 26–27] WordPiece (usato da BERT) è una variante di BPE a vocabolario fissato, che a ogni passo fonde non la coppia più frequente ma quella che aumenta di più la probabilità del corpus secondo un language model [dal libro, pp. 272–273; non nelle slide]; Unigram LM (o SentencePiece) parte invece da un vocabolario enorme (caratteri e sequenze frequenti) e pota i token meno utili fino alla dimensione voluta. [dal libro, p. 273; non nelle slide]

### 1.4.2 Byte-Pair Encoding (BPE)

#### L'idea

Si parte dai singoli caratteri e si fondono (*merge*) ripetutamente le due unità adiacenti più frequenti. Le parole frequenti finiscono per diventare un token unico; le parole rare restano spezzate in pezzi che spesso coincidono con morfemi, le più piccole unità di significato (*unlikelyst* = *un-* + *likely* + *-est* [slide Normalizzazione p. 37]). Non esistono più parole sconosciute: nel caso peggiore una parola si scrive carattere per carattere.

#### BPE token learner e token segmenter

1. *Preprocessing*: separa sugli spazi e spezza ogni parola in caratteri più il simbolo di fine parola `_`; i merge non attraversano i confini di parola. [slide Normalizzazione p. 30]
2.  $V \leftarrow$  caratteri distinti del corpus; poi ripeti  $k$  volte: trova la coppia di token adiacenti  $(t_L, t_R)$  più frequente, aggiungi a  $V$  il token  $t_{NEW} = t_L t_R$  e sostituisci ogni  $t_L t_R$  adiacenti nel corpus con  $t_{NEW}$ . [slide Normalizzazione pp. 28–29]
3. Restituisci  $V$  (caratteri più  $k$  token nuovi) e la lista ordinata dei merge.
4. *Token segmenter*: applica alla frase nuova i merge nell'ordine in cui sono stati appresi; le frequenze del testo di test non contano. [slide Normalizzazione p. 36]

**L'esempio svolto** (figura 1.2). Corpus di 18 parole; le frequenze sommano i conteggi: al merge 1 *e* e *r* compare in *newer* (6) e *wider* (3), cioè 9 volte (nei pareggi, come qui con *r* `_`, vince la coppia incontrata per prima [non nelle fonti del corso]). Dopo 8 merge il vocabolario ha 19 token; *low* e *newer* sono token unici, *lowest* resta *low* e *s t* `_`. [slide Normalizzazione pp. 31–35] Il *segmenter* riapplica i merge nello stesso ordine: «newer» → *newer* `_`; «lower», mai visto, → *low* *er* `_`. [slide Normalizzazione p. 36]

| corpus iniziale                                                          | dopo 8 merge    | n. | merge      | dove         | freq. | nuovo token |
|--------------------------------------------------------------------------|-----------------|----|------------|--------------|-------|-------------|
| 5 l o w _                                                                | 5 l o w _       | 1  | (e, r)     | newer, wider | 9     | er          |
| 2 l o w e s t _                                                          | 2 l o w e s t _ | 2  | (er, _)    | newer, wider | 9     | er_         |
| 6 n e w e r _                                                            | 6 n e w e r _   | 3  | (n, e)     | newer, new   | 8     | ne          |
| 3 w i d e r _                                                            | 3 w i d e r _   | 4  | (ne, w)    | newer, new   | 8     | new         |
| 2 n e w _                                                                | 2 n e w _       | 5  | (l, o)     | low, lowest  | 7     | lo          |
| vocabolario iniziale (11): <code>_</code> , d, e, i, l, n, o, r, s, t, w |                 | 6  | (lo, w)    | low, lowest  | 7     | low         |
|                                                                          |                 | 7  | (new, er_) | newer        | 6     | newer_      |
|                                                                          |                 | 8  | (low, _)   | low          | 5     | low_        |

Figura 1.2: Esempio BPE delle slide: corpus, otto merge con frequenza ricalcolata, corpus finale. Fonte: slide Normalizzazione pp. 31–36.

### 1.4.3 Word normalization: case folding, lemmatization, stemming

**Word normalization** significa scegliere una forma standard: U.S.A. → USA, *am*, *is*, *are* alla forma base. [slide Normalizzazione p. 38] Il **case folding** porta tutto in minuscolo: conviene in information retrieval (ricerca di documenti), salvo le maiuscole a metà frase (*General Motors*, *Fed* vs *fed*); in sentiment analysis, machine translation e information extraction (estrarre dati strutturati dal testo) la maiuscola porta significato (*US* vs *us*) e si conserva. [slide Normalizzazione p. 39]

#### Lemmatization e stemming

**Lemmatization**: rappresentare ogni parola con il suo *lemma*, la forma del dizionario: *am*, *are*, *is* → *be*; *car*, *cars*, *car's*, *cars'* → *car*; «He is reading detective stories» → «He be read detective story». Richiede il **morphological parsing**: scomporre la parola in morfemi,

lo *stem* (porta il significato) e gli *affissi* (funzione grammaticale):  $cats \rightarrow cat + s$ . [slide Normalizzazione pp. 40–41] In una lingua agglutinante come il turco [dal libro, p. 17; non nelle slide] una parola vale una frase: *Uygarlastiramadiklarimizdanmissinizcasina*, «(comportandosi) come se foste fra quelli che non siamo riusciti a civilizzare», è *uygar* 'civilizzato' + *las* 'diventare' + *tir* 'causativo' + *ama* 'non potere' + *dik* 'passato' + ... + *siniz* '2<sup>a</sup> pl.' + *casina* 'come se': 11 morfemi. [slide Normalizzazione p. 44] **Stemming**: riduzione grossolana che taglia gli affissi con regole, senza dizionario. Il **Porter stemmer** è una cascata di regole di riscrittura applicate in sequenza, dove ogni passata alimenta la successiva: *things* → *thing*, *Billy* → *Billi*, *accurate* → *accur*, *single* → *singl*, *This* → *Thi*. Veloce, ma lo stem spesso non è una parola. [slide Normalizzazione pp. 42–43]

#### 1.4.4 Sentence segmentation

Il punto è ambiguo: chiude una frase, oppure un'abbreviazione (*Inc.*, *Dr.*, *Ph.D.*, *etc.*), oppure sta in un numero (.02%, 4.3). Soluzione: prima si tokenizza, poi si classifica ogni punto con regole o machine learning, aiutandosi con un dizionario di abbreviazioni. [slide Normalizzazione p. 46] In NLTK: `sent_tokenize` e `PunktSentenceTokenizer`, addestrabile su un corpus. [slide Normalizzazione p. 47]

##### «Descrivere la tokenization subword e l'algoritmo BPE»

1. *Problema e idea*: le parole non bastano come unità (lingue senza spazi; vocabolario senza limite per Heaps, quindi unknown word): token appresi dai dati, parole intere o pezzi, con token learner e token segmenter.
2. *Learner* (sez. 1.4.2): caratteri più simbolo di fine parola;  $k$  volte: coppia adiacente più frequente → nuovo token, sostituito nel corpus. *Segmenter*: i merge nell'ordine appreso, ignorando le frequenze del test.
3. *Esempio* (figura 1.2): corpus low/lowest/newer/wider/new; merge (e, r), (er, \_), (n, e), (ne, w), ..., (low, \_); «newer» → token unico, «lower» → low + er\_.
4. *Proprietà e alternative*: parole frequenti = token singoli, pezzi  $\approx$  morfemi, nessuna unknown word; WordPiece e Unigram LM (sez. 1.4.1); BPE e Unigram LM sono le tokenization dei language model moderni [dal libro, p. 21; non nelle slide] (capp. 7, 11).

##### «Descrivere la pipeline di normalizzazione; confrontare stemming e lemmatization»

1. *Tre passi*: tokenization, word normalization, sentence segmentation (figura 1.1); token e type con l'esempio: 9 token, 8 type.
2. *Tokenization*: spazi; i casi difficili (abbreviazioni, numeri, clitici, URL, espressioni di più parole); regole; lingue senza spazi; subword con BPE.
3. *Normalizzazione*: case folding sì per IR, no quando la maiuscola distingue (US/us). *Lemmatization*: lemma del dizionario via morphological parsing (is → be, cars → car): precisa, ma richiede la morfologia della lingua (il turco). *Stemming*: suffissi tagliati con regole in cascata (Porter): veloce, ma grossolano (accurate → accur, this → thi).
4. *Sentence segmentation*: punto ambiguo (abbreviazioni, numeri): tokenizzare, poi classificare i punti con dizionari di abbreviazioni. *Limite*: la scelta dipende dal task.

##### Errore tipico da evitare

- Token  $\neq$  type:  $N$  conta le occorrenze,  $|V|$  le parole distinte («9 token, 8 type»); i conteggi dipendono dalle convenzioni (punteggiatura, maiuscole, lemmi).
- In BPE *er* e *er\_* sono token diversi (il simbolo di fine parola conta), i merge non scavalcano i confini di parola e il segmenter non riconta nulla.
- Stemming non è lemmatization: lo stem (*accur*) può non essere una parola, il lemma sì.

## Capitolo 2

### Modelli di linguaggio n-gram

Un *language model* (modello di linguaggio, LM) assegna una probabilità a una frase e serve a scegliere fra alternative: in machine translation «high winds tonight» deve battere «large winds tonight»; nello spell correction (correzione ortografica) «about fifteen *minutes* from» deve battere «about fifteen *minuets* from»; nello speech recognition (riconoscimento del parlato) «I saw a van» deve battere «eyes awe of an» [slide N-gram p. 3]. Il capitolo: stima, perplexity, smoothing (domanda 1 del programma).

#### 2.1 Dalla chain rule all'assunzione di Markov

##### L'idea

Non si può stimare la probabilità di una frase contando quante volte compare in un corpus: quasi ogni frase è nuova. La *chain rule* la spezza in un prodotto di probabilità condizionate «parola data tutta la storia precedente», ma anche le storie lunghe sono quasi tutte nuove. L'*assunzione di Markov* taglia la storia alle ultime  $k - 1$  parole: i pezzi da stimare sono gli *n-gram*, sequenze di  $n$  parole consecutive («I want» è un bigram, «I want to» un trigram), che si ripetono abbastanza da poterli contare.

##### Language model e chain rule

Un *language model* calcola  $P(w_{1:n}) = P(w_1 w_2 \dots w_n)$  oppure  $P(w_n | w_{1:n-1})$ , la probabilità della parola successiva [slide N-gram p. 5] (le slide scrivono  $W_i$ ). I due compiti sono equivalenti:  $P(w_n | w_{1:n-1}) = P(w_{1:n})/P(w_{1:n-1})$ , e viceversa la chain rule [slide N-gram pp. 4, 6]

$$P(w_{1:n}) = P(w_1) P(w_2 | w_1) P(w_3 | w_{1:2}) \dots P(w_n | w_{1:n-1}) = \prod_{i=1}^n P(w_i | w_{1:i-1}).$$

Ma contare la storia intera («its water is so transparent that») seguita dalla parola non funziona: le frasi possibili sono troppe [slide N-gram pp. 8–9].

##### Assunzione di Markov, modelli n-gram

Ogni fattore si approssima con la probabilità condizionata alle sole ultime  $k - 1$  parole [slide N-gram pp. 10–11]:

$$P(w_i | w_{1:i-1}) \approx P(w_i | w_{i-k+1:i-1}), \quad P(w_{1:n}) \approx \prod_{i=1}^n P(w_i | w_{i-k+1:i-1}).$$

Con  $k = 1$  si ha il modello a *unigram*,  $P(w_{1:n}) \approx \prod_i P(w_i)$  [slide N-gram p. 12]; con  $k = 2$  i *bigram*,  $P(w_i | w_{i-1})$  [slide N-gram p. 13]; poi trigram, 4-gram, 5-gram. Il linguaggio ha dipendenze a lunga distanza che gli n-gram non vedono («The computer which I had just put into the machine room on the fifth floor *crashed*»), ma spesso bastano [slide N-gram p. 14].

##### Stima di massima verosimiglianza (MLE)

La *maximum likelihood estimate* di un parametro è il valore che massimizza la probabilità del training set dato il modello [slide N-gram p. 16]; per i bigram è la frequenza relativa, con

$C(\cdot)$  il conteggio nel corpus [slide N-gram p. 17]:

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1} w_i)}{C(w_{i-1})}.$$

«Bagel» 400 volte su un milione di parole: MLE  $P(\text{bagel}) = 0.0004$ , la stima che rende più probabili proprio quelle 400 occorrenze [slide N-gram p. 16]. **Esempio** [slide N-gram p. 18]: sul corpus «<s> I am Sam </s>», «<s> Sam I am </s>», «<s> I do not like green eggs and ham </s>» (<s> e </s> delimitano la frase; </s> fa sommare a 1 le probabilità delle frasi [dal libro, p. 49; non nelle slide]) si ha  $P(I | <s>) = 2/3$ ,  $P(\text{Sam} | <s>) = 1/3$ ,  $P(\text{am} | I) = 2/3$ ,  $P(</s> | \text{Sam}) = 1/2$ ,  $P(\text{Sam} | \text{am}) = 1/2$ ,  $P(\text{do} | I) = 1/3$ .

## 2.2 L'esempio del Berkeley Restaurant Project

Il corpus sono domande a un sistema di dialogo sui ristoranti di Berkeley («can you tell me about any good cantonese restaurants close by») [slide N-gram p. 19]; la figura 2.1 dà i conteggi dei bigram per otto parole [slide N-gram p. 20].

|         | i    | want | to   | eat | chinese | food | lunch | spend |
|---------|------|------|------|-----|---------|------|-------|-------|
| i       | 5    | 827  | 0    | 9   | 0       | 0    | 0     | 2     |
| want    | 2    | 0    | 608  | 1   | 6       | 6    | 5     | 1     |
| to      | 2    | 0    | 4    | 686 | 2       | 0    | 6     | 211   |
| eat     | 0    | 0    | 2    | 0   | 16      | 2    | 42    | 0     |
| chinese | 1    | 0    | 0    | 0   | 0       | 82   | 1     | 0     |
| food    | 15   | 0    | 15   | 0   | 1       | 4    | 0     | 0     |
| lunch   | 2    | 0    | 0    | 0   | 0       | 1    | 0     | 0     |
| spend   | 1    | 0    | 1    | 0   | 0       | 0    | 0     | 0     |
| $C(w)$  | 2533 | 927  | 2417 | 746 | 158     | 1093 | 341   | 278   |

Figura 2.1: Conteggi dei bigram  $C(w_{i-1} w_i)$  del Berkeley Restaurant Project (riga  $w_{i-1}$ , colonna  $w_i$ ; zeri in grigio) e degli unigram  $C(w)$ . Fonte: slide N-gram pp. 20–21

**Probabilità di una frase** [slide N-gram p. 22]. Con  $P(i | <s>) = 0.25$ ,  $P(\text{english} | \text{want}) = 0.0011$ ,  $P(\text{food} | \text{english}) = 0.5$  e  $P(</s> | \text{food}) = 0.68$  [dal libro, p. 51; non nelle slide], la probabilità di «<s> i want english food </s>» è

$$\begin{aligned} &P(i | <s>) P(\text{want} | i) P(\text{english} | \text{want}) P(\text{food} | \text{english}) P(</s> | \text{food}) \\ &= 0.25 \cdot 0.33 \cdot 0.0011 \cdot 0.5 \cdot 0.68 = 0.000031. \end{aligned}$$

In pratica si lavora nello spazio dei logaritmi ( $\log(p_1 p_2 p_3) = \log p_1 + \log p_2 + \log p_3$ ) per evitare l'*underflow* del prodotto di molte probabilità piccole, e sommare è più veloce [slide N-gram p. 24].

## 2.3 Valutare un modello di linguaggio: la perplexity

### L'idea

Un buon modello dà probabilità più alta alle frasi reali o frequenti che a quelle sgrammaticate o rare; i parametri si stimano su un *training set*, la qualità si misura su un *test set* mai usato [slide N-gram p. 29]. La valutazione *estrinseca* (dentro un'applicazione, confrontando l'accuratezza) è la migliore ma costa giorni o settimane [slide N-gram pp. 30–31]. Quella *intrinseca* usa la *perplexity* (quanto il modello è «sorpreso» dal testo di test). L'intuizione è lo *Shannon game* («I always order pizza with cheese and \_\_\_\_»): il modello migliore dà la probabilità più alta alla parola che compare davvero; gli unigram, senza contesto, sono

Dividendo ogni riga per il conteggio dell'unigram si ottengono le probabilità [slide N-gram p. 21]:  $P(\text{want} | i) = 827/2533 = 0.33$ ,  $P(\text{to} | \text{want}) = 608/927 = 0.66$ ,  $P(\text{eat} | \text{to}) = 686/2417 = 0.28$ ,  $P(\text{chinese} | \text{want}) = 6/927 = 0.0065$ ; metà delle 64 celle è zero. I numeri catturano conoscenze diverse [slide N-gram p. 23] [dal libro, p. 51; non nelle slide]: grammatica ( $P(\text{to} | \text{want}) = 0.66$ ,  $P(\text{want} | \text{spend}) = 0$ ), il compito ( $P(i | <s>) = 0.25$ : le domande cominciano spesso con «I») e persino la cultura ( $P(\text{chinese} | \text{want}) > P(\text{english} | \text{want})$ ).

pessimi [slide N-gram p. 32].

### Perplexity

La perplexity di un modello su un testo di test  $W = w_{1:n}$  è l'inverso della probabilità del testo, normalizzato per il numero di parole [slide N-gram p. 33]:

$$PP(W) = P(w_{1:n})^{-\frac{1}{n}} = \sqrt[n]{\frac{1}{P(w_{1:n})}} = \sqrt[n]{\prod_{i=1}^n \frac{1}{P(w_i | w_{1:i-1})}}$$

e per i bigram si usa  $P(w_i | w_{i-1})$  in ogni fattore. Minimizzare la perplexity equivale a massimizzare la probabilità del test set: **perplexity più bassa = modello migliore**.

**Perplexity come *branching factor*.** È il fattore di diramazione medio (pesato): il numero di scelte equiprobabili fra cui il modello esita a ogni passo [slide N-gram p. 34]: una frase di  $n$  cifre casuali, con  $P(w_i) = 1/10$  per ogni cifra, ha  $PP = ((1/10)^n)^{-1/n} = 10$  [slide N-gram p. 35]; riconoscere uno fra 30 000 nomi ha perplexity 30 000. **Esempio di Josh Goodman** [slide N-gram p. 36]: un centralino (*call-routing*) riceve 120K chiamate: tre reparti (R&D, Sales, Technical Support), ciascuno in 1/4 delle chiamate, e 30 000 nomi diversi, ognuno una volta sola (1/120K). Le 120K probabilità, 90K da 1/4 e 30K da 1/120K, elevate a  $-1/120K$  si semplificano in quattro fattori:  $PP = (\frac{1}{4} \cdot \frac{1}{4} \cdot \frac{1}{4} \cdot \frac{1}{120\,000})^{-1/4} = 52.6$ . Sul Wall Street Journal (training 38 milioni di parole, test 1.5 milioni) la perplexity scende da **962** (unigram) a **170** (bigram) a **109** (trigram) [slide N-gram p. 37].

## 2.4 Generalizzazione, zeri e smoothing

Campionando dal modello [slide N-gram p. 39] si vede che cosa ha imparato: addestrato su Shakespeare, un unigram produce parole a caso e un 4-gram versi interi [slide N-gram p. 40], perché Shakespeare ha  $N = 884\,647$  token e  $|V| = 29\,066$  parole ma solo 300 000 tipi di bigram su  $|V|^2 = 844$  milioni possibili: il 99.96% dei bigram possibili non è mai stato visto, e i 4-gram sembrano Shakespeare perché *sono* Shakespeare [slide N-gram p. 41]. È l'*overfitting* (il modello si adatta troppo al training set): gli n-gram funzionano solo se il test set somiglia al training set [slide N-gram p. 42]. Il caso tipico sono gli **zeri**: n-gram assenti nel training ma presenti nel test, come «denied the offer» quando il training ha solo «denied the allegations / reports / claims / request», quindi  $P(\text{offer} | \text{denied the}) = 0$  [slide N-gram p. 43]; un solo bigram a probabilità zero azzerava la probabilità dell'intero test set e la perplexity non si può calcolare (divisione per zero) [slide N-gram p. 44].

### L'idea dello smoothing

Con statistiche sparse «rubiamo» massa agli eventi visti per darla a quelli mai visti: è lo *smoothing* (smussamento delle stime) [slide N-gram p. 46]. Per  $P(w | \text{denied the})$  i conteggi sono 3 allegations, 2 reports, 1 claims, 1 request (7 in totale): la MLE dà  $3/7, 2/7, 1/7, 1/7$  e zero al resto; dopo lo smoothing  $2.5/7, 1.5/7, 0.5/7, 0.5/7$  e  $2/7$  per tutte le altre parole (attack, man, outcome, ...).

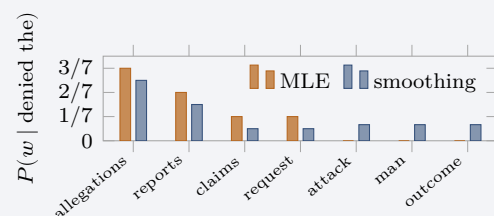


Figura 2.2: Lo smoothing sposta massa dagli eventi visti a quelli mai visti. Fonte: slide N-gram p. 46

### 2.4.1 Add-one (Laplace)

#### Add-one smoothing

Si fa finta di aver visto ogni parola una volta in più:  $+1$  a tutti i conteggi, e il denominatore cresce di  $|V|$  (uno per ogni parola del vocabolario), così le probabilità sommano ancora a 1 [slide N-gram p. 47]. I *conteggi ricostruiti*  $C^*$  (probabilità add-one per il denominatore originale  $C(w_{i-1})$ ) mostrano quanto lo smoothing ha alterato i conteggi veri [slide N-gram p. 50]:

$$P_{\text{Add-1}}(w_i | w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + |V|}, \quad C_{\text{Add-1}}^*(w_{i-1}w_i) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + |V|} \cdot C(w_{i-1}).$$

|         | i    | want  | to    | eat   | chin. | food | lunch | spend |
|---------|------|-------|-------|-------|-------|------|-------|-------|
| i       | 3.8  | 527   | 0.64  | 6.4   | 0.64  | 0.64 | 0.64  | 1.9   |
| want    | 1.2  | 0.39  | 238   | 0.78  | 2.7   | 2.7  | 2.3   | 0.78  |
| to      | 1.9  | 0.63  | 3.1   | 430   | 1.9   | 0.63 | 4.4   | 133   |
| eat     | 0.34 | 0.34  | 1     | 0.34  | 5.8   | 1    | 15    | 0.34  |
| chinese | 0.2  | 0.099 | 0.099 | 0.099 | 0.099 | 8.2  | 0.2   | 0.099 |
| food    | 6.9  | 0.43  | 6.9   | 0.43  | 0.86  | 2.2  | 0.43  | 0.43  |
| lunch   | 0.57 | 0.19  | 0.19  | 0.19  | 0.19  | 0.38 | 0.19  | 0.19  |
| spend   | 0.32 | 0.16  | 0.32  | 0.16  | 0.16  | 0.16 | 0.16  | 0.16  |

Figura 2.3: Conteggi ricostruiti  $C_{\text{Add-1}}^*$  del Berkeley Restaurant Project con  $|V| = 1446$  [dal libro, p. 60; non nelle slide]; in grigio le celle che erano zero (i conteggi  $C + 1$  sono quelli della figura 2.1 più uno).

Fonte: slide N-gram pp. 48, 50

Il confronto con la figura 2.1 mostra quanto add-one sia pesante [slide N-gram p. 51]:  $P(\text{want} | i)$  passa da 0.33 a  $828/(2533 + 1446) = 0.21$ , «want to» da 608 a 238 e «chinese food» da 82 a 8.2, mentre  $P(\text{food} | \text{to})$  passa da 0 a  $1/3863 = 0.00026$ . Poiché  $|V| = 1446$  è grande rispetto a molti  $C(w_{i-1})$  (158 per «chinese»), ogni riga regala troppa massa alle parole mai viste: add-one è uno «strumento grossolano», non si usa per i LM ma va bene per la classificazione di testi (capitolo 3), dove gli zeri sono pochi [slide N-gram p. 52].

### 2.4.2 Backoff e interpolazione

A volte aiuta usare *meno* contesto, per i contesti su cui si sa poco. Il *backoff* usa il trigram se c'è buona evidenza, altrimenti il bigram, altrimenti l'unigram; l'*interpolazione* li mescola sempre tutti e tre, e funziona meglio [slide N-gram p. 53].

#### Interpolazione lineare

$$\hat{P}(w_i | w_{i-2}w_{i-1}) = \lambda_1 P(w_i | w_{i-2}w_{i-1}) + \lambda_2 P(w_i | w_{i-1}) + \lambda_3 P(w_i), \quad \sum_j \lambda_j = 1.$$

I  $\lambda$  sommano a 1 perché  $\hat{P}$  resti una distribuzione di probabilità; possono dipendere dal contesto,  $\lambda_j(w_{i-2}w_{i-1})$ : più il contesto è frequente, più ci si fida dell'ordine alto [slide N-gram p. 54].

**Come si scelgono i  $\lambda$ .** Con un corpus *held-out*, separato sia dal training sia dal test: si fissano le probabilità degli n-gram stimate sul training set e si cercano i  $\lambda$  che massimizzano la probabilità dell'*held-out*, per esempio con Expectation-Maximization (si fissa  $\lambda_1$  e si cerca  $\lambda_2$  con  $\lambda_3 = 1 - \lambda_1 - \lambda_2$ , poi si fissa  $\lambda_2$  e si cerca  $\lambda_1$ , fino a convergenza) [slide N-gram p. 55].

**Esempio con i  $\lambda$**  (esercizio del libro [dal libro, p. 68; non nelle slide]): il corpus «I am Sam» della sezione 2.1 con «ham»  $\rightarrow$  «Sam» e la prima frase ripetuta due volte; contando  $\langle s \rangle$  e  $\langle /s \rangle$  come token,  $N = 25$ ,  $C(\text{Sam}) = 4$ ,  $C(\text{am}) = 3$ ,  $C(\text{am Sam}) = 2$ . Con  $\lambda_1 = \lambda_2 = 1/2$  fra bigram e unigram,  $\hat{P}(\text{Sam} | \text{am}) = \frac{1}{2} \cdot \frac{2}{3} + \frac{1}{2} \cdot \frac{4}{25} = 0.413$ ; per il bigram mai visto «I Sam»,  $\hat{P}(\text{Sam} | \text{I}) = \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 0.16 = 0.08 \neq 0$ .

**Parole sconosciute.** Con vocabolario fissato il compito è *closed vocabulary*; con parole fuori vocabolario (OOV, *open vocabulary*) si usa un token  $\langle \text{UNK} \rangle$ : ogni parola del training fuori da un

lessico  $L$  fissato diventa  $\langle \text{UNK} \rangle$ , se ne stimano le probabilità come per una parola normale e in decodifica le usa ogni parola mai vista [slide N-gram p. 56].

### 2.4.3 n-gram web-scale e stupid backoff

#### Stupid backoff (Brants et al. 2007)

Per il corpus web di Google servono *pruning* (potatura: solo n-gram con conteggio sopra una soglia, via i singleton di ordine alto; o per entropia: via gli n-gram che cambiano meno la distribuzione) ed efficienza: trie (alberi di prefissi: n-gram con lo stesso inizio condividono il cammino), filtri di Bloom (insiemi compressi con falsi positivi: LM approssimati), parole come indici (codifica di Huffman: codici più corti alle parole frequenti, per stare in due byte), probabilità quantizzate a 4–8 bit (non float a 8 byte) [slide N-gram p. 57] (glosse: [non nelle fonti del corso]). Smoothing senza sconti: frequenze relative e, se l'n-gram non è stato visto, si scende di ordine moltiplicando per 0.4; i valori  $S$  sono punteggi (*score*; le slide scrivono  $P_S$ ), non probabilità [slide N-gram pp. 58–59]:

$$S(w_i | w_{i-k+1:i-1}) = \begin{cases} \frac{C(w_{i-k+1:i})}{C(w_{i-k+1:i-1})} & \text{se } C(w_{i-k+1:i}) > 0 \\ 0.4 \cdot S(w_i | w_{i-k+2:i-1}) & \text{altrimenti} \end{cases} \quad S(w_i) = \frac{C(w_i)}{N}.$$

### 2.4.4 Absolute discounting

Quanto togliere a ogni conteggio? Church e Gale (1991), su 22 milioni di parole di AP Newswire divise in training e held-out, hanno misurato per ogni conteggio  $c$  nel training il conteggio medio nell'held-out [slide N-gram p. 60]:

|                            |           |       |      |      |      |      |      |      |      |      |
|----------------------------|-----------|-------|------|------|------|------|------|------|------|------|
| conteggio nel training $c$ | 0         | 1     | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    |
| conteggio nell'held-out    | 0.0000270 | 0.448 | 1.25 | 2.24 | 3.23 | 4.21 | 5.23 | 6.21 | 7.21 | 8.26 |

#### Absolute discounting con interpolazione

Da  $c = 2$  in poi  $c^* \approx c - 0.75$ : lo sconto è una costante. Si sottrae allora  $d = 0.75$  (magari diverso per  $c = 1$  e  $c = 2$ ) a ogni conteggio e la massa risparmiata si ridistribuisce con l'unigram, pesato da  $\lambda(w_{i-1})$  [slide N-gram p. 61]:

$$P_{AD}(w_i | w_{i-1}) = \frac{C(w_{i-1}w_i) - d}{C(w_{i-1})} + \lambda(w_{i-1}) P(w_i).$$

### 2.4.5 Kneser-Ney

#### L'idea: quanto una parola è una «nuova continuazione»

«I can't see without my reading —»: come unigram «Kong» è più frequente di «glasses», ma segue sempre «Hong». L'unigram serve proprio quando il bigram non è stato visto: la domanda giusta non è «quanto è probabile  $w$ » ma «quanto è probabile che  $w$  sia una continuazione nuova», cioè quante parole diverse la precedono [slide N-gram p. 62].

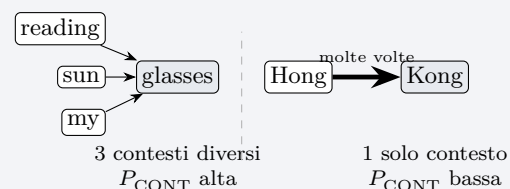


Figura 2.4: Kneser-Ney conta i contesti distinti che precedono una parola. Fonte: idea da slide N-gram p. 62

### Kneser-Ney smoothing

La *probabilità di continuazione* è il numero di parole diverse che precedono  $w_i$ , normalizzato dal numero totale di tipi di bigram [slide N-gram pp. 63–64]:

$$P_{\text{CONT}}(w_i) = \frac{|\{w_{i-1} : C(w_{i-1}w_i) > 0\}|}{|\{(w'_{i-1}, w'_i) : C(w'_{i-1}w'_i) > 0\}|} = \frac{|\{w_{i-1} : C(w_{i-1}w_i) > 0\}|}{\sum_{w'_i} |\{w'_{i-1} : C(w'_{i-1}w'_i) > 0\}|}.$$

Kneser-Ney è *absolute discounting* con  $P_{\text{CONT}}$  al posto dell'unigram [slide N-gram p. 65]:

$$P_{\text{KN}}(w_i | w_{i-1}) = \frac{\max(C(w_{i-1}w_i) - d, 0)}{C(w_{i-1})} + \lambda(w_{i-1}) P_{\text{CONT}}(w_i),$$

$$\lambda(w_{i-1}) = \frac{d}{C(w_{i-1})} \cdot |\{w : C(w_{i-1}w) > 0\}|,$$

dove  $\lambda$  è la massa scontata: lo sconto normalizzato  $d/C(w_{i-1})$  per il numero di tipi di parola che seguono  $w_{i-1}$  (le volte in cui lo sconto è stato applicato). **Formulazione ricorsiva** [slide N-gram p. 66], con  $C_{\text{KN}}$  = conteggio vero all'ordine più alto e *continuation count* (contesti distinti) agli ordini inferiori:

$$P_{\text{KN}}(w_i | w_{i-k+1:i-1}) = \frac{\max(C_{\text{KN}}(w_{i-k+1:i}) - d, 0)}{C_{\text{KN}}(w_{i-k+1:i-1})} + \lambda(w_{i-k+1:i-1}) P_{\text{KN}}(w_i | w_{i-k+2:i-1}).$$

#### 2.4.6 Advanced language modeling

Tre estensioni degli n-gram [slide N-gram p. 68]. I *modelli discriminativi* scelgono i pesi degli n-gram per migliorare il compito finale, non per adattarsi al training set; i modelli *parsing-based* usano la struttura sintattica della frase (capitolo 9); i *caching model* (modelli con cache) sfruttano il fatto che una parola usata di recente tende a ricomparire, mescolando il trigram con la frequenza di  $w_i$  nella storia recente (*history*):

$$P_{\text{CACHE}}(w_i | \text{history}) = \lambda P(w_i | w_{i-2}w_{i-1}) + (1 - \lambda) \frac{C(w_i \in \text{history})}{|\text{history}|};$$

nello speech recognition, però, i caching model si sono rivelati pessimi.

#### Domanda 1 del programma: le tecniche di smoothing, con vantaggi e limiti

1. **Il problema.** Un LM n-gram stima  $P(w_i | w_{i-1}) = C(w_{i-1}w_i)/C(w_{i-1})$  (MLE); gli n-gram mai visti nel training ma presenti nel test (*zeri*) azzerano la probabilità del test set e rendono la perplexity incalcolabile (Shakespeare: 99.96% dei bigram mai visti).
2. **L'idea.** Lo smoothing (o discounting) toglie massa di probabilità agli eventi visti e la dà a quelli mai visti (figura 2.2); i metodi differiscono per quanto togliere e a chi darlo.
3. **Add-one (Laplace).**  $(C + 1)/(C(w_{i-1}) + |V|)$ . Pro: semplice, niente zeri. Contro: troppo pesante («want to» da 608 a 238) perché  $|V|$  è grande rispetto a  $C(w_{i-1})$ ; non si usa per i LM, va bene per la classificazione di testi [slide N-gram p. 67].
4. **Backoff e interpolazione.** Meno contesto quando l'evidenza è poca: il backoff scende di ordine, l'interpolazione  $\hat{P} = \lambda_1 P(w_i | w_{i-2}w_{i-1}) + \lambda_2 P(w_i | w_{i-1}) + \lambda_3 P(w_i)$  mescola sempre, con  $\lambda$  stimati su un corpus held-out (EM). Pro: batte il backoff. Contro: serve l'held-out.
5. **Stupid backoff** (web-scale): frequenze relative, altrimenti  $0.4S$  dell'ordine inferiore. Pro: nessuno sconto, scala a miliardi di n-gram. Contro: dà punteggi, non probabilità.
6. **Absolute discounting.** Church-Gale: nell'held-out i conteggi valgono circa  $c - 0.75$ ; quindi  $P_{\text{AD}} = (C - d)/C(w_{i-1}) + \lambda(w_{i-1})P(w_i)$ . Pro: sconto motivato dai dati. Contro:

l'unigram è la stima sbagliata (Hong Kong).

7. **Kneser-Ney.**  $P_{KN} = \max(C - d, 0)/C(w_{i-1}) + \lambda(w_{i-1}) P_{\text{CONT}}(w_i)$ : come l'absolute discounting, ma al posto dell'unigram c'è  $P_{\text{CONT}}(w)$ , la frazione dei tipi di bigram che  $w$  completa («Kong» ne completa uno solo);  $\lambda(w_{i-1})$  è la massa scontata. È il metodo più usato (*extended interpolated Kneser-Ney* [slide N-gram p. 67]). Contro: più complesso.
8. **Chiusura.** La perplexity misura il guadagno; add-one torna in Naive Bayes (capitolo 3); i LM neurali (capitolo 6) superano gli zeri con rappresentazioni dense delle parole.

#### Errore tipico da evitare

- **Perplexity alta è peggio:** è l'inverso della probabilità per parola; si confronta solo a parità di vocabolario e su un test set mai usato in training [dal libro, p. 55; non nelle slide].
- Con add-one il denominatore diventa  $C(w_{i-1}) + |V|$ , non  $C(w_{i-1}) + 1$ ; i  $\lambda$  si stimano sull'held-out, non sul test set; stupid backoff dà punteggi, non probabilità; Kneser-Ney conta i *contesti distinti*, non le occorrenze.

## Capitolo 3

### Classificazione di testi: Naive Bayes e valutazione

Dato un testo, scegliere una classe: Naive Bayes e le misure di qualità (sez. 3.3).

#### 3.1 Il problema della classificazione

##### L'idea

Molti compiti di NLP si riducono a *mettere un'etichetta su un testo*: questa email è spam? chi ha scritto questo saggio (nel 1963 Mosteller e Wallace attribuirono con metodi bayesiani i *Federalist papers* contesi fra Hamilton e Madison)? di che cosa parla un articolo medico? questa recensione è positiva o negativa? L'ultimo caso è la **sentiment analysis**. Scherer distingue cinque tipi di stati affettivi: *emotion* (emozione: breve, legata a un evento; *angry*), *mood* (umore: diffuso, debole, duraturo; *gloomy*), *interpersonal stance* (posizione affettiva verso un'altra persona in un'interazione; *cold*), *attitude* (disposizione duratura verso oggetti o persone; *liking*, *hating*) e *personality trait* (tratto stabile; *jealous*). La sentiment analysis «is the detection of attitudes»; qui, nel caso più semplice: il testo è positivo o negativo? [slide Naive Bayes pp. 3–12]

##### Text classification; regole a mano e apprendimento supervisionato

**Classificazione di testi.** Input: un documento  $d$  e un insieme fisso di classi  $C = \{c_1, \dots, c_J\}$ ; output: una classe predetta  $c \in C$ . [slide Naive Bayes p. 13] **Regole scritte a mano** (hand-coded rules) su parole o altre feature, per esempio *spam se l'indirizzo è in black-list OR («dollars» AND «you have been selected»)*: accuratezza anche alta se un esperto le raffina, ma costruirle e mantenerle costa. [slide Naive Bayes p. 14] **Apprendimento supervisionato** (supervised machine learning): dati  $C$  e un training set di  $m$  documenti etichettati a mano  $(d_1, c_1), \dots, (d_m, c_m)$ , si apprende un classificatore  $\gamma: d \mapsto c$ ; va bene qualunque classificatore: Naive Bayes, regressione logistica, reti neurali,  $k$ -nearest neighbors. [slide Naive Bayes pp. 15–16]

**Bag of words.** Naive Bayes, classificatore «ingenuo» (*naive*) basato sulla regola di Bayes, rappresenta il documento nel modo più semplice possibile: il **bag of words** («sacchetto di parole»). Si dimentica l'ordine delle parole e si tiene solo quante volte compare ciascuna (figura 3.1: *it* 6, *I* 5, *the* 4, ...); il classificatore  $\gamma$  riceve questi conteggi. [slide Naive Bayes pp. 17–19]

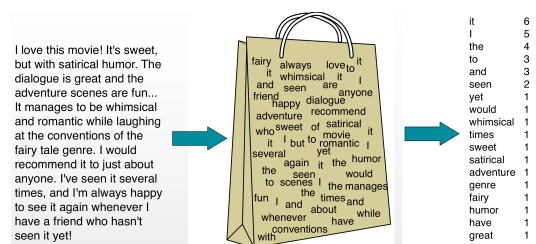


Figura 3.1: Bag of words: dal testo ai conteggi delle parole.

Fonte: slide Naive Bayes p. 18

#### 3.2 Naive Bayes

##### L'idea

Vorremmo la classe più probabile dato il documento,  $P(c | d)$ , ma nessuno ci dice quanto vale. La regola di Bayes la ribalta in due pezzi che si possono *contare* su un corpus etichettato: quanto è frequente la classe (il **prior**, probabilità a priori  $P(c)$ ) e quanto è probabile vedere quelle parole in un documento di quella classe (la **likelihood**, verosimiglianza  $P(d | c)$ ).

Il passo «ingenuo» è fingere che le parole siano indipendenti fra loro una volta nota la classe: la verosimiglianza del documento diventa il prodotto delle probabilità delle singole parole, e ogni fattore si stima contando. L'assunzione è falsa, ma il classificatore funziona sorprendentemente bene.

### Il classificatore Naive Bayes

da saper dimostrare

Regola di Bayes per un documento  $d$  e una classe  $c$ :  $P(c | d) = \frac{P(d|c)P(c)}{P(d)}$ . [slide Naive Bayes p. 20] La classe scelta è quella *MAP* (maximum a posteriori, la più probabile dato  $d$ ):

$$c_{\text{MAP}} = \operatorname{argmax}_{c \in C} P(c | d) = \operatorname{argmax}_{c \in C} \frac{P(d | c) P(c)}{P(d)} = \operatorname{argmax}_{c \in C} P(d | c) P(c),$$

perché il denominatore  $P(d)$  è lo stesso per tutte le classi e si elimina. [slide Naive Bayes p. 21] Rappresentato  $d$  con le sue feature  $x_1, \dots, x_n$  (le parole),  $c_{\text{MAP}} = \operatorname{argmax}_c P(x_1, \dots, x_n | c) P(c)$ : likelihood per prior. [slide Naive Bayes p. 22] Il prior si stima con le frequenze relative delle classi nel corpus; la verosimiglianza congiunta ha  $O(|X|^n \cdot |C|)$  parametri ( $|X|$  valori possibili per feature), stimabili solo con un numero enorme di esempi. [slide Naive Bayes p. 23]

**Le due assunzioni del multinomial Naive Bayes:** (1) *bag of words*: la posizione delle parole non conta; (2) *indipendenza condizionale*: le  $P(x_i | c)$  sono indipendenti data la classe,  $P(x_1, \dots, x_n | c) = P(x_1 | c) \cdot P(x_2 | c) \cdots P(x_n | c)$ . [slide Naive Bayes p. 24] Quindi, con positions = tutte le posizioni di parola del documento di test,

$$c_{\text{NB}} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j).$$

[slide Naive Bayes pp. 25–26] **Spazio dei logaritmi** (come per gli n-gram, sez. 2.2). Moltiplicare molte probabilità piccole provoca *underflow* in virgola mobile; poiché  $\log(ab) = \log a + \log b$ , si sommano i logaritmi:

$$c_{\text{NB}} = \operatorname{argmax}_{c_j \in C} \left[ \log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i | c_j) \right].$$

Il logaritmo non cambia la classifica delle classi (la più probabile ha anche il log più alto) e il punteggio è una somma di pesi, funzione lineare degli input: Naive Bayes è un **classificatore lineare**. [slide Naive Bayes pp. 27–28]

### Apprendimento: stima dei parametri e add-1 smoothing

Dal training set si estrae il vocabolario  $V$ . Il **prior** conta i documenti; per la **likelihood** si concatenano tutti i documenti di classe  $c_j$  in un solo «mega-documento»  $\text{Text}_j$  e si conta la frazione delle occorrenze di  $w_i$  fra tutte le sue parole (stima di massima verosimiglianza, MLE, come per gli n-gram: def. 2.1):

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{\text{total}}}, \quad \hat{P}(w_i | c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)}.$$

[slide Naive Bayes pp. 29–30] **Il problema della MLE.** Se nel training nessun documento positivo contiene *fantastic*,  $\hat{P}(\text{fantastic} | +) = 0$  e il prodotto si azzerava qualunque siano le altre parole: uno zero non si recupera con altra evidenza. [slide Naive Bayes p. 31] **Laplace (add-1) smoothing** (lo stesso add-one dei language model, def. 2.4.1):

$$\hat{P}(w_i | c) = \frac{\text{count}(w_i, c) + 1}{(\sum_{w \in V} \text{count}(w, c) + |V|)}, \quad \text{in generale} \quad P(w_k | c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |V|},$$

con  $n_k$  occorrenze di  $w_k$  in  $Text_j$  e  $n$  parole totali di  $Text_j$ . [slide Naive Bayes pp. 32–33] **Parole sconosciute** (nel test ma non in  $V$ ): si eliminano dal documento di test, come se non ci fossero; un modello delle parole sconosciute non aiuta, perché sapere quale classe ne ha di più non è utile. **Stop word** (parole frequentissime come *the*, *a*: le 10 o 50 più frequenti del training): alcuni sistemi le tolgono, ma di solito non migliora, e in pratica Naive Bayes usa tutte le parole. [slide Naive Bayes pp. 34–35]

### 3.2.1 Esempio svolto: sentiment con add-1

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | –   | just plain boring                     |
|          | –   | entirely predictable and lacks energy |
|          | –   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable <i>with</i> no fun        |

1. **Prior:**  $P(-) = 3/5$ ,  $P(+) = 2/5$  (3 e 2 documenti su 5).
2. *with* non è nel vocabolario di training: si elimina.
3. **Likelihood add-1:** i documenti – hanno 14 parole, i + ne hanno 9,  $|V| = 20$ :  $P(\text{predictable} | -) = \frac{1+1}{14+20}$ ,  $P(\text{predictable} | +) = \frac{0+1}{9+20}$ ;  $P(\text{no} | -) = \frac{1+1}{14+20}$ ,  $P(\text{no} | +) = \frac{0+1}{9+20}$ ;  $P(\text{fun} | -) = \frac{0+1}{14+20}$ ,  $P(\text{fun} | +) = \frac{1+1}{9+20}$ .
4. **Punteggi** ( $S$  è la frase di test):  $P(-)P(S | -) = \frac{3}{5} \cdot \frac{2 \cdot 2 \cdot 1}{34^3} = 6.11 \cdot 10^{-5}$ ,  $P(+)P(S | +) = \frac{2}{5} \cdot \frac{1 \cdot 1 \cdot 2}{29^3} = 3.28 \cdot 10^{-5}$ : vince la classe negativa. In spazio dei logaritmi (naturali):  $-9.70$  contro  $-10.3$ , stessa classifica.

[slide Naive Bayes pp. 36–37]

### 3.2.2 Varianti per il sentiment e osservazioni

**Binary NB.** Per il sentiment conta più la *presenza* di una parola che la sua frequenza: che compaia *fantastic* dice molto, che compaia 5 volte poco di più. Nel **binary multinomial Naive Bayes** si tolgono i duplicati *dentro ciascun documento* (di training e di test) e poi si usano le stesse formule; i conteggi per classe possono restare maggiori di 1: nell'esempio delle slide, nella classe + *great* passa da 3 a 2 e *and* da 2 a 1, mentre *scenes* resta 2 nella classe – perché compare in due documenti.

[slide Naive Bayes pp. 39–43] Diverso è il *Bernoulli naive Bayes* [slide Naive Bayes p. 39]: ogni parola di  $V$  è una variabile binaria presente/assente e la likelihood include anche, per le parole *assenti*, il fattore  $1 - P(w | c)$ ; il binary NB resta multinomiale e conta solo le parole presenti. [non nelle fonti del corso]

**Negazione.** In «I really don't like this movie» la negazione rovescia il senso di *like* (e *don't dismiss this film* è quasi positivo). Una *baseline* (metodo semplice di riferimento): si aggiunge il prefisso **NOT\_** a ogni parola fra la negazione e la punteggiatura seguente: *didn't like this movie* , *but I* diventa *didn't NOT like NOT this NOT movie but I*. [slide Naive Bayes pp. 44–45]

**Lessici.** Se i dati etichettati sono pochi si usano liste di parole pre-costruite (*lexicon*: MPQA, 2718 parole positive e 4912 negative; General Inquirer, 1915 e 2291) con una feature «la parola è nel lessico positivo (negativo)»: valgono meno di tutte le parole, ma aiutano se il training è scarso o non rappresentativo. [slide Naive Bayes pp. 46–49] **Altri compiti:** spam, con feature come «menziona milioni di dollari» o oggetto tutto maiuscolo; language identification, con n-gram di caratteri, addestrando su molte varietà di ogni lingua. [slide Naive Bayes pp. 50–51]

**Naive Bayes come modello di linguaggio.** Se le feature sono tutte e sole le parole del testo, ogni classe è un modello di linguaggio unigram (cap. 2):  $P(s | c) = \prod P(\text{word} | c)$ . Nell'esempio delle slide  $P(\text{I love this fun film} | \text{pos}) = 0.1 \cdot 0.1 \cdot 0.01 \cdot 0.05 \cdot 0.1 = 5 \cdot 10^{-7}$ , contro  $0.2 \cdot 0.001 \cdot 0.01 \cdot 0.005 \cdot 0.1 = 10^{-9}$  per neg: senza prior, è un confronto di massima verosimiglianza.

[slide Naive Bayes pp. 53–55] **«Not so naive»:** velocissimo, poca memoria, funziona con pochissimi dati, robusto alle feature irrilevanti (si cancellano a vicenda), ottimo con molte feature ugualmente importanti, dove i *decision tree* (alberi di decisione) soffrono di *fragmentation* (frammentazione: a ogni ramificazione restano meno dati [non nelle fonti del corso]), specie se i dati sono pochi; ottimale se l'indipendenza vale davvero; una buona baseline, ma altri classificatori (cap. 4) sono più accurati.

[slide Naive Bayes p. 52]

### 3.3 Valutare un classificatore

#### Perché non basta l'accuracy

Immagina un rilevatore di tweet che parlano della «Delicious Pie Company» (classe positiva) contro tutti gli altri. Su un milione di tweet, 100 parlano della ditta. Un classificatore stupido che risponde sempre «non parla di torte» indovina 999 900 volte su un milione: accuracy 99.99%, ma è inutile, perché non trova nessuno dei 100 tweet cercati (recall = 0). Quando la classe interessante è rara l'accuracy inganna; precision e recall mettono al centro i *veri positivi*, le cose che stiamo cercando. [slide Naive Bayes pp. 56–61]

#### Matrice di confusione, precision, recall, misura F

|                      |                 | gold standard labels |                |
|----------------------|-----------------|----------------------|----------------|
|                      |                 | gold positive        | gold negative  |
| system output labels | system positive | true positive        | false positive |
|                      | system negative | false negative       | true negative  |

precision =  $\frac{tp}{tp+fp}$

recall =  $\frac{tp}{tp+fn}$  accuracy =  $\frac{tp+tn}{tp+fp+tn+fn}$

Figura 3.2: La matrice di confusione 2 × 2: righe = sistema, colonne = gold.

Fonte: slide Naive Bayes p. 57

**Matrice di confusione:** righe = etichette del sistema, colonne = etichette *gold* (le corrette, umane). [slide Naive Bayes p. 57] **Precision** =  $tp/(tp+fp)$ : quanti, fra gli elementi etichettati positivi dal sistema, lo sono davvero. **Recall** =  $tp/(tp+fn)$ : quanti, fra i positivi presenti nell'input, il sistema ha trovato. [slide Naive Bayes pp. 59–60] **Misura F:** combina  $P$  e  $R$  in un numero solo,  $F_\beta = \frac{(\beta^2+1)PR}{\beta^2 P + R}$ ; quasi sempre  $\beta = 1$ :  $F_1 = \frac{2PR}{P+R}$ . [slide Naive Bayes p. 62] È la media armonica di  $P$  e  $R$ , vicina al minore dei due. [dal libro, p. 92; non nelle slide]

**Più classi.** Classificazione *binaria* (due classi: spam/non spam), *multi-class* (una sola di  $N$  classi: positive, negative, neutral) e *multi-label* (zero, una o più classi: «What a surprise, I'm happy to see you!» esprime Joy e Surprise). Nel multi-label ogni classe è una classificazione binaria a sé e le etichette di un testo sono un vettore binario (one-hot encoding) con una posizione per classe (1 se il testo appartiene alla classe, 0 altrimenti). [slide Naive Bayes p. 63] Con più classi la matrice di confusione ha una riga e una colonna per classe e precision e recall si calcolano classe per classe (figura 3.3). [slide Naive Bayes p. 64]

|               |        | gold labels |        |      |
|---------------|--------|-------------|--------|------|
|               |        | urgent      | normal | spam |
| system output | urgent | 8           | 10     | 1    |
|               | normal | 5           | 60     | 50   |
|               | spam   | 3           | 30     | 200  |

precision<sub>u</sub> =  $\frac{8}{8+10+1}$

precision<sub>n</sub> =  $\frac{60}{5+60+50}$

precision<sub>s</sub> =  $\frac{200}{3+30+200}$

recall<sub>u</sub> =  $\frac{8}{8+5+3}$  recall<sub>n</sub> =  $\frac{60}{10+60+30}$  recall<sub>s</sub> =  $\frac{200}{1+50+200}$

Figura 3.3: Matrice di confusione a tre classi (righe: sistema; colonne: gold).

Fonte: slide Naive Bayes p. 64

| classe | $P$             | $R$             | $F_1$ |
|--------|-----------------|-----------------|-------|
| urgent | 8/19 = 0.421    | 8/16 = 0.500    | 0.457 |
| normal | 60/115 = 0.522  | 60/100 = 0.600  | 0.558 |
| spam   | 200/233 = 0.858 | 200/251 = 0.797 | 0.826 |

**Macroaveraging:** si calcola la misura per ogni classe e si fa la media: macro precision =  $\frac{0.421+0.522+0.858}{3} = 0.600$ . **Microaveraging:** si raccolgono le decisioni di tutte le classi in una sola matrice 2 × 2 (pooled: tp = 268, fp = 99, fn = 99, tn = 635) e da quella si calcolano  $P$  e  $R$ : micro precision =  $\frac{268}{268+99} = 0.730$  (qui uguale all'accuracy, 268/367). [slide Naive Bayes pp. 65–66] La micro è dominata dalla classe frequente (spam); la macro riflette anche le classi piccole. [dal libro, p. 93; non nelle slide]

**Devset e cross-validation.** Si addestra sul training set, si regolano le scelte (tuning) sul *development test set* (devset) e si riporta il risultato sul test set: evita l'overfitting al test set («tuning to the test set») e dà una stima più prudente. Paradosso: si vorrebbero più dati possibile

sia per il training sia per il dev; la **cross-validation** divide il training in  $k$  parti (*fold*; 10 nelle slide), a turno una fa da dev e le altre da training, e si mettono insieme (pool) i risultati sui dev; il test set resta a parte. [slide Naive Bayes pp. 68–69]

**Un classificatore è davvero migliore? Test di ipotesi e bootstrap.** Dati i sistemi A e B e una metrica  $M$ ,  $\delta(x) = M(A, x) - M(B, x)$  è l'*effect size* sul test set  $x$ ;  $\delta(x) > 0$  potrebbe essere un caso di quel test set. Ipotesi nulla  $H_0$ : «A non è meglio di B»; il **p-value**  $P(\delta(X) \geq \delta(x) \mid H_0)$  è la probabilità di vedere una differenza almeno così grande se  $H_0$  fosse vera; se sta sotto una soglia (0.05 o 0.01) si rifiuta  $H_0$  e il risultato è *statisticamente significativo*. In NLP si usano test non parametrici basati sul campionamento: l'*approximate randomization* (randomizzazione approssimata: si scambiano a caso, esempio per esempio, le risposte di A e B e si conta quante volte la differenza raggiunge quella osservata [non nelle fonti del corso]) e il **bootstrap**, in forma *paired*: A e B sugli stessi esempi. Bootstrap: dal test set si estraggono *con reinserimento*  $b$  campioni di  $n$  elementi e su ognuno si calcola  $\delta(x^{(i)})$ ; poiché i campioni vengono da  $x$ , già sbilanciato di  $\delta(x)$  a favore di A, si conta quante volte  $\delta(x^{(i)})$  supera il proprio valore atteso,  $\delta(x)$ , di almeno  $\delta(x)$ :  $p\text{-value} = \frac{1}{b} \sum_{i=1}^b \mathbb{1}(\delta(x^{(i)}) \geq 2\delta(x))$ . Esempio: 10 documenti, accuracy di A 0.70 e di B 0.50,  $\delta(x) = 0.20$ ; se solo 47 campioni su 10 000 danno  $\delta(x^{(i)}) \geq 2\delta(x)$ , il p-value è  $0.0047 < 0.01$ : A è meglio di B. [slide Naive Bayes pp. 70–81]

**Danni.** I classificatori di sentiment danno sentiment più negativo a frasi con nomi afroamericani e i rilevatori di tossicità segnalano frasi innocue che menzionano identità (donne, ciechi, gay): le cause sono nei dati di training, nelle etichette, nei lessici e nel modello; ogni modello va documentato con una *model card* (dati, uso previsto, prestazioni per gruppo). [slide Naive Bayes pp. 82–85]

#### «Descrivere il classificatore Naive Bayes per la classificazione di testi»

1. *Compito*: dato un documento  $d$  e le classi  $C$ , predire  $c \in C$ ; apprendimento supervisionato da un training set etichettato (sez. 3.1). *Rappresentazione*: bag of words.
2. *Modello*: regola di Bayes,  $c_{\text{MAP}} = \operatorname{argmax}_c P(d \mid c) P(c)$  (il denominatore non dipende da  $c$ ); assunzioni di bag of words e di indipendenza condizionale data la classe, da cui  $c_{\text{NB}} = \operatorname{argmax}_c P(c) \prod_i P(x_i \mid c)$ ; calcolo in spazio log (underflow): classificatore lineare.
3. *Apprendimento*: prior = frazione di documenti della classe; likelihood = frequenza relativa della parola nel mega-documento della classe, con add-1 contro gli zeri; parole sconosciute ignorate.
4. *Esempio*: il worked example (sez. 3.2.1):  $P(-) = 3/5$ , add-1 con  $|V| = 20$ , punteggi  $6.11 \cdot 10^{-5}$  contro  $3.28 \cdot 10^{-5}$ .
5. *Varianti, limiti, collegamenti*: binary NB per il sentiment, prefisso NOT\_ per la negazione, lessici; l'indipendenza è falsa ma il modello è veloce, robusto e una buona baseline; è un classificatore *generativo* (un modello unigram per classe), mentre la regressione logistica (cap. 4) è *discriminativa* e di solito più accurata [slide Regressione logistica p. 4].

#### «Come si valuta un classificatore? Confrontare accuracy, precision, recall e $F_1$ »

1. *Matrice di confusione*: sistema contro gold; tp, fp, fn, tn (fig. 3.2).  $\text{Accuracy} = (\text{tp} + \text{tn})/N$ : inganna con classi sbilanciate (99.99% per il classificatore che non trova nulla).
2.  $\text{Precision} = \text{tp}/(\text{tp} + \text{fp})$ : quanto è pulito ciò che il sistema restituisce;  $\text{recall} = \text{tp}/(\text{tp} + \text{fn})$ : quanta parte dei positivi veri trova;  $F_1 = 2PR/(P + R)$  le combina (media armonica).
3. *Più classi*:  $P$  e  $R$  per classe; macro average (media fra le classi, pesa anche quelle rare) e micro average (una matrice pooled, dominata dalla classe frequente): nell'esempio 0.600 contro 0.730 (fig. 3.3).
4. *Protocollo*: training/devset/test, cross-validation a  $k$  fold; significatività della differenza fra due sistemi con il paired bootstrap e il p-value.

**Errore tipico da evitare**

- Nel denominatore dell'add-1 si somma  $|V|$ , il numero di *tipi* del vocabolario di training (20 nell'esempio), non il numero di token; e le parole del test assenti da  $V$  si *eliminano*, non si smussano.
- Il prior conta *documenti*, la likelihood conta *parole*:  $P(-) = 3/5$  perché 3 documenti su 5, non perché 14 parole su 23. Binary NB taglia i conteggi a 1 *per documento*: nella tabella di classe un conteggio può restare 2.
- Precision = «di quelli che ho detto positivi, quanti lo sono», recall = «dei positivi veri, quanti ho trovato»: non si scambiano, e accuracy alta non implica recall alto.

## Capitolo 4

### Classificazione di testo con la regressione logistica

La *logistic regression* (regressione logistica) è il classificatore supervisionato di riferimento (*baseline*) del machine learning e il fondamento delle reti neurali (capitolo 6) [slide Regressione logistica p. 3].

#### 4.1 Classificatori generativi e discriminativi

##### L'idea

Per distinguere foto di gatti da foto di cani ci sono due strade. Un classificatore *generativo* impara com'è fatto un gatto (baffi, orecchie, occhi) e com'è fatto un cane, poi chiede a entrambi i modelli «quanto è probabile questa immagine?» e sceglie quello che la spiega meglio. Un classificatore *discriminativo* non modella le immagini: cerca solo ciò che *separa* le classi («i cani hanno il collare!») e ignora il resto. Naive Bayes (capitolo 3) è generativo, la regressione logistica è discriminativa [slide Regressione logistica pp. 4–7].

##### Generativo e discriminativo

Per assegnare a un documento  $d$  una classe  $\hat{c}$  dell'insieme  $C$  [slide Regressione logistica p. 15]:

$$\text{Naive Bayes: } \hat{c} = \operatorname{argmax}_{c \in C} \overbrace{P(d | c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}} \quad \text{Regressione logistica: } \hat{c} = \operatorname{argmax}_{c \in C} \underbrace{P(c | d)}_{\text{posterior}}$$

Il generativo stima quanto è probabile *osservare* il documento data la classe (likelihood, verosimiglianza: potrebbe «generarlo») per la probabilità a priori della classe (prior); il discriminativo stima direttamente la probabilità della classe dato il documento (posterior), ciò che serve per decidere.

**Modello, parametri, loss.** Un modello supervisionato è una funzione  $f$  che da un input  $x$  predice  $\hat{y} = f(x)$ . Si fissa una famiglia di funzioni  $\mathcal{F}$  e sul training set  $\{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$  si sceglie la  $f$  che minimizza la *loss* (perdita) media, cioè la divergenza fra predizione e valore vero:  $f = \operatorname{argmin}_{f' \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f'(x^{(i)}), y^{(i)})$ . Esempio: lo stipendio in funzione degli anni di esperienza con  $\mathcal{F} = \{f \mid f(x) = a + bx\}$  e  $\mathcal{L}(y, y') = |y - y'|$  (regressione lineare);  $a$  e  $b$  sono i *parametri*, raccolti nel vettore  $\theta$ . La scelta di  $\mathcal{F}$  è delicata [slide Regressione logistica p. 9]: se è troppo ampia, trovare la  $f$  migliore può essere impraticabile e quella migliore sul training set può non generalizzare ad altri dati (*overfitting*, sezione 4.5); se è troppo stretta, la  $f$  migliore non approssima bene il training set; la si sceglie quindi in modo che l'ottimizzazione sia fattibile e la si restringe ancora guardando come i risultati generalizzano (tuning). Si scelgono funzioni *derivabili*, ottimizzabili con lo stochastic gradient descent: per questo il modello restituisce una distribuzione di probabilità sulle classi e non direttamente l'etichetta (una funzione che salta da una classe all'altra non è derivabile); i dati si dividono in training, validation (development) e test set [slide Regressione logistica pp. 8–14]. Un classificatore probabilistico ha quattro componenti: le *feature* (le caratteristiche dell'input)  $[x_1, \dots, x_n]$ , la funzione di classificazione che calcola  $p(c | \mathbf{x})$  (sigmoide o softmax), la loss (cross-entropy) e l'algoritmo di ottimizzazione (SGD); e due fasi: *training* dei pesi  $\mathbf{w}$ ,  $b$  e *test*, in cui si calcola  $p(y | \mathbf{x})$  e si sceglie l'etichetta più probabile [slide Regressione logistica pp. 16–17].

## 4.2 Il modello: feature, punteggio, sigmoide, decisione

### L'idea

Ogni feature  $x_i$  del testo riceve un peso  $w_i$  che dice quanto conta e in che direzione: +10 se la recensione contiene *awesome*, -10 per *abysmal*, -2 per *mediocre*. Sommando le feature pesate, più una costante  $b$  detta *bias*, si ottiene un punteggio  $z$ : alto vuol dire  $y = 1$ , basso  $y = 0$ . Ma  $z$  è un numero qualsiasi, non una probabilità: lo si schiaccia in  $(0, 1)$  con la *sigmoide* e il risultato si tratta come  $P(y = 1 \mid \mathbf{x})$  [slide Regression logistica pp. 21–27].

### Regressione logistica binaria

Input: feature  $\mathbf{x} = [x_1, \dots, x_n]$ ; parametri: pesi  $\mathbf{w} = [w_1, \dots, w_n]$  e bias  $b$  [slide Regression logistica pp. 22–28]:

$$z = \left( \sum_{i=1}^n w_i x_i \right) + b = \mathbf{w} \cdot \mathbf{x} + b, \quad \sigma(z) = \frac{1}{1 + e^{-z}},$$

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x} + b), \quad P(y = 0 \mid \mathbf{x}) = 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{e^{-z}}{1 + e^{-z}}.$$

Proprietà della sigmoide [slide Regression logistica p. 29]:  $\sigma(-z) = 1 - \sigma(z)$  e  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ . *Decisione* [slide Regression logistica pp. 30, 32]:  $\hat{y} = 1$  se  $P(y = 1 \mid \mathbf{x}) > 0.5$ , altrimenti  $\hat{y} = 0$ ; la soglia 0.5 è la *decision boundary* e, poiché  $\sigma(0) = 0.5$ , equivale a  $\mathbf{w} \cdot \mathbf{x} + b > 0$ .

**Esempio: il sentiment di una recensione.** La recensione della figura 4.2 è positiva ( $y = 1$ ) o negativa ( $y = 0$ )? Sei feature:  $x_1$  = parole del lessico positivo (3),  $x_2$  = del lessico negativo (2),  $x_3 = 1$  se c'è «no» (1),  $x_4$  = pronomi di prima e seconda persona (3),  $x_5 = 1$  se c'è «!» (0),  $x_6 = \ln(\text{numero di parole}) = \ln 66 = 4.19$ . Pesi e bias, supposti già appresi:  $\mathbf{w} = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$ ,  $b = 0.1$  [slide Regression logistica pp. 33–36]. Allora  $z = 2.5 \cdot 3 - 5.0 \cdot 2 - 1.2 \cdot 1 + 0.5 \cdot 3 + 2.0 \cdot 0 + 0.7 \cdot 4.19 + 0.1 = 0.833$ , quindi  $P(y = 1 \mid \mathbf{x}) = \sigma(0.833) = 0.70$  e  $P(y = 0 \mid \mathbf{x}) = 0.30$ : poiché  $0.70 > 0.5$  (cioè  $z > 0$ ) la recensione è classificata positiva.

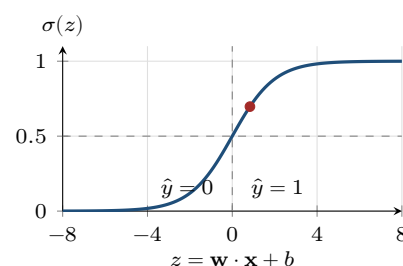


Figura 4.1: La sigmoide:  $\sigma(0) = 0.5$  è la decision boundary; il punto rosso è l'esempio,  $\sigma(0.833) = 0.70$ .

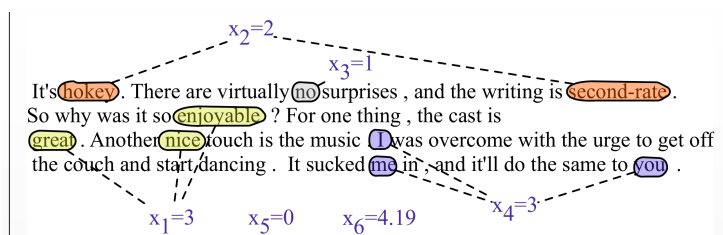


Figura 4.2: La recensione dell'esempio con i valori delle sei feature.

Fonte: slide Regression logistica p. 34

**Un secondo esempio di feature: la disambiguazione del punto.** Le feature valgono per qualunque compito, per esempio decidere se un punto chiude la frase. Per la parola  $w_i$  che porta il punto:  $x_1 = 1$  se è minuscola ( $\text{Case}(w_i) = \text{Lower}$ ),  $x_2 = 1$  se sta in un dizionario di acronimi ( $w_i \in \text{AcronymDict}$ ),  $x_3 = 1$  se  $w_i = \text{«St.»}$  e la parola prima è maiuscola ( $\text{Case}(w_{i-1}) = \text{Cap}$ ): in «The house at 465 Main St. is new.» il punto di «St.» ( $x_3 = 1$ ) non chiude la frase, l'ultimo ( $x_1 = 1$ ) sì [slide Regression logistica p. 37].

### 4.3 Apprendimento: la cross-entropy loss

#### L'idea

Da dove vengono i pesi? Nel training conosciamo l'etichetta vera  $y$  (0 o 1) di ogni esempio, mentre il modello produce una stima  $\hat{y}$  fra 0 e 1: vogliamo i  $\mathbf{w}$  e  $b$  che rendono  $\hat{y}$  il più vicino possibile a  $y$  sul training set. Servono una misura della distanza fra  $\hat{y}$  e  $y$  (la loss) e un algoritmo che muova i pesi per ridurla (SGD, sezione 4.4). La loss premia i pesi che danno *alta probabilità all'etichetta giusta*: piccola se il modello è vicino al vero, grande se è confuso [slide Regressione logistica pp. 39–42, 46].

#### Cross-entropy loss

da saper dimostrare

Per un'osservazione  $x$  con etichetta  $y \in \{0, 1\}$  e stima  $\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$  [slide Regressione logistica p. 45]:

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -\log p(y | \mathbf{x}) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1 - y) \log(1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b))]. \end{aligned}$$

*Derivazione* [slide Regressione logistica pp. 42–45]: con due soli esiti, la probabilità che il classificatore assegna all'etichetta vera è  $p(y | \mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y}$  (vale  $\hat{y}$  se  $y = 1$ ,  $1 - \hat{y}$  se  $y = 0$ ); vogliamo massimizzarla, e il logaritmo non sposta i massimi:  $\log p(y | \mathbf{x}) = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$ . Cambiando segno si ottiene qualcosa da *minimizzare*: la cross-entropy loss, così chiamata perché è la formula della cross-entropy fra  $y$  e  $\hat{y}$ . È una stima di massima verosimiglianza condizionata (massimizza la log-probabilità delle etichette vere; logaritmo naturale). Sul training set di  $m$  esempi si minimizza la loss media, con  $\hat{y} = f(x; \theta)$  e  $\theta = (w_1, \dots, w_n, b)$ :  $\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$  [slide Regressione logistica p. 50].

**Esempio.** Sulla recensione della sezione 4.2 il modello dà  $\hat{y} = 0.70$ . Se  $y = 1$  resta solo il primo termine:  $L_{\text{CE}} = -\ln 0.70 = 0.36$ ; se fosse  $y = 0$  resta il secondo:  $L_{\text{CE}} = -\ln(1 - 0.70) = -\ln 0.30 = 1.2$ . La loss è più grande quando il modello sbaglia, come volevamo [slide Regressione logistica pp. 47–49].

### 4.4 Gradient descent e stochastic gradient descent

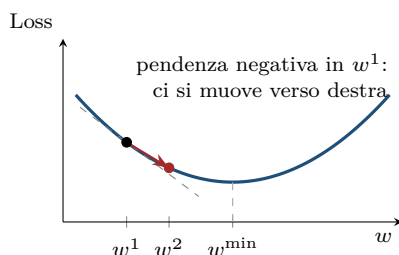


Figura 4.3: Gradient descent con un solo peso: da  $w^1$  si va contro la pendenza ( $w \leftarrow w - \eta \frac{dL}{dw}$ ) verso  $w^{\min}$ .

#### L'idea

Come si arriva in fondo a un canyon nella nebbia? Ci si guarda intorno a 360°, si trova la direzione in cui la discesa è più ripida, si fa un passo e si ripete. La loss è il canyon, i parametri sono la posizione. Con un solo peso  $w$ : se la pendenza (la derivata) della loss in  $w$  è negativa, la loss scende aumentando  $w$ : ci si muove nel verso *opposto* alla pendenza (figura 4.3). In più dimensioni la pendenza è il *gradiente*, che punta dove la loss cresce di più. La loss della regressione logistica è *convessa*: un solo minimo, trovato da ogni punto di partenza (nelle reti neurali no) [slide Regressione logistica pp. 51–56].

### Gradiente e regola di aggiornamento

Il gradiente della loss è il vettore delle derivate parziali rispetto a ogni parametro (la componente  $i$  dice quanto una piccola variazione di  $w_i$  cambia la loss); con  $\hat{y} = f(x; \theta)$  e learning rate  $\eta$  (eta: più è alto, più il passo è lungo) [slide Regressione logistica pp. 57–62]:

$$\nabla_{\theta} L(f(x; \theta), y) = \left[ \frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_n} \right]^{\top}, \quad \theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(f(x; \theta), y),$$

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y] x_j, \quad \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} = \sigma(\mathbf{w} \cdot \mathbf{x} + b) - y.$$

Derivate «elegantissime»: l'errore  $\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y$  (stima meno etichetta) per la feature  $x_j$ ; dalla regola della catena con  $\sigma' = \sigma(1 - \sigma)$ :  $\partial L_{\text{CE}} / \partial z = \sigma(z) - y$ ,  $\partial z / \partial w_j = x_j$  [dal libro, pp. 101–102; non nelle slide].

### Stochastic gradient descent (SGD)

Input: loss  $L$ ,  $f$  con parametri  $\theta$ , training set  $(x^{(i)}, y^{(i)})_{i=1}^m$  [slide Regressione logistica p. 63].

1. Inizializza  $\theta \leftarrow 0$ .
2. Ripeti fino alla convergenza; per ogni esempio  $(x^{(i)}, y^{(i)})$  in ordine casuale: (a) facoltativo, per monitorare: calcola  $\hat{y}^{(i)} = f(x^{(i)}; \theta)$  e la loss  $L(\hat{y}^{(i)}, y^{(i)})$ ; (b)  $g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$ , la direzione in cui la loss cresce di più; (c)  $\theta \leftarrow \theta - \eta g$ , vai dalla parte opposta.
3. Restituisci  $\theta$ .

«Stocastico»: aggiorna a ogni esempio, con un cammino a scatti; più comune il *mini-batch* training, gradiente su gruppi di 512 o 1024 esempi (tutto il dataset: batch training) [slide Regressione logistica p. 68]. Il learning rate  $\eta$  è un *iperparametro* (scelto dal progettista, non appreso dai dati): troppo alto si salta oltre il minimo (*overshoot*), troppo basso si è lentissimi [slide Regressione logistica p. 64].

**Esempio: un passo di gradient descent.** Etichetta vera  $y = 1$ , feature  $x_1 = 3$  (parole positive) e  $x_2 = 2$  (parole negative), parametri iniziali  $w_1 = w_2 = b = 0$ ,  $\eta = 0.1$ . Con  $\theta^0 = 0$ :  $\mathbf{w} \cdot \mathbf{x} + b = 0$ ,  $\sigma(0) = 0.5$ , errore  $\sigma(0) - y = -0.5$  [slide Regressione logistica pp. 65–67]:

$$\nabla_{w,b} = \begin{bmatrix} (\sigma(0) - 1) x_1 \\ (\sigma(0) - 1) x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5 \cdot 3 \\ -0.5 \cdot 2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}, \quad \theta^1 = \theta^0 - \eta \nabla_{w,b} = -0.1 \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.10 \\ 0.05 \end{bmatrix}.$$

Cresce anche  $w_2$  (l'unico esempio era positivo; abbastanza esempi negativi lo renderebbero negativo) [slide Regressione logistica p. 67]; la loss scende da 0.693 a 0.403 [non nelle fonti del corso].

## 4.5 Overfitting e regolarizzazione

### L'idea

Un modello che si adatta perfettamente al training set modella anche il rumore (*overfitting*): una parola rara che per caso compare solo in recensioni positive riceve un peso enorme e sul test set, dove manca, fallisce; feature come il 4-gram «the same to you» *memorizzano* il training set (accuratezza 100%) e falliscono sui 4-gram nuovi del test set. Rimedio: penalizzare i pesi grandi: adattarsi un po' meno bene ai dati con pesi piccoli è meglio che adattarli benissimo con pesi enormi (nelle reti neurali: dropout) [slide Regressione logistica pp. 69–72].

### Regolarizzazione L2 e L1

All'obiettivo (qui scritto come log-probabilità da massimizzare) si aggiunge un termine di regolarizzazione  $R(\theta)$  con coefficiente  $\alpha$  [slide Regressione logistica pp. 72–74]:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^m \log P(y^{(i)} | x^{(i)}) - \alpha R(\theta), \quad R(\theta) = \underbrace{\|\theta\|_2^2 = \sum_j \theta_j^2}_{\text{L2 (ridge)}} \text{ o } \underbrace{\|\theta\|_1 = \sum_j |\theta_j|}_{\text{L1 (lasso)}}.$$

L2 è il quadrato della distanza euclidea di  $\theta$  dall'origine, L1 la distanza di Manhattan; L2 è più facile da ottimizzare (derivata  $2\theta_j$ ) e dà molti pesi piccoli, L1 dà soluzioni *sparse*, con molti pesi esattamente zero [dal libro, p. 100; non nelle slide].

## 4.6 Regressione logistica multinomiale e softmax

### L'idea

Spesso le classi sono più di due: positivo/negativo/neutro, le parti del discorso, gli SMS di emergenza. Le probabilità devono ancora sommare a 1: serve una generalizzazione della sigmoide, la *softmax*, che trasforma  $k$  numeri qualsiasi in una distribuzione di probabilità (l'esponenziale rende tutto positivo, la somma normalizza). Nomi: regressione logistica multinomiale, softmax regression, multinomial logit (un tempo MaxEnt); «regressione logistica» da sola indica il caso binario [slide Regressione logistica pp. 75–76].

### Softmax e regressione logistica multinomiale

Per  $\mathbf{z} = [z_1, \dots, z_k]$  e, nel modello, pesi  $\mathbf{w}_c$  e bias  $b_c$  per ogni classe  $c$  [slide Regressione logistica pp. 77, 79]:

$$\operatorname{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)}, \quad 1 \leq i \leq k; \quad p(y = c | \mathbf{x}) = \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^k \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}.$$

**Esempio.** Per  $\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$  la softmax dà  $[0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$ : la somma è 1 e il valore più alto, 3.2, prende il 74%. Nel caso binario la feature  $x_5$  («!») ha un solo peso,  $w_5 = 3.0$ , verso  $y = 1$ ; nel multinomiale uno per classe,  $w_{5,+} = 3.5$ ,  $w_{5,-} = 3.1$ ,  $w_{5,0} = -5.3$ : «!» indica positivo o negativo, quasi mai neutro [slide Regressione logistica pp. 78, 80].

### Come rispondere: «Descrivere la regressione logistica per la classificazione di testo»

1. **Definizione.** Classificatore supervisionato *discriminativo*: stima direttamente  $P(c | d)$  (Naive Bayes, generativo, stima  $P(d | c)P(c)$ ); baseline della classificazione supervisionata.
2. **Modello.** Feature  $\mathbf{x}$ , pesi  $\mathbf{w}$ , bias  $b$ ;  $z = \mathbf{w} \cdot \mathbf{x} + b$ ;  $\sigma(z) = 1/(1 + e^{-z})$  dà  $P(y = 1 | \mathbf{x})$ ; decisione con soglia 0.5 ( $z > 0$ ). Esempio del sentiment:  $z = 0.833$ ,  $\sigma = 0.70$ , positiva.
3. **Apprendimento.** Cross-entropy loss  $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] = -\log p(y | \mathbf{x})$  (0.36 se  $y = 1$ , 1.2 se  $y = 0$ ); SGD  $\theta \leftarrow \theta - \eta \nabla L$ ,  $\partial L / \partial w_j = (\sigma(z) - y) x_j$ ; loss convessa.
4. **Overfitting:** regolarizzazione L2 ( $\sum \theta_j^2$ ) o L1 ( $\sum |\theta_j|$ ). **Più classi:** softmax, un vettore di pesi per classe. **Collegamenti:** una rete neurale è una pila di regressioni logistiche (capitolo 6), con le stesse sigmoide, softmax, cross-entropy e SGD.

### Come rispondere: «Confrontare Naive Bayes e regressione logistica»

1. Entrambi probabilistici e supervisionati; cambia *quale* probabilità modellano.
2. **Naive Bayes, generativo:** modella  $P(d | c)P(c)$ , com'è fatto un documento di ogni

classe, e sceglie quella che lo «spiega meglio»; parametri da conteggi e smoothing, con l'ipotesi di indipendenza delle parole (capitolo 3).

3. **Regressione logistica, discriminativa:** modella direttamente  $P(c | d) = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$  e impara i pesi minimizzando la cross-entropy con SGD; cerca solo ciò che separa le classi.
4. **Feature e limiti.** Naive Bayes: conteggi, training immediato, ipotesi forte. Regressione logistica: ogni feature, pesi appresi insieme (feature correlate non contano due volte [non nelle fonti del corso]), ma training iterativo e regolarizzazione contro l'overfitting.

#### Errore tipico da evitare

- «Regressione» nel nome, ma è un *classificatore*: la classe la dà la soglia 0.5 su  $\sigma(z)$ .
- Il gradient descent va *contro* il gradiente ( $\theta - \eta \nabla L$ ), che indica dove la loss cresce. La loss usa il logaritmo naturale e non è l'accuratezza ( $-\ln 0.70 = 0.36$  anche se la predizione è giusta); la softmax non è «una sigmoide per componente»: le  $k$  probabilità sommano a 1.

## Capitolo 5

### Semantica vettoriale e word embedding

Finora una parola era una stringa o un indice  $w_i$  nel vocabolario: *cat* non era più vicina a *dog* che a *tax*. La **semantica vettoriale** (vector semantics) rappresenta il significato con un **vettore** costruito dai testi, così che parole simili abbiano vettori vicini.

#### 5.1 Dal significato alla distribuzione

##### L'idea: che cosa vorremmo da una teoria del significato

La semantica lessicale (lexical semantics) elenca ciò che una buona rappresentazione dovrebbe catturare [slide Semantica vettoriale pp. 4–15]. Un **lemma** (*mouse*) può avere più **sensi** (roditore, dispositivo); lemmi e sensi sono catalogati in risorse lessicali come **WordNet** (la più usata) e **BabelNet** (multilingue) [slide Semantica vettoriale p. 5]. **Sinonimia**: stesso significato in alcuni o tutti i contesti (*couch/sofa*), ma quella perfetta quasi non esiste. **Similarità**: elementi di significato in comune (*car/bicycle*); la si misura chiedendo a persone un punteggio da 0 a 10, come nel dataset **SimLex-999** (*vanish/disappear* 9.8, *hole/agreement* 0.3) [slide Semantica vettoriale p. 10][dal libro, p. 106; non nelle slide]. **Relatedness**: *coffee* e *cup* non sono simili ma associati; un **campo semantico** raccoglie le parole di un dominio (*surgeon, scalpel, nurse*). **Antonimia**: opposti su un solo tratto, per il resto molto simili: un'opposizione binaria o gli estremi di una scala (*long/short*), oppure «reversives», cambiamenti in direzioni opposte (*rise/fall*) [slide Semantica vettoriale p. 13]. **Connotazione**: *copy* contro *fake*; tre dimensioni affettive, valence (piacevolezza), arousal (intensità), dominance (controllo), con punteggi fra 0 e 1 nel lessico **NRC VAD** (*love*: valence 1.000, *toxic*: 0.008) [slide Semantica vettoriale p. 15].

##### Ipotesi distribuzionale e significato come punto nello spazio

**Idea 1.** «Il significato di una parola è il suo uso nella lingua» (Wittgenstein): il significato è definito dalla **distribuzione** della parola, cioè dalle parole vicine e dagli ambienti grammaticali in cui compare [slide Semantica vettoriale pp. 17, 20]. Non sappiamo che cosa sia *ongchoi*, ma compare con le stesse parole di *spinach* e *chard* («delicious», «sautéed with garlic»): è una verdura a foglia [slide Semantica vettoriale p. 18]. **Idea 2.** Osgood (1957): la connotazione come tre numeri (valence, arousal, dominance), un punto in uno spazio 3D [slide Semantica vettoriale p. 21]. Unendo le due idee, ogni parola è un **vettore**, detto **embedding** perché «immerso» in uno spazio costruito automaticamente da quali parole compaiono vicine nei testi; parole simili sono vicine: è la rappresentazione standard del significato in NLP [slide Semantica vettoriale pp. 22–24].

Due famiglie di embedding *statici* (un vettore fisso per parola) [slide Semantica vettoriale p. 25]: **tf-idf**, vettori *sparsi* (quasi tutte le componenti a zero) ottenuti contando (sezioni 5.2–5.4); **word2vec**, vettori *densi* (pochi numeri reali, quasi tutti non nulli) imparati da un classificatore (sezione 5.5).

## 5.2 Vettori sparsi: le matrici di co-occorrenza

### L'idea: contare chi compare con chi

Una tabella di conteggi: se le colonne sono documenti, ogni colonna è il vettore di un documento e ogni riga quello di una parola; se le colonne sono parole, la riga di *cherry* dice quante volte *pie*, *sugar*, *computer*... compaiono vicino a *cherry*, cioè la sua distribuzione resa numerica.

### Matrice termine-documento e matrice parola-parola

**Matrice termine-documento** (term-document matrix): una riga per parola del vocabolario  $V$ , una colonna per documento; la cella  $(t, d)$  conta quante volte  $t$  compare in  $d$ . Ogni documento è il vettore (colonna) delle sue parole [slide Semantica vettoriale p. 26]; documenti simili hanno vettori simili, ed è la base dell'**information retrieval** (la ricerca di documenti) [slide Semantica vettoriale p. 28]. Anche le righe sono vettori: *battle* è «il tipo di parola che compare in *Julius Caesar* e *Henry V*» [slide Semantica vettoriale p. 29]. **Matrice parola-parola** (word-word o term-context matrix), la più usata:  $|V| \times |V|$ , la cella  $(w, c)$  conta quante volte  $c$  compare nel corpus entro una **finestra** (context window) di poche parole prima e dopo  $w$ ; due parole sono simili se i loro vettori di contesto sono simili [slide Semantica vettoriale p. 30].

|        | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|--------|----------------|---------------|---------------|---------|
| battle | 1              | 0             | 7             | 13      |
| good   | 14             | 80            | 62            | 89      |
| fool   | 36             | 58            | 1             | 4       |
| wit    | 20             | 15            | 2             | 3       |

|             | aardvark | ... | computer | data | result | pie | sugar | ... |
|-------------|----------|-----|----------|------|--------|-----|-------|-----|
| cherry      | 0        | ... | 2        | 8    | 9      | 442 | 25    | ... |
| strawberry  | 0        | ... | 0        | 0    | 1      | 60  | 19    | ... |
| digital     | 0        | ... | 1670     | 1683 | 85     | 5   | 4     | ... |
| information | 0        | ... | 3325     | 3982 | 378    | 5   | 13    | ... |

Figura 5.1: A sinistra la matrice termine-documento di quattro parole in quattro opere di Shakespeare: le colonne (in rosso) sono i vettori dei documenti; nel piano (*fool*, *battle*) le commedie [36, 1] e [58, 0] puntano quasi nella stessa direzione, *Henry V* [4, 13] e *Julius Caesar* [1, 7] in un'altra. A destra la matrice parola-parola: la riga di *digital* (in rosso) è il suo vettore di contesto.

Fonte: slide Semantica vettoriale pp. 26–27, 30

## 5.3 Misurare la similarità: prodotto scalare e coseno

### L'idea: conta la direzione, non la lunghezza

Due vettori sono simili se hanno valori grandi nelle stesse dimensioni: il prodotto scalare lo misura. Ma una parola frequente (*of*, *the*) ha numeri grandi ovunque, quindi prodotto scalare alto con tutti; per confrontare direzioni e non lunghezze si normalizza, e si ottiene il coseno dell'angolo fra i due vettori: la **cosine similarity**.

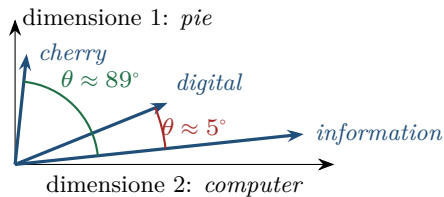
### Prodotto scalare, lunghezza, coseno

$$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i, \quad |\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}, \quad \cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\sum_i v_i w_i}{\sqrt{\sum_i v_i^2} \sqrt{\sum_i w_i^2}}$$

[slide Semantica vettoriale pp. 32–34]. Il prodotto scalare grezzo favorisce i vettori lunghi, cioè le parole frequenti [slide Semantica vettoriale p. 33]; da  $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$ , il rapporto normalizzato è il coseno dell'angolo: +1 stessa direzione, 0 ortogonali, -1 opposti; con conteggi (non negativi) sta fra 0 e 1 [slide Semantica vettoriale pp. 34–35].

**Esempio delle slide** [slide Semantica vettoriale p. 36]. Dalla matrice parola-parola prendiamo tre dimensioni, *pie*, *data*, *computer* (tabella qui sotto): quale parola è più vicina a *information*? Prodotti

scalari:  $\text{cherry} \cdot \text{information} = 442 \cdot 5 + 8 \cdot 3982 + 2 \cdot 3325 = 40716$  e  $\text{digital} \cdot \text{information} = 5 \cdot 5 + 1683 \cdot 3982 + 1670 \cdot 3325 = 12\,254\,481$ . Lunghezze:  $|\text{cherry}| = \sqrt{442^2 + 8^2 + 2^2} = 442.1$ ,  $|\text{digital}| = 2371.0$ ,  $|\text{information}| = 5187.7$ . Quindi  $\cos(\text{cherry}, \text{information}) = 40716 / (442.1 \cdot 5187.7) = 0.018$  e  $\cos(\text{digital}, \text{information}) = 12\,254\,481 / (2371.0 \cdot 5187.7) = 0.996$ : *information* è molto più vicina a *digital*, come ci si aspetta.



|             | pie | data | computer |
|-------------|-----|------|----------|
| cherry      | 442 | 8    | 2        |
| digital     | 5   | 1683 | 1670     |
| information | 5   | 3982 | 3325     |

Figura 5.2: Schema (non in scala) di *cherry*, *digital* e *information*; gli angoli sono quelli dei vettori a tre dimensioni: circa 5° (coseno 0.996) e 89° (coseno 0.018).

Fonte: slide Semantica vettoriale p. 37

Tre dimensioni della matrice parola-parola (slide Semantica vettoriale p. 36)

## 5.4 tf-idf: pesare i conteggi

**L'idea: la frequenza serve, ma non tutta**

Che *sugar* compaia spesso vicino ad *apricot* è informativo; che *the* o *it* compaiano spesso vicino ad *apricot* non dice nulla, compaiono vicino a tutto. Vogliamo un peso che cresca con la frequenza nel documento ma schiacci le parole presenti in *tutti* i documenti [slide Semantica vettoriale p. 38].

**Term frequency, inverse document frequency, tf-idf**

Con  $N$  documenti nella collezione,  $df_t$  il numero di documenti in cui compare  $t$ , e logaritmi in base 10:

$$tf_{t,d} = \log_{10}(\text{count}(t, d) + 1), \quad idf_t = \log_{10}\left(\frac{N}{df_t}\right), \quad w_{t,d} = tf_{t,d} \times idf_t$$

[slide Semantica vettoriale pp. 39–41]. Il logaritmo nel tf fa sì che 100 occorrenze non pesino 100 volte una (+1 per evitare  $\log 0$ ); l'idf è massimo per una parola presente in un solo documento e vale 0 per una presente in tutti. Il vettore tf-idf ha i pesi  $w_{t,d}$  al posto dei conteggi.

**Esempio delle slide** (idf sulle  $N = 37$  opere di Shakespeare; le  $df_t$  sono quelle del libro [dal libro, p. 248; non nelle slide]) [slide Semantica vettoriale p. 41]:

|        | $df_t$ | $idf_t$ | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|--------|--------|---------|----------------|---------------|---------------|---------|
| battle | 21     | 0.246   | 0.074          | 0             | 0.22          | 0.28    |
| good   | 37     | 0       | 0              | 0             | 0             | 0       |
| fool   | 36     | 0.012   | 0.019          | 0.021         | 0.0036        | 0.0083  |
| wit    | 34     | 0.037   | 0.049          | 0.044         | 0.018         | 0.022   |

Per esempio *battle* in *As You Like It*:  $tf = \log_{10}(1 + 1) = 0.301$ ,  $idf = \log_{10}(37/21) = 0.246$ ,  $w = 0.301 \times 0.246 = 0.074$ ; in *Henry V*:  $\log_{10}(14) \times 0.246 = 0.28$ . La parola *good* è la più frequente ma compare in tutte le 37 opere:  $idf = \log_{10} 1 = 0$  e il suo peso è zero ovunque, mentre nei conteggi grezzi dominava tutti i vettori.

**Limiti** [slide Semantica vettoriale pp. 42–43]. I vettori tf-idf sono **lunghi** ( $|V|$  da 20 000 a 50 000) e **sparsi**. Vorremmo vettori **corti** (50–1000) e **densi**: meno pesi da imparare quando sono feature di un classificatore; generalizzano meglio dei conteggi espliciti; catturano meglio la sinonimia (*car* e *automobile* sono dimensioni distinte: due parole vicine l’una a *car* e l’altra ad *automobile* non risultano simili); in pratica funzionano meglio.

#### Domanda 3 del programma: vettori sparsi, tf-idf e cosine similarity, limiti

1. **Idea.** Ipotesi distribuzionale: il significato di una parola è dato dalle parole con cui compare. Matrice di co-occorrenza termine-documento o, più spesso, parola-parola (la cella conta quante volte la parola di colonna compare in una finestra attorno a quella di riga); ogni parola è il suo vettore riga:  $|V|$  dimensioni, quasi tutte zero, perciò *sparso*.
2. **tf-idf.** I conteggi grezzi premiano parole frequenti ovunque (*the*, *good*), poco informative. Si pesa ogni cella con  $w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$ ,  $\text{tf} = \log_{10}(\text{count} + 1)$ ,  $\text{idf} = \log_{10}(N/\text{df}_t)$ : il tf cresce lentamente con la frequenza nel documento, l’idf azzerà le parole presenti in tutti i documenti. Esempio: *good* è in tutte le 37 opere di Shakespeare,  $\text{idf} = 0$ ; *battle* in 21,  $\text{idf} = 0.246$ , e in *As You Like It* pesa  $0.301 \times 0.246 = 0.074$ .
3. **Cosine similarity.** Il prodotto scalare  $\mathbf{v} \cdot \mathbf{w}$  è alto quando i vettori hanno valori grandi nelle stesse dimensioni, ma favorisce i vettori lunghi (parole frequenti). Si normalizza:  $\cos(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} / (|\mathbf{v}| |\mathbf{w}|)$ , il coseno dell’angolo, fra 0 e 1 per conteggi:  $\cos(\text{digital}, \text{information}) = 0.996$ ,  $\cos(\text{cherry}, \text{information}) = 0.018$ . Vale per parole, documenti e query.
4. **Limiti.** Vettori lunghi e sparsi: molti pesi da imparare se usati come feature, generalizzano peggio; i sinonimi (*car/automobile*) sono dimensioni distinte, quindi la sinonimia non viene catturata; conteggi, non rappresentazioni apprese; un vettore per parola. Soluzione: embedding densi e corti (word2vec), che in pratica funzionano meglio.

## 5.5 Embedding densi: word2vec

### L’idea: predire invece di contare

Invece di contare quante volte ogni parola compare vicino ad *apricot*, addestriamo un classificatore binario a rispondere «la parola *c* è probabile vicino ad *apricot*?». Del compito in sé non ci importa: ci teniamo i *pesi* imparati, che sono gli embedding. Il trucco è l’**auto-supervisione** (self-supervision): le parole davvero vicine ad *apricot* nel corpus sono le risposte «giuste», quelle pescate a caso le «sbagliate» (i «negativi»); nessuna etichetta umana [slide Semantica vettoriale pp. 45–46]. Word2vec è veloce da addestrare; qui si vede la variante **skip-gram with negative sampling** (SGNS) [slide Semantica vettoriale p. 45].

### Skip-gram con negative sampling, passo per passo

1. La parola target  $w$  e ogni parola di contesto  $c$  nella finestra (per esempio  $\pm 2$ ) sono gli **esempi positivi**: nella frase della figura 5.3, (*apricot*, *tablespoon*), (*apricot*, *of*), (*apricot*, *jam*), (*apricot*, *a*).
2. Per ogni positivo si pescano a caso  $k$  parole del lessico, campionate in base alla frequenza, come **esempi negativi** (noise word): con  $k = 2$ , (*apricot*, *aardvark*), (*apricot*, *seven*), (*apricot*, *my*), ...
3. Si addestra con la regressione logistica un classificatore che distingue i due casi.
4. I pesi imparati sono gli embedding; il classificatore si butta via.

[slide Semantica vettoriale pp. 48, 52–54, 62]

### Il classificatore skip-gram

Ogni parola ha **due** embedding di dimensione  $d$ : uno come target (riga della matrice  $W$ ) e uno come contesto o rumore (riga della matrice  $C$ ); i parametri  $\theta$  sono le  $2|V|$  righe di  $W$  e  $C$  [slide Semantica vettoriale p. 51]. La similarità fra target e contesto è il prodotto scalare,  $\text{Sim}(w, c) \approx \mathbf{c} \cdot \mathbf{w}$ , e la sigmoide della regressione logistica (def. 4.2) lo trasforma in probabilità:

$$P(+ | w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}, \quad P(- | w, c) = 1 - P(+ | w, c) = \sigma(-\mathbf{c} \cdot \mathbf{w})$$

[slide Semantica vettoriale pp. 49–50].

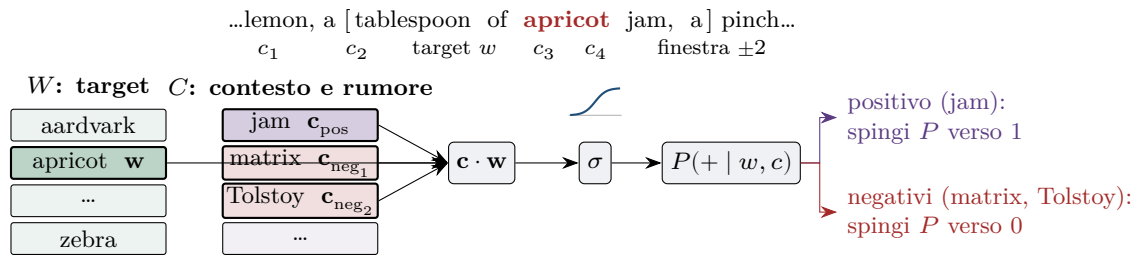


Figura 5.3: Skip-gram con negative sampling: il target *apricot* prende  $\mathbf{w}$  da  $W$ , il contesto vero *jam* e i  $k = 2$  negativi prendono  $\mathbf{c}$  da  $C$ ; il prodotto scalare passa nella sigmoide e dà  $P(+ | w, c)$ .

Fonte: schema costruito dalle slide Semantica vettoriale pp. 47, 49, 51, 57

### Loss e aggiornamento SGD per un target $w$ con $\mathbf{c}_{\text{pos}}$ e $\mathbf{c}_{\text{neg}_1}, \dots, \mathbf{c}_{\text{neg}_k}$

Vogliamo alta la probabilità che il contesto vero sia un vicino e alta quella che ogni parola di rumore *non* lo sia; assumendo indipendenza si moltiplicano e si prende  $-\log$  (logaritmo naturale): è la cross-entropy loss della regressione logistica (def. 4.3), da cui il pedice CE:

$$L_{\text{CE}} = -\log \left[ P(+ | w, \mathbf{c}_{\text{pos}}) \prod_{i=1}^k P(- | w, \mathbf{c}_{\text{neg}_i}) \right] = -\left[ \log \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) + \sum_{i=1}^k \log \sigma(-\mathbf{c}_{\text{neg}_i} \cdot \mathbf{w}) \right]$$

[slide Semantica vettoriale p. 55]. Si minimizza con lo **stochastic gradient descent** (def. 4.4), passo  $\eta$  (learning rate) [slide Semantica vettoriale pp. 56, 58]; derivate [slide Semantica vettoriale p. 59] e aggiornamenti [slide Semantica vettoriale p. 60], con le derivate calcolate nei valori correnti  $\mathbf{w}^t$ ,  $\mathbf{c}^t$ :

$$\begin{aligned} \frac{\partial L_{\text{CE}}}{\partial \mathbf{c}_{\text{pos}}} &= [\sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) - 1] \mathbf{w}, & \frac{\partial L_{\text{CE}}}{\partial \mathbf{c}_{\text{neg}_i}} &= [\sigma(\mathbf{c}_{\text{neg}_i} \cdot \mathbf{w})] \mathbf{w}, \\ \frac{\partial L_{\text{CE}}}{\partial \mathbf{w}} &= [\sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) - 1] \mathbf{c}_{\text{pos}} + \sum_{i=1}^k [\sigma(\mathbf{c}_{\text{neg}_i} \cdot \mathbf{w})] \mathbf{c}_{\text{neg}_i}, \\ \mathbf{c}_{\text{pos}}^{t+1} &= \mathbf{c}_{\text{pos}}^t - \eta \frac{\partial L_{\text{CE}}}{\partial \mathbf{c}_{\text{pos}}}, & \mathbf{c}_{\text{neg}_i}^{t+1} &= \mathbf{c}_{\text{neg}_i}^t - \eta \frac{\partial L_{\text{CE}}}{\partial \mathbf{c}_{\text{neg}_i}}, & \mathbf{w}^{t+1} &= \mathbf{w}^t - \eta \frac{\partial L_{\text{CE}}}{\partial \mathbf{w}} \end{aligned}$$

Si parte da  $W$  e  $C$  casuali e si aggiorna esempio dopo esempio; poiché  $\sigma - 1 < 0$  e  $\sigma > 0$ ,  $\mathbf{c}_{\text{pos}}$  si sposta *verso*  $\mathbf{w}$  e ogni  $\mathbf{c}_{\text{neg}_i}$  si allontana [slide Semantica vettoriale p. 57]. Alla fine ogni parola ha due embedding; di solito si usa la somma  $\mathbf{w}_i + \mathbf{c}_i$  [slide Semantica vettoriale p. 61].

**Un passo con i numeri** (esempio inventato [non nelle fonti del corso];  $d = 2$ ,  $k = 2$ ,  $\eta = 0.5$ ). Target  $\mathbf{w} = (1.0, 0.5)$ , contesto vero  $\mathbf{c}_{\text{pos}} = (0.8, 0.6)$ , rumore  $\mathbf{c}_{\text{neg}_1} = (-0.4, 0.9)$  e  $\mathbf{c}_{\text{neg}_2} = (0.2, -1.0)$ . Loss prima del passo:  $L_{\text{CE}} = -[\ln 0.750 + \ln 0.488 + \ln 0.574] = 1.56$ .

|                                       | $\mathbf{c} \cdot \mathbf{w}$ | probabilità    | derivata di $L_{CE}$                                                                                                    | valore             | dopo il passo     |
|---------------------------------------|-------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------|
| $\mathbf{c}_{\text{pos}}$ (jam)       | 1.10                          | $P(+) = 0.750$ | $(0.750 - 1) \mathbf{w}$                                                                                                | $(-0.250, -0.125)$ | $(0.925, 0.662)$  |
| $\mathbf{c}_{\text{neg}_1}$ (matrix)  | 0.05                          | $P(-) = 0.488$ | $0.512 \mathbf{w}$                                                                                                      | $(0.512, 0.256)$   | $(-0.656, 0.772)$ |
| $\mathbf{c}_{\text{neg}_2}$ (Tolstoy) | -0.30                         | $P(-) = 0.574$ | $0.426 \mathbf{w}$                                                                                                      | $(0.426, 0.213)$   | $(-0.013, -1.11)$ |
| $\mathbf{w}$ (apricot)                |                               |                | $-0.250 \mathbf{c}_{\text{pos}} + 0.512 \mathbf{c}_{\text{neg}_1} + 0.426 \mathbf{c}_{\text{neg}_2} = (-0.320, -0.114)$ |                    |                   |
|                                       |                               |                |                                                                                                                         |                    | $(1.16, 0.557)$   |

Ogni vettore si sposta di  $-\eta$  volte la sua derivata; dopo il passo i prodotti scalari sono 1.44,  $-0.331$  e  $-0.631$  (erano 1.10, 0.05,  $-0.30$ ) e la loss scende a 1.18: il contesto vero si è avvicinato, i rumori allontanati.

#### Domanda 2 del programma: importanza dei word embeddings e Word2Vec

1. **Che cosa sono.** Un word embedding è un vettore che rappresenta il significato di una parola, imparato dai testi secondo l'ipotesi distribuzionale (contesti simili, significati simili); parole simili hanno vettori vicini.
2. **Perché contano per la comprensione del linguaggio.** Una stringa o un indice non dice nulla del significato. Con gli embedding la similarità diventa calcolabile (*cat* vicino a *dog*) e un sistema *generalizza*: un classificatore di sentiment che riceve embedding classifica bene anche frasi con parole mai viste nel training, se hanno vettori simili a quelle viste [dal libro, p. 109; non nelle slide]. Catturano anche relazioni ( $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ ), connotazione e campi semantici. Per questo sono l'ingresso di ogni algoritmo moderno di NLP; quelli di word2vec sono *statici* (uno per parola), BERT li rende *contestuali*.
3. **Word2vec (skip-gram with negative sampling).** Predire invece di contare: un classificatore binario « $c$  compare vicino a  $w$ ?», auto-supervisionato dal corpus. (a) Positivi:  $w$  con ogni  $c$  nella finestra ( $\pm 2$ ); negativi: per ogni positivo  $k$  parole a caso, pesate per frequenza. (b) Due embedding per parola, matrici  $W$  (target) e  $C$  (contesto),  $d$  fra 50 e 1000, casuali all'inizio. (c)  $P(+ | w, c) = \sigma(\mathbf{c} \cdot \mathbf{w})$ ,  $P(- | w, c) = \sigma(-\mathbf{c} \cdot \mathbf{w})$ . (d) Loss  $L = -[\log \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) + \sum_i \log \sigma(-\mathbf{c}_{\text{neg}_i} \cdot \mathbf{w})]$ , minimizzata con SGD:  $\mathbf{c}_{\text{pos}}$  si avvicina a  $\mathbf{w}$ , i  $\mathbf{c}_{\text{neg}_i}$  si allontanano. (e) Si butta via il classificatore, si tengono gli embedding ( $\mathbf{w}_i + \mathbf{c}_i$ ).
4. **Proprietà e limiti.** Densi e corti: meno pesi, generalizzano, catturano la sinonimia meglio di tf-idf; finestra piccola dà vicini sintattici, grande lo stesso campo semantico; analogie col parallelogramma (solo parole frequenti e certe relazioni); un vettore per parola (niente polisemia); bias del corpus ( $\text{man} : \text{programmer} :: \text{woman} : \text{homemaker}$ ).

## 5.6 Proprietà degli embedding, analogie e bias

### L'idea: anche le differenze fra vettori hanno un significato

Se i vettori riflettono i contesti d'uso, lo spostamento  $\overrightarrow{\text{tree}} - \overrightarrow{\text{apple}}$  («dal frutto alla pianta») vale anche per altri frutti; e lo spazio eredita tutto ciò che c'è nei testi, stereotipi compresi.

**Finestra.** Con finestre piccole ( $\pm 2$ ) i vicini sono parole sintatticamente simili della stessa tassonomia (*Hogwarts: Sunnydale, Evernight*, altre scuole di fantasia), con finestre grandi ( $\pm 5$ ) parole dello stesso campo semantico (*Dumbledore, Malfoy*) [slide Semantica vettoriale p. 63].

**Analogie e parallelogramma.** Per «*apple* sta a *tree* come *grape* sta a \_\_\_\_» si somma a *grape* il vettore  $\overrightarrow{\text{tree}} - \overrightarrow{\text{apple}}$  e si prende la parola più vicina, *vine* [slide Semantica vettoriale p. 65]; per  $a : a^* :: b : b^*$ ,  $\hat{b}^* = \arg\min_x \text{distance}(x, a^* - a + b)$  [slide Semantica vettoriale p. 66]:  $\text{king} - \text{man} + \text{woman}$  è vicino a *queen*; vale per embedding sparsi e densi e si vede bene nello spazio di GloVe (Global Vectors), l'altro embedding statico scaricabile, costruito dalle statistiche globali di co-occorrenza del corpus (rapporti di probabilità nella matrice parola-parola) [slide Semantica vettoriale pp. 44, 66–67][dal libro, p. 119; non nelle slide]. *Caveat*: solo per parole frequenti, distanze piccole e certe relazioni [slide Semantica vettoriale p. 68].

**Semantica storica e bias.** Embedding addestrati su decenni diversi mostrano come cambia il significato [slide Semantica vettoriale p. 69]; riflettono i bias culturali del corpus («father : doctor :: mother :  $x$ » dà *nurse*, «man : computer programmer :: woman :  $x$ » dà *homemaker*), e un sistema di selezione del personale basato su embedding può discriminare [slide Semantica vettoriale p. 70]; con embedding storici si misurano i bias del passato [slide Semantica vettoriale p. 71].

#### Confusioni tipiche

- **tf-idf è sparso, word2vec è denso:** conteggi pesati in  $|V|$  dimensioni contro 50–1000 numeri reali imparati (anche negativi); e  $\text{idf} = 0$  non è un errore (*good* è ovunque).
- **Prodotto scalare  $\neq$  coseno:** il primo premia i vettori lunghi (parole frequenti), il coseno divide per le lunghezze; skip-gram usa però il prodotto scalare.
- **Il classificatore non è il prodotto finale:** serve solo a imparare  $W$  e  $C$ . E un embedding statico non distingue i sensi di *mouse*: servono embedding contestuali (BERT, capitolo 7).

## Capitolo 6

### Reti neurali per il linguaggio

Finora il classificatore era *lineare* (capp. 3 e 4); qui più strati di «regressioni logistiche» imparano da soli le feature, a partire dai word embedding (cap. 5).

#### 6.1 L'unità neurale

##### L'idea: una regressione logistica in miniatura

Un'unità (*neural unit*, «neurone») fa tre cose: moltiplica ogni input per un peso e somma, aggiunge una costante (il *bias*) e passa il risultato in una funzione non lineare, l'*activation function*. Con la sigmoide è esattamente la regressione logistica (cap. 4). Da sola separa gli input con una retta (un iperpiano); la potenza arriva quando se ne collegano tante in strati.

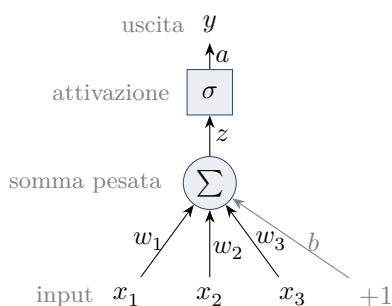


Figura 6.1: L'unità neurale: somma pesata  $z$ , attivazione  $\sigma$ , uscita  $y = a$ ; il bias  $b$  è il peso di un input fisso a  $+1$ . Fonte: slide Reti neurali p. 3

##### Unità neurale e funzioni di attivazione

Dati gli input  $x_1, \dots, x_n$ , i pesi  $w_1, \dots, w_n$  e il bias  $b$ :

$$z = b + \sum_i w_i x_i = \mathbf{w} \cdot \mathbf{x} + b, \quad y = a = f(z).$$

[slide Reti neurali pp. 3–4]. Le attivazioni  $f$  del corso [slide Reti neurali pp. 5–7]: la *sigmoide*  $\sigma(z) = 1/(1 + e^{-z})$  della regressione logistica (def. 4.2), con valori in  $(0, 1)$  (una probabilità); la *tanh*  $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$ , in  $(-1, 1)$ ; la *ReLU* (rectified linear unit)  $\max(z, 0)$ , la più usata. L'unità sigmoide calcola quindi  $y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = 1/(1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)})$  [slide Reti neurali p. 6].

Esempio [dal libro, p. 130; non nelle slide]:  $\mathbf{w} = [0.2, 0.3, 0.9]$ ,  $b = 0.5$ ,  $\mathbf{x} = [0.5, 0.6, 0.1]$ :  $z = 0.5 \cdot 0.2 + 0.6 \cdot 0.3 + 0.1 \cdot 0.9 + 0.5 = 0.87$  e  $y = \sigma(0.87) = 0.705$ . La ReLU è la più usata perché sigmoide e *tanh saturano* (derivata quasi zero per  $|z|$  grande) e il segnale d'errore svanisce nell'addestramento (*vanishing gradient*); la ReLU ha derivata 1 per  $z > 0$  [dal libro, p. 131; non nelle slide].

#### 6.2 Perché serve un hidden layer: il problema XOR

##### L'idea

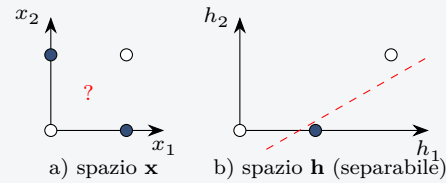
Una sola unità decide con una retta:  $w_1 x_1 + w_2 x_2 + b = 0$  divide il piano in due. Per AND e OR basta; per XOR no: i punti positivi  $(0, 1)$  e  $(1, 0)$  sono agli angoli opposti e nessuna retta li isola dagli altri due (Minsky e Papert, 1969). Con un *hidden layer* (strato nascosto) la rete prima *trasforma* gli input in nuove coordinate  $\mathbf{h}$ , dove i punti diventano separabili: gli hidden layer imparano *rappresentazioni*, l'idea che regge il deep learning.

##### XOR con una rete a due strati di unità ReLU

XOR non è *linearly separable*: nessuna unità singola lo calcola, una rete a strati sì [slide Reti neurali pp. 8–10]. Con i pesi delle slide (la ReLU agisce componente per componente):

$$\mathbf{h} = \text{ReLU}(W\mathbf{x} + \mathbf{b}), \quad W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}; \quad y = \text{ReLU}(h_1 - 2h_2).$$

| $x_1$ | $x_2$ | $W\mathbf{x} + \mathbf{b}$ |    | $h_1$ | $h_2$ | $y$ |
|-------|-------|----------------------------|----|-------|-------|-----|
| 0     | 0     | 0                          | -1 | 0     | 0     | 0   |
| 0     | 1     | 1                          | 0  | 1     | 0     | 1   |
| 1     | 0     | 1                          | 0  | 1     | 0     | 1   |
| 1     | 1     | 2                          | 1  | 2     | 1     | 0   |



Nello spazio  $\mathbf{h}$  i due input positivi collasano nello stesso punto  $(1, 0)$  e una retta li separa da  $(0, 0)$  e  $(2, 1)$  [slide Reti neurali p. 11]; in una rete vera i pesi li trova l'addestramento (sez. 6.5). Serve anche la *non linearità*: senza  $f$ , due strati lineari  $W^{[2]}(W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]} = W'\mathbf{x} + \mathbf{b}'$  equivalgono a uno solo [dal libro, p. 137; non nelle slide].

### 6.3 Reti feed-forward per la classificazione del testo

#### L'idea

Una *feed-forward network* (o *multi-layer perceptron*, MLP) è fatta di strati di unità collegati solo in avanti, senza cicli, e *fully connected*: ogni unità riceve tutte le uscite dello strato precedente. L'input non si conta: la regressione logistica è una rete «a 1 strato», con un *hidden layer* si ha una rete «a 2 strati». L'hidden layer costruisce una rappresentazione  $\mathbf{h}$  dell'input e l'*output layer* (strato d'uscita) fa una regressione logistica (sigmoide o softmax) su  $\mathbf{h}$ : così la rete sfrutta interazioni non lineari fra le feature.

#### Rete a uno e a due strati, con le dimensioni

*Uno strato* = regressione logistica:  $y = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$  (binaria,  $y$  scalare) oppure  $\mathbf{y} = \text{softmax}(W\mathbf{x} + \mathbf{b})$  ( $K$  classi), con la softmax della regressione logistica multinomiale (def. 4.6) [slide Reti neurali pp. 13–15]. *Due strati* (un hidden layer) [slide Reti neurali pp. 16–19]:

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b}), \quad \mathbf{z} = U\mathbf{h}, \quad y = \sigma(z) \quad \text{oppure} \quad \mathbf{y} = \text{softmax}(\mathbf{z})$$

(uscita scalare, oppure un'unità per classe e un softmax layer [slide Reti neurali p. 30]); nell'hidden layer  $\sigma$  sta per una qualunque attivazione (sigmoide, ReLU o tanh). Dimensioni, con  $n_0$  input,  $n_1$  unità nascoste e  $n_2$  uscite:  $\mathbf{x} \in \mathbb{R}^{n_0}$ ,  $W \in \mathbb{R}^{n_1 \times n_0}$  ( $W_{ji}$  = peso dall'input  $i$  all'unità nascosta  $j$ ),  $\mathbf{b}, \mathbf{h} \in \mathbb{R}^{n_1}$ ,  $U \in \mathbb{R}^{n_2 \times n_1}$ ,  $\mathbf{y} \in \mathbb{R}^{n_2}$ ;  $n_2 = 1$  per l'uscita scalare [slide Reti neurali p. 17] [dal libro, pp. 135–136; non nelle slide]. *Notazione multistrato* [slide Reti neurali p. 20]:  $\mathbf{a}^{[0]} = \mathbf{x}$  e, per ogni strato  $i$ ,  $\mathbf{z}^{[i]} = W^{[i]}\mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$ ,  $\mathbf{a}^{[i]} = g^{[i]}(\mathbf{z}^{[i]})$ , con  $g^{[1]} = \text{ReLU}$  e  $g^{[2]} = \text{sigmoide}$  o softmax;  $\hat{\mathbf{y}} = \mathbf{a}^{[2]}$ . *Bias come peso*: con un input fisso  $x_0 = 1$  il bias diventa  $W_{j0} = b_j$ ,  $h_j = \sigma(\sum_{i=0}^{n_0} W_{ji}x_i)$  e  $\mathbf{y} = \text{softmax}(U\sigma(W\mathbf{x}))$  [slide Reti neurali pp. 21–22].

**Dalle feature a mano agli embedding.** Per la sentiment analysis si può fare come con la regressione logistica: in ingresso le sue sei feature a mano (fig. 4.2), in uscita 0/1; l'hidden layer aggiunge interazioni non lineari fra feature, «che possono o no migliorare» il risultato [slide Reti neurali pp. 24–26]. La vera forza del deep learning è il *representation learning*: al posto delle feature progettate dall'uomo si usano rappresentazioni *imparate dai dati*, i word embedding [slide Reti neurali p. 27]. Nella figura 6.2 ogni parola seleziona la sua riga nella matrice degli embedding  $E$  ( $|V| \times d$ ); i tre embedding concatenati (*projection layer*,  $3d \times 1$ ) entrano in una rete a due strati con uscita sigmoide  $\hat{y} = P(\text{positivo} \mid \text{«The dessert is...»})$  [slide Reti neurali p. 28].

**Il pooling.** Così l'input ha lunghezza fissa (3 parole), ma i testi no. Due soluzioni [slide Reti neurali p. 29]: (1) *padding* con embedding nulli fino alla lunghezza della recensione più lunga (e troncamento); (2) un unico *sentence embedding* di dimensione  $d$  con il *pooling*: la *media* degli embedding (mean pooling) oppure il *massimo componente per componente* (max pooling). Con il

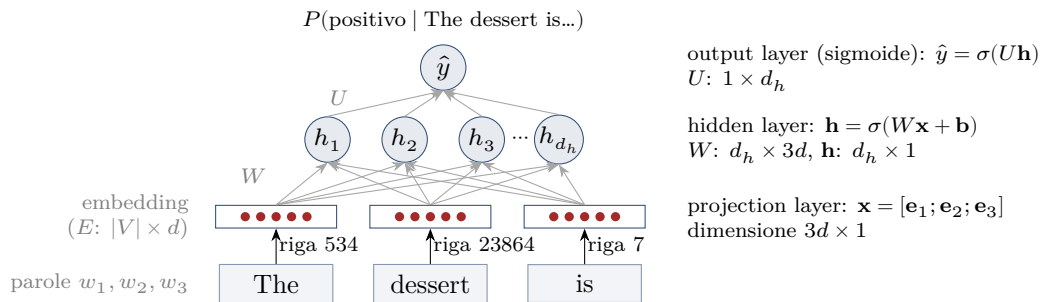


Figura 6.2: Rete a due strati per il sentiment: embedding in ingresso e dimensioni. Fonte: slide Reti neurali p. 28

pooling  $\mathbf{x} \in \mathbb{R}^d$  e  $W$  è  $d_h \times d$ ; si perde l'ordine delle parole, che per il sentiment conta meno della loro identità [dal libro, p. 142; non nelle slide].

**Domanda 5 del programma:** «Descrivere l'utilizzo di una feed-forward network con un hidden layer per la classificazione di testo»

1. *Definizione.* Rete di unità (somma pesata + bias + attivazione non lineare) in strati fully connected senza cicli; con un hidden layer è «a due strati»: l'hidden layer costruisce la rappresentazione  $\mathbf{h}$  del testo, l'output layer fa una regressione logistica su  $\mathbf{h}$ .
2. *Formule e dimensioni.*  $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$ ,  $\mathbf{z} = U\mathbf{h}$ ,  $\hat{y} = \sigma(z)$  (binaria) oppure  $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$  ( $K$  classi);  $W \in \mathbb{R}^{d_h \times n_0}$ ,  $\mathbf{b}, \mathbf{h} \in \mathbb{R}^{d_h}$ ,  $U \in \mathbb{R}^{1 \times d_h}$  oppure  $\mathbb{R}^{K \times d_h}$ ;  $\hat{y} = P(\text{classe} \mid \text{testo})$ .
3. *Input.* (a) Le sei feature a mano della regressione logistica, con l'hidden layer che aggiunge interazioni non lineari. (b) *Representation learning*: word embedding dalla matrice  $E$  ( $|V| \times d$ ), concatenati ( $3d \times 1$ ) oppure *pooled* (media o max per componente) in un vettore di dimensione  $d$ , per testi di lunghezza variabile.
4. *Perché l'hidden layer; addestramento.* XOR: una sola unità separa solo linearmente, l'hidden layer trasforma lo spazio e rende i casi separabili (senza non linearità gli strati collapsano in uno). Addestramento: cross-entropy, gradient descent, backpropagation.
5. *Esempio, limiti, collegamenti.* «The dessert is...»  $\rightarrow P(\text{positivo})$ ; il pooling perde l'ordine, la concatenazione impone la finestra fissa; oggi: transformer pre-addestrati (BERT, cap. 7) con fine-tuning.

## 6.4 Il modello di linguaggio neurale feed-forward

### L'idea

Un language model dà la probabilità della prossima parola data la storia (cap. 2). I modelli neurali battono nettamente gli n-gram; i migliori sono i transformer (cap. 7), ma il feed-forward LM mostra già il meccanismo: una *finestra scorrevole* di lunghezza fissa, gli embedding delle parole nella finestra concatenati, un softmax su tutto il vocabolario.

### Feed-forward LM con finestra di 3 parole

Con  $\mathbf{x}_{t-k}$  il vettore *one-hot* della parola  $w_{t-k}$  (un 1 al posto della parola, 0 altrove: il prodotto con  $E$  ne seleziona la riga, cioè l'embedding) ed  $E$  la matrice degli embedding:

$$\mathbf{e} = [E\mathbf{x}_{t-3}; E\mathbf{x}_{t-2}; E\mathbf{x}_{t-1}] \quad (3d \times 1), \quad \mathbf{h} = \sigma(W\mathbf{e} + \mathbf{b}) \quad (d_h \times 1), \quad \hat{\mathbf{y}} = \text{softmax}(U\mathbf{h}) \quad (|V| \times 1),$$

con  $W \in \mathbb{R}^{d_h \times 3d}$ ,  $U \in \mathbb{R}^{|V| \times d_h}$  e  $\hat{y}_i = P(w_t = V_i \mid w_{t-3}, w_{t-2}, w_{t-1})$  [slide Reti neurali pp. 32–33] [dal libro, p. 145; non nelle slide].

Perché batte gli n-gram [slide Reti neurali p. 34]: nel training si è visto «the cat gets fed», mai «dog gets fed». Davanti a «the dog gets \_\_\_\_\_» un n-gram LM non può predire «fed»; il modello neurale sì, perché gli embedding di *cat* e *dog* sono simili e la rete generalizza da un contesto all'altro.

## 6.5 Come si addestra: loss, gradiente, backpropagation

### L'idea

È apprendimento supervisionato: per ogni esempio  $(\mathbf{x}, y)$  la rete produce  $\hat{y}$  (*forward pass*), una *loss* misura quanto  $\hat{y}$  è lontano da  $y$ , e ogni peso si sposta un po' *contro* il gradiente della loss, la direzione in cui la loss cresce di più (*gradient descent*, cap. 4). La difficoltà: la loss si calcola in fondo alla rete, ma servono le derivate rispetto ai pesi di ogni strato. La soluzione è la regola della catena applicata all'indietro, nodo per nodo, su un *computation graph*: la *backpropagation* (Rumelhart, Hinton e Williams, 1986), un caso di backward differentiation.

### Loss, aggiornamento dei pesi, chain rule

La loss è la cross-entropy della regressione logistica (def. 4.3; logaritmo naturale) [slide Reti neurali p. 38]: per l'uscita binaria  $L_{\text{CE}}(\hat{y}, y) = -\log p(y | \mathbf{x})$ ; per  $K$  classi  $L_{\text{CE}} = -\sum_k y_k \log \hat{y}_k = -\log \hat{y}_c$ , con  $c$  la classe giusta [dal libro, p. 146; non nelle slide]. Aggiornamento per gradient descent con learning rate  $\eta$  (def. 4.4), e gradiente della regressione logistica (def. 4.4) [slide Reti neurali p. 39]:

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(\mathbf{x}; w), y), \quad \frac{\partial L_{\text{CE}}}{\partial w_j} = [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y] x_j.$$

Viene dalla chain rule (regola della catena) applicata ai tre pezzi dell'unità:  $\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$  (loss  $\times$  attivazione  $\times$  somma pesata) [slide Reti neurali p. 40]. Derivate che servono:  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ ;  $\text{ReLU}'(z) = 0$  per  $z < 0$  e  $1$  per  $z \geq 0$  [slide Reti neurali p. 54];  $\tanh'(z) = 1 - \tanh^2(z)$  [dal libro, p. 151; non nelle slide].

### Backpropagation su un computation graph

Un computation graph rappresenta un calcolo: un nodo per ogni operazione, archi dalle uscite agli input successivi [slide Reti neurali p. 43]. Per ogni esempio  $(\mathbf{x}, y)$  [slide Reti neurali pp. 36–37, 46]:

1. *Forward pass*: si valutano i nodi da sinistra a destra ( $\mathbf{z}^{[1]}, \mathbf{a}^{[1]}, \dots, \hat{y}$ ) memorizzando i valori intermedi, e si calcola la loss  $L(\hat{y}, y)$  al nodo finale.
2. *Backward pass*: da  $L$  all'indietro, ogni nodo riceve il gradiente *upstream*  $\partial L / \partial (\text{sua uscita})$ , lo moltiplica per il gradiente *locale* (derivata della sua uscita rispetto a ciascun input) e passa il prodotto (gradiente *downstream*) al nodo precedente [dal libro, p. 148; non nelle slide].
3. Quando il gradiente arriva a un peso  $w$ :  $w \leftarrow w - \eta \partial L / \partial w$ .

**L'esempio delle slide.**  $L(a, b, c) = c(a + 2b)$ , spezzato in  $d = 2b$ ,  $e = a + d$ ,  $L = c \cdot e$ ; con  $a = 3$ ,  $b = 1$ ,  $c = -2$  il forward pass dà  $d = 2$ ,  $e = 5$ ,  $L = -10$  [slide Reti neurali pp. 44–45]. Backward pass, con i gradienti locali sugli archi della figura 6.3:  $\partial L / \partial c = e = 5$  e  $\partial L / \partial e = c = -2$ ; quindi  $\partial L / \partial a = \partial L / \partial d = -2 \cdot 1 = -2$  e  $\partial L / \partial b = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b} = -2 \cdot 1 \cdot 2 = -4$  [slide Reti neurali pp. 49–52].

**Sulla rete a due strati** (notazione della def. 6.3, con  $g^{[1]} = \text{ReLU}$  e  $\hat{y} = \sigma(z^{[2]})$ ) il computation graph ha un nodo per ogni prodotto  $w_{ji}x_i$ , somma, ReLU, la sigmoide e la loss [slide Reti neurali pp. 53–55]; il backward [dal libro, p. 151; non nelle slide]: il primo passo dà  $\partial L / \partial z^{[2]} = a^{[2]} - y$ , come nella regressione logistica, poi il gradiente attraversa  $W^{[2]}$ , passa le ReLU solo dove  $z_j^{[1]} \geq 0$  e arriva

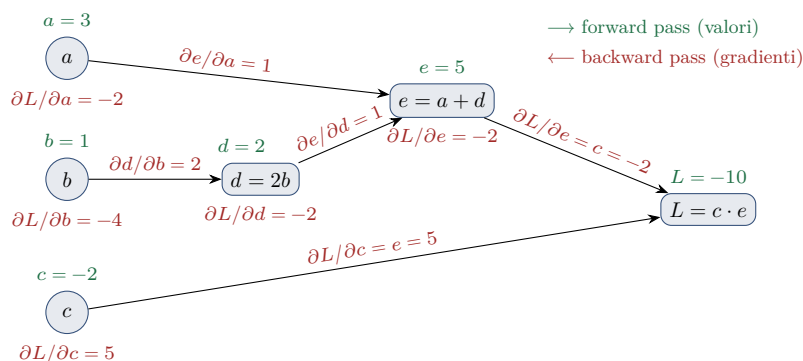


Figura 6.3: Computation graph di  $L = c(a + 2b)$  con  $a = 3$ ,  $b = 1$ ,  $c = -2$ : in verde il forward pass; in rosso i gradienti locali sugli archi e, sotto i nodi, i gradienti  $\partial L / \partial(\cdot)$  del backward pass.

Fonte: slide Reti neurali pp. 45 e 52

a  $W^{[1]}$ . **In pratica** PyTorch (`torch.nn`) costruisce il computation graph, calcola il backward da solo e gestisce la GPU [slide Reti neurali pp. 56–58]. L’addestramento dura più *epoche* (un’epoca = una passata completa sul training set) [dal libro, p. 214; non nelle slide]; il gradient descent ha varianti come Adam (Kingma e Ba, 2015), l’*optimizer* (ottimizzatore) di GPT [slide GPT p. 9] [dal libro, p. 152; non nelle slide]: un SGD (def. 4.4) con learning rate adattivo per ogni peso [non nelle fonti del corso]; pesi iniziali piccoli e casuali, *dropout* (unità spente a caso a ogni passo) contro l’overfitting, iperparametri ( $\eta$ , mini-batch, strati e unità) scelti sul dev set [dal libro, p. 152; non nelle slide].

**Come rispondere: «come si addestra una rete neurale? che cos’è la backpropagation?»**

(1) Supervisionato: coppie  $(\mathbf{x}, y)$ , forward pass  $\rightarrow \hat{y}$ ; (2) loss cross-entropy  $-\log p(y | \mathbf{x})$ ; (3) gradient descent  $w \leftarrow w - \eta \partial L / \partial w$ ; (4) pesi interni: chain rule sul computation graph, il forward memorizza i valori, il backward moltiplica gradiente upstream  $\times$  locale (esempio  $L = c(a + 2b)$ :  $\partial L / \partial b = -4$ ); (5) derivate di  $\sigma$  e ReLU,  $\partial L / \partial z^{[2]} = a^{[2]} - y$ ; (6) PyTorch fa il backward da solo; più epoche, Adam, dropout e iperparametri sul dev set.

## 6.6 Oltre la finestra fissa: RNN e LSTM, in breve

### L’idea

Il feed-forward LM vede solo una finestra fissa di parole. Una rete *ricorrente* legge la sequenza una parola alla volta e porta avanti una memoria  $\mathbf{h}$  di tutto ciò che ha letto; la LSTM aggiunge dei *gate* («rubinetti») che decidono che cosa tenere e che cosa dimenticare (nelle slide sono solo nominati; libro pp. 292–317).

- Una *recurrent neural network* (RNN) ha un ciclo: l’hidden layer al tempo  $t$  dipende dall’input e dal proprio valore precedente,  $\mathbf{h}_t = g(U\mathbf{h}_{t-1} + W\mathbf{x}_t)$ ,  $\mathbf{y}_t = \text{softmax}(V\mathbf{h}_t)$ ;  $\mathbf{h}_{t-1}$  è una memoria di tutto il contesto, senza finestra fissa [dal libro, pp. 293–294; non nelle slide]. Si addestra con la backpropagation sul grafo srotolato nel tempo, a pesi condivisi (*backpropagation through time*) [dal libro, p. 295; non nelle slide]; come LM predice la parola successiva a ogni passo [dal libro, p. 297; non nelle slide].
- Varianti [dal libro, pp. 302–304; non nelle slide]: le *stacked RNN* impilano più RNN (l’uscita di una è l’input della successiva: rappresentazioni via via più astratte); una *bidirectional RNN* legge la sequenza con due RNN, da sinistra a destra e da destra a sinistra, e concatena i due stati,  $\mathbf{h}_t = [\mathbf{h}_t^f; \mathbf{h}_t^b]$ : così usa anche il contesto a destra (utile per il tagging, cap. 8).
- Limiti: l’informazione in  $\mathbf{h}$  resta locale e, moltiplicando i gradienti a ogni passo, si annullano (*vanishing gradients*): le dipendenze lontane si perdono [dal libro, pp. 304–305; non nelle slide].

- La *LSTM* (long short-term memory) aggiunge un vettore di contesto  $\mathbf{c}_t$  e tre *gate*, ciascuno  $\sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t)$  usato come maschera con il prodotto componente per componente  $\odot$ : il *forget gate*  $\mathbf{f}_t$  cancella il superfluo, l'*add gate*  $\mathbf{i}_t$  aggiunge l'informazione nuova  $\mathbf{g}_t = \tanh(U_g\mathbf{h}_{t-1} + W_g\mathbf{x}_t)$ , l'*output gate*  $\mathbf{o}_t$  sceglie che cosa esporre:  $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$ ,  $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$  [dal libro, p. 306; non nelle slide]. Dal collo di bottiglia dell'encoder-decoder RNN per la machine translation (un solo vettore di contesto) nasce l'*attention* (cap. 7) [dal libro, p. 313; non nelle slide].

#### Errori tipici

- Contare gli strati: con un hidden layer la rete è «a due strati» (hidden + output): l'input non si conta. Senza non linearità più strati valgono uno solo (XOR resta impossibile).
- Nel sentiment con uscita scalare  $U$  è  $1 \times d_h$ ;  $|V| \times d_h$  è la forma del language model (uscita su tutto il vocabolario).
- Il gradient descent va *contro* il gradiente ( $-\eta$ ); la backpropagation non è un algoritmo di apprendimento alternativo: calcola i gradienti che il gradient descent usa.

## Capitolo 7

### Transformer, BERT ed encoder-decoder

Prima del 2017 i modelli di sequenza erano reti ricorrenti (RNN e LSTM, sez. 6.6), che leggono una parola alla volta; «Attention is all you need» (Vaswani et al., 2017) propone il **transformer**: via la ricorrenza, resta solo l'attention, con calcolo più parallelo, addestramento più rapido e machine translation migliore; nel 2018 ne nasce BERT [slide Transformer pp. 2 e 52]. Da allora i language model crescono di quasi quattro ordini di grandezza, da ELMo (94M parametri, 2018) a BERT-Large (340M), GPT-2 (1.5B), GPT-3 (175B) e Megatron-Turing NLG (530B, 2021) [slide Transformer p. 4], in tre famiglie [slide Transformer p. 3]: auto-encoding (BERT), auto-regressive (GPT), seq2seq (BART). Il capitolo copre le domande 6 e 4 del programma; il confronto GPT/BERT (domanda 7) è nel capitolo 11.

#### 7.1 Dall'attenzione al self-attention layer

##### L'idea: ogni parola guarda le altre

Per classificare un testo con una feed-forward network (cap. 6) si possono concatenare gli embedding delle parole, ma con 512 parole di dimensione  $d = 100$  l'ingresso ha  $51\,200$  componenti e un hidden layer della stessa lunghezza costa  $51\,200^2 \approx 2.6$  miliardi di pesi; se invece si aggregano gli embedding si perdono le dipendenze fra le parole [slide Transformer p. 6]. L'idea: tenere gli embedding e aggiungere (molti) layer che considerano ogni parola *in relazione alle altre* e imparano su quali concentrarsi, il meccanismo di **attention** (attenzione) [slide Transformer p. 7]. La nuova rappresentazione di un token è una media pesata dei token che lo precedono, e pesa di più chi «si accorda» di più con lui: ogni token emette una *query* (che cosa cerco), espone una *key* (che cosa offro) e porta un *value* (ciò che consegna); il prodotto query·key dice quanto il token  $j$  serve al token  $i$ .

##### Self-attention layer, versione semplificata

Dati i vettori  $\mathbf{x}_1, \dots, \mathbf{x}_n$  dei token, il punteggio fra due token è il prodotto scalare  $score(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$ ; l'uscita  $\mathbf{y}_i$  è la media dei token fino a  $i$ , pesata con il softmax dei punteggi (il softmax, cap. 4, trasforma numeri qualsiasi in pesi positivi che sommano a 1) [slide Transformer p. 8]:

$$\alpha_{ij} = \text{softmax}(score(\mathbf{x}_i, \mathbf{x}_j)) = \frac{\exp(\mathbf{x}_i \cdot \mathbf{x}_j)}{\sum_{k=1}^i \exp(\mathbf{x}_i \cdot \mathbf{x}_k)} \quad (j \leq i), \quad \mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j;$$

in forma matriciale  $\mathbf{Y} = \text{softmax}(\mathbf{X}\mathbf{X}^T) \mathbf{X}$ , dove la matrice  $\mathbf{X}$  ha una riga per token.

##### Self-attention layer, versione dettagliata: query, key, value

Tre matrici di parametri  $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$  (ciascuna  $d \times d_k$ , con  $d_k$  la dimensione dei vettori  $\mathbf{q}, \mathbf{k}, \mathbf{v}$ ) proiettano ogni token in una *query*, una *key* e un *value* [slide Transformer p. 9]:

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q, \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K, \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V, \quad \alpha_{ij} = \text{softmax}\left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}\right), \quad \mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j.$$

In forma matriciale, con  $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$ ,  $\mathbf{K} = \mathbf{X}\mathbf{W}^K$ ,  $\mathbf{V} = \mathbf{X}\mathbf{W}^V$  [slide Transformer p. 10]:

$$\mathbf{Y} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}.$$

Tre note [slide Transformer p. 11]: (1) la parte triangolare superiore della matrice dei punteggi (le coppie  $j > i$ ) è «cancellata», cioè posta a  $-\infty$ , altrimenti ogni token guarderebbe anche i token che seguono; il softmax trasforma  $-\infty$  in peso 0 (*maschera causale*); (2) il softmax si calcola riga per riga; (3)  $\mathbf{W}^Q$ ,  $\mathbf{W}^K$ ,  $\mathbf{W}^V$  sono condivise fra tutte le posizioni. La divisione per  $\sqrt{d_k}$  evita che prodotti scalari grandi, esponentiati nel softmax, diano problemi numerici e gradienti nulli [dal libro, p. 186; non nelle slide].

**Considerazioni** [slide Transformer p. 12]. Il numero di parametri non dipende più dal numero di token: con 512 token e  $d = d_k = 100$  le tre matrici hanno  $3 \cdot 100^2 = 30\,000$  pesi, contro i circa 2.6 miliardi del layer fully connected; l'attenzione fra token è esplicita (la matrice  $\alpha$  si può leggere); il calcolo resta però quadratico nel numero di token, un limite dei transformer.

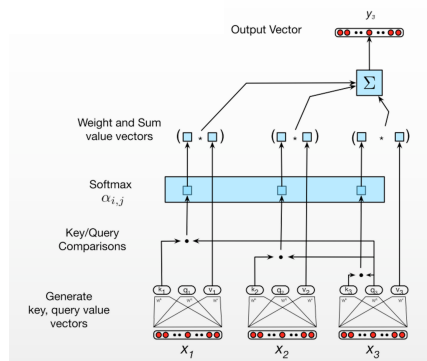


Figura 7.1: Il self-attention layer per il token  $\mathbf{x}_3$ : da ogni  $\mathbf{x}_j$  si generano  $\mathbf{k}_j, \mathbf{q}_j, \mathbf{v}_j$ ; si confrontano  $\mathbf{q}_3$  con  $\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3$ ; il softmax dà gli  $\alpha_{3j}$ , che pesano i  $\mathbf{v}_j$  sommati in  $\mathbf{y}_3$ .

Fonte: slide Transformer p. 9

**Esempio con 3 token e  $d = d_k = 2$**  (numeri scelti da noi [non nelle fonti del corso]). Siano  $\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$ ,  $\mathbf{W}^Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ ,  $\mathbf{W}^K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ ,  $\mathbf{W}^V = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ . Allora

$$\mathbf{Q} = \mathbf{X}, \quad \mathbf{K} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{V} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{Q}\mathbf{K}^T = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 3 \end{pmatrix}$$

(per esempio  $\mathbf{q}_3 \cdot \mathbf{k}_1 = (1, 1) \cdot (1, 1) = 2$ ). Si divide per  $\sqrt{2}$ , si maschera ( $j > i \rightarrow -\infty$ ) e si fa il softmax riga per riga:

$$\begin{pmatrix} 0.707 & -\infty & -\infty \\ 0.707 & 0.707 & -\infty \\ 1.414 & 0.707 & 2.121 \end{pmatrix} \rightarrow \alpha = \begin{pmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 0.284 & 0.140 & 0.576 \end{pmatrix}.$$

Terza riga:  $e^{1.414} = 4.113$ ,  $e^{0.707} = 2.028$ ,  $e^{2.121} = 8.342$ , somma 14.483, quindi  $\alpha_{31} = 4.113/14.483 = 0.284$ , ecc. Infine  $\mathbf{Y} = \alpha\mathbf{V}$ :  $\mathbf{y}_1 = \mathbf{v}_1 = (2, 0)$ ;  $\mathbf{y}_2 = 0.5\mathbf{v}_1 + 0.5\mathbf{v}_2 = (1, 0.5)$ ;  $\mathbf{y}_3 = 0.284\mathbf{v}_1 + 0.140\mathbf{v}_2 + 0.576\mathbf{v}_3 = (1.720, 0.716)$ . Il terzo token «guarda» soprattutto se stesso (0.576), poi il primo (0.284).

## 7.2 Il transformer block

### Perché il self-attention non basta da solo

Il self-attention layer mescola l'informazione *fra* i token; il feed-forward layer la rielabora *dentro* ciascun token, con la stessa piccola rete per tutti. Le residual connection (ingresso sommato all'uscita di ogni sottolivello) e la layer normalization tengono i valori stabili e danno al gradiente una via diretta, così si impilano decine di blocchi; più *head* in parallelo catturano relazioni diverse (fig. 7.2).

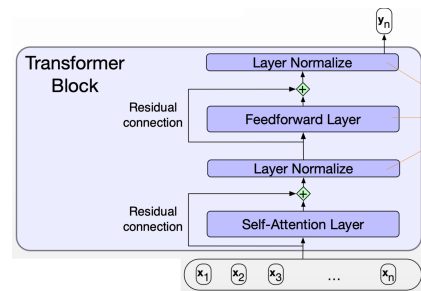


Figura 7.2: Il transformer block.

Fonte: slide Transformer p. 13

### Transformer block

Un blocco riceve i vettori  $\mathbf{x}_1, \dots, \mathbf{x}_n$  e restituisce altrettanti vettori  $\mathbf{y}_i$  della stessa dimensione; i blocchi si **impilano**, l'uscita di uno è l'ingresso del successivo [slide Transformer pp. 13 e 17]:

$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x})), \quad \mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z})).$$

Le somme  $\mathbf{x} + \dots$  e  $\mathbf{z} + \dots$  sono le *residual connection*; il feed-forward layer è calcolato per ogni token indipendentemente, con parametri condivisi.

### I pezzi del blocco

**Feed-forward layer** [slide Transformer p. 14]: due linear layer con ReLU in mezzo e nessuna attivazione alla fine, applicati a ogni token separatamente con parametri condivisi; l'hidden layer è di solito  $4d$ :  $\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$ .

**Layer normalization** [slide Transformer p. 15]: un vettore  $\mathbf{x}$  di dimensione  $d_h$  si ricentra e riscalda con la propria media  $\mu$  e deviazione standard  $\sigma$  (qui  $\sigma$  non è la sigmoide), poi si applicano un guadagno  $\gamma$  (gain) e un offset  $\beta$ , parametri appresi:

$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i, \quad \sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2}, \quad \hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sigma}, \quad \text{LayerNorm} = \gamma \hat{\mathbf{x}} + \beta.$$

**Multi-head attention** [slide Transformer p. 16]:  $h$  self-attention in parallelo (le *head*), ognuna con le proprie  $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V$  e uscita di dimensione  $d/h$ ; le uscite si concatenano ( $\oplus$ ) e si riproiettano a dimensione  $d$  con  $\mathbf{W}^O$ :

$$\text{head}_i = \text{SelfAttention}(\mathbf{X}\mathbf{W}_i^Q, \mathbf{X}\mathbf{W}_i^K, \mathbf{X}\mathbf{W}_i^V), \\ \text{MultiHeadAttention}(\mathbf{X}) = (\text{head}_1 \oplus \text{head}_2 \oplus \dots \oplus \text{head}_h) \mathbf{W}^O.$$

### La posizione: positional embedding

Il self-attention layer non tiene conto della posizione (tutti i token precedenti sono trattati allo stesso modo), ma la posizione delle parole conta per il significato [slide Transformer p. 18]. Rimedio: un embedding della posizione assoluta  $(1, 2, 3, \dots)$  *sommato* al word embedding prima del primo blocco [slide Transformer p. 19], appreso (ma per le ultime posizioni ci sono pochi dati) oppure dato da una funzione fissa dagli interi ai vettori che catturi le relazioni fra posizioni, per esempio seni e coseni che generalizzano la codifica binaria [slide Transformer p. 20].

## 7.3 Il transformer come modello di linguaggio

### Predire il token successivo

Come i modelli n-gram (cap. 2), Naive Bayes (cap. 3) e le feed-forward network (cap. 6), un transformer può fare da *language model* (modello di linguaggio): lo si addestra a predire il token successivo, cioè l'uscita in posizione  $t$  deve essere il token  $t + 1$  dell'ingresso [slide Transformer p. 21]. Grazie alla maschera causale ogni posizione vede solo il passato, e una frase di  $T$  token dà  $T$  esempi in una sola passata. Per generare si parte dal token di inizio, si predice il successivo, lo si aggiunge all'ingresso e si ripete.

### Architettura, loss e addestramento

Sopra la pila di transformer block, in ogni posizione, un *linear layer*  $\mathbf{yW} + \mathbf{b}$  (un punteggio per ogni parola del vocabolario) seguito da un softmax dà una distribuzione di probabilità sul vocabolario [slide Transformer p. 22]; la loss è la cross-entropy della regressione logistica (def. 4.3) con  $|V|$  classi,  $L_{CE}(\hat{\mathbf{y}}, y) = -\log p(y | x) = -\log \hat{\mathbf{y}}_y$ : meno il logaritmo (naturale) della probabilità predetta per la classe vera, qui la parola successiva [slide Transformer p. 23]. La loss di una frase è la media sulle posizioni,  $\frac{1}{T} \sum_{t=1}^T L_{CE}$ , e si usa il **teacher forcing**: in ogni posizione si dà in ingresso il token corretto, non quello predetto [slide Transformer p. 24].

### Generare una frase: campionamento, greedy, beam search

**Campionamento** (*sampling*) [slide Transformer p. 25]: si estrae una parola dal softmax ottenuto dando in ingresso il marcatore di inizio  $\langle s \rangle$ , la si dà in ingresso al passo successivo e si estrae la seguente, finché esce  $\langle /s \rangle$  o si raggiunge una lunghezza massima; le frasi probabili escono più spesso. Se invece si vuole *la frase più probabile* [slide Transformer p. 26]:

- approccio **greedy**:  $\hat{y}_t = \arg\max_{w \in V} P(w | y_1 \dots y_{t-1})$  a ogni passo  $\rightarrow$  impreciso;
- approccio **ottimo**: calcolare tutte le frasi  $\rightarrow$  impraticabile;
- **beam search**, un buon compromesso [slide Transformer p. 28]: (1) a ogni passo si tengono le  $k$  migliori ipotesi (sottosequenze),  $k$  è la *beam width*; (2) per ciascuna si predice il token successivo; (3) fra le  $k \cdot |V|$  possibilità si tengono le  $k$  più probabili e si ripete; il punteggio di un'ipotesi è la somma dei  $\log p$  dei suoi token.

**Limite del greedy** [slide Transformer p. 27]. Albero di ricerca con vocabolario  $\{\text{ok, yes, } \langle /s \rangle\}$ : da start,  $P(\text{ok}) = 0.4$ ,  $P(\text{yes}) = 0.5$ ,  $P(\langle /s \rangle) = 0.1$ ; dopo *ok*: 0.7 (ok), 0.2 (yes), 0.1 ( $\langle /s \rangle$ ); dopo *yes*: 0.3 (ok), 0.4 (yes), 0.3 ( $\langle /s \rangle$ ); al terzo passo  $\langle /s \rangle$  con probabilità 1.0. Il greedy sceglie *yes* (0.5) e poi *yes* (0.4): «yes yes» ha probabilità  $0.5 \cdot 0.4 \cdot 1.0 = 0.20$ , ma «ok ok» ha  $0.4 \cdot 0.7 \cdot 1.0 = 0.28$  ed è la più probabile: la scelta migliore al primo passo ha chiuso la strada alla migliore globale.

**Esempio di beam search con  $k = 2$**  [slide Transformer pp. 29–30]: si traduce una frase  $x$ ; sugli archi  $\log P(y_t | y_1 \dots y_{t-1}, x)$ , in verde la somma lungo il cammino (fig. 7.3).

1. Passo 1: le due parole più probabili sono *the* ( $-0.92$ ) e *arrived* ( $-1.6$ ).
2. Passo 2: si estendono entrambe; restano *the green* ( $-0.92 - 0.69 = -1.61$ ) e *the witch* ( $-0.92 - 1.2 = -2.12$ ); escono dal beam *arrived the* ( $-2.29$ ) e *arrived witch* ( $-3.9$ ).
3. Passo 3: *the green witch* ( $-1.61 - 0.51 = -2.12$ ) e *the witch arrived* ( $-2.12 - 0.11 = -2.23$ ); escono *the green mage* ( $-3.21$ ) e *the witch who* ( $-4.42$ ).
4. Passo 4: *the green witch arrived* ( $-2.12 - 0.36 = -2.48$ ) e l'ipotesi completa *the witch arrived* ( $-2.23 - 0.51 = -2.74$ ). Passo 5: *the green witch arrived* ( $-2.48 - 0.22 = -2.70$ ) ( $P = e^{-2.70} = 0.067$ ) batte  $-2.74$ : è la traduzione scelta.

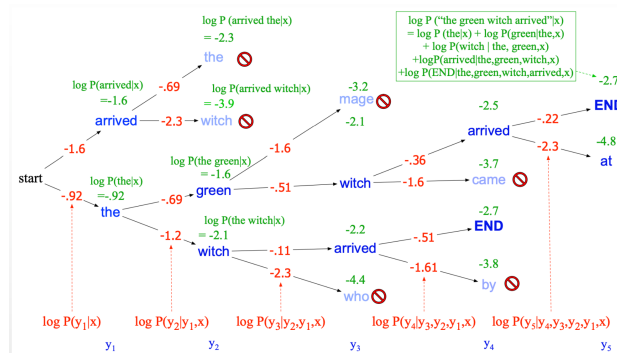


Figura 7.3: Beam search con  $k = 2$ : in rosso  $\log P$  del token, in verde la somma lungo il cammino. Fonte: slide Transformer p. 30

### Domanda 6: il ruolo dei transformer nei modelli neurali di linguaggio e il self-attention layer

1. **Ruolo.** Nasce con «Attention is all you need» (2017) al posto delle reti ricorrenti: solo attention, senza ricorrenza, quindi calcolo parallelo e addestramento rapido; da lì BERT e la crescita dei language model da  $10^8$  a oltre  $10^{11}$  parametri in pochi anni [slide Transformer pp. 2–4]. Blocchi impilati di self-attention e feed-forward trasformano gli embedding in vettori contestuali; come language model predice il token successivo (linear layer + softmax sul vocabolario, cross-entropy, teacher forcing) e genera token per token (sampling, greedy, beam search), con parametri indipendenti dal numero di token e attenzione esplicita; è la base dei modelli auto-regressivi (GPT), auto-encoding (BERT) e seq2seq (encoder-decoder: BART) [slide Transformer p. 3].
2. **Self-attention layer.** Ogni token  $\mathbf{x}_i$  produce  $\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q$ ,  $\mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K$ ,  $\mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$ ; il punteggio  $\mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d_k}$  misura quanto  $j$  serve a  $i$ ; nella versione causale i punteggi con  $j > i$  sono posti a  $-\infty$ ; il softmax riga per riga dà i pesi  $\alpha_{ij}$  e l'uscita è  $\mathbf{y}_i = \sum_j \alpha_{ij} \mathbf{v}_j$ ; in forma matriciale  $\mathbf{Y} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$ . Le tre matrici sono condivise fra le posizioni; con più head si catturano relazioni diverse.
3. **Nel blocco.**  $\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x}))$ ,  $\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z}))$ : residui, layer norm, feed-forward per token; i positional embedding sommati all'ingresso danno l'ordine.
4. **Esempio e limiti.** L'esempio con 3 token e  $d = 2$  della sez. 7.1; il calcolo è quadratico nel numero di token; senza maschera un language model «vedrebbe» la risposta.

## 7.4 BERT: un encoder bidirezionale pre-addestrato

### Leggere in entrambe le direzioni

Un language model causale legge da sinistra a destra: il token  $i$  vede solo i precedenti. Per capire un testo (classificarlo, riconoscerne le named entity, rispondere a domande) conviene che ogni token veda tutta la frase, anche ciò che segue. Ma allora predire la parola successiva è banale (è già in ingresso), e BERT cambia compito: nasconde alcune parole e chiede di indovinarle dal contesto (*cloze task*), e chiede se due frasi sono consecutive (*next sentence prediction*). Così impara da testo non annotato (*pre-training*); poi, con pochi dati etichettati, lo si adatta al compito con un piccolo layer in cima (*fine-tuning*): è *transfer learning*, si trasferisce a un nuovo compito ciò che si è imparato in un altro.

### BERT e il suo pre-training

**BERT** (Bidirectional Encoder Representations from Transformers) [slide Transformer p. 31]: una pila di  $L = 12/24$  transformer block con  $A = 12/16$  attention head e hidden size  $H = 768/1024$ , pre-addestrata su cloze task e next sentence prediction, poi fine-tuning sul compito specifico. Il self-attention è **bidirezionale**: senza maschera, la matrice di attenzione  $N \times N$  contiene tutti i  $\mathbf{q}_i \cdot \mathbf{k}_j$  e  $\text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$  [slide Transformer p. 32].

**Cloze task, o masked language model** [slide Transformer pp. 33–35]: si predice una parola mancante invece della successiva. Si sceglie il 15% dei token; di questi, l'80% è sostituito dal token speciale [MASK], il 10% da un token casuale, il 10% resta invariato. Nelle posizioni scelte un linear layer  $\mathbf{W}_V + \text{softmax}$  sul vocabolario predice il token originale, con loss cross-entropy  $-\log y_{\text{parola}}$  solo in quelle posizioni.

**Next sentence prediction (NSP)** [slide Transformer pp. 36–37]: date due frasi, predire se nel testo originale la seconda segue la prima: il 50% delle coppie sono consecutive (etichetta

*IsNext*), il 50% scelte a caso (*NotNext*). Token speciali [CLS] (all'inizio) e [SEP] (fra le frasi e alla fine); il vettore d'uscita di [CLS] passa per  $\mathbf{W}_{\text{NSP}} + \text{softmax}$  a due classi, con loss cross-entropy.

**Ingresso** [slide Transformer p. 39]: ogni token (*subword*, pezzo di parola) è la somma di token embedding + segment embedding ( $E_A$  o  $E_B$ , la frase di appartenenza) + position embedding; le due loss si addestrano insieme sulla stessa coppia di frasi mascherate (fig. 7.4) [slide Transformer p. 38].

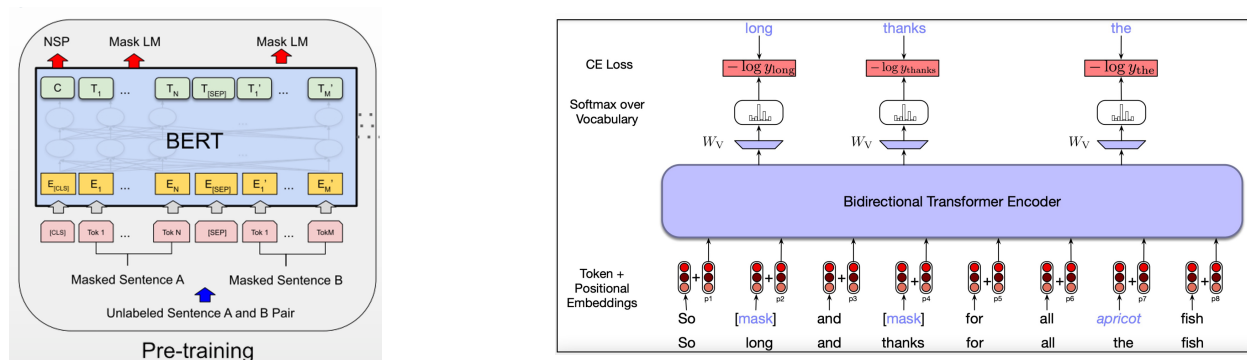


Figura 7.4: Pre-training di BERT: a sinistra la testa NSP sul vettore di [CLS] e le teste Mask LM; a destra il cloze task predice *long* e *thanks* al posto di [mask] e *the* al posto del token casuale *apricot*.

Fonte: slide Transformer pp. 38 e 35

### Usare BERT: embedding contestuali, fine-tuning, classificatori

I vettori dell'ultimo layer sono **embedding contestuali**: rappresentano il significato della parola *in quel contesto* e possono sostituire gli embedding statici (word2vec, cap. 5); questa capacità si trasferisce ad altri compiti (**transfer learning**) con il **fine-tuning** [slide Transformer p. 40]: una piccola testa in cima e si continua l'addestramento sui dati del compito [slide Transformer p. 41]: MNLI (*natural language inference*: decidere se una frase implica, contraddice o è neutra rispetto a un'altra [dal libro, p. 220; non nelle slide]), NER (named entity recognition), SQuAD (question answering).

- **Classificatore** [slide Transformer p. 42]: ingresso [CLS]  $w_1 \dots w_n$  [SEP]; il vettore d'uscita di [CLS] entra in un linear layer + softmax che dà la classe (nell'esempio *Care*).
- **Multi-label** [slide Transformer pp. 43–44]: zero, una o più classi non esclusive per testo, etichetta = vettore binario (sez. 3.3); la testa è linear layer + *sigmoide*, una per classe.
- **Senza fine-tuning** [slide Transformer p. 45]: i vettori di BERT si usano come feature, sommando o concatenando l'uscita degli ultimi quattro hidden layer (NER CoNLL-2003: F1 94.9 con il solo ultimo layer, 96.1 concatenando gli ultimi quattro).

### Domanda 4: descrivere BERT e la sua strategia di pre-training

1. **Che cos'è.** BERT è un *encoder* transformer bidirezionale: 12/24 blocchi, 12/16 head,  $H = 768/1024$ ; il self-attention non ha maschera, ogni token attende a tutti i token della sequenza, precedenti e seguenti; l'uscita è un embedding contestuale per token.
2. **Perché non il token successivo.** Con attenzione bidirezionale predire la parola seguente è banale (è già in ingresso): serve un compito che non riveli la risposta.
3. **Cloze / masked LM.** Si sceglie il 15% dei token (80%  $\rightarrow$  [MASK], 10%  $\rightarrow$  token casuale, 10% invariato) e si predicono i token originali in quelle posizioni con softmax sul vocabolario e cross-entropy. Il token casuale e quello invariato servono perché nel

fine-tuning [MASK] non compare mai: il modello deve rappresentare bene ogni token [dal libro, p. 211; non nelle slide].

4. **NSP.** Coppie di frasi, 50% consecutive (IsNext) e 50% casuali (NotNext), con [CLS] in testa e [SEP] fra le frasi; la predizione esce dal vettore di [CLS] con un classificatore a due classi. Ingresso = token + segment + position embedding.
5. **Dopo il pre-training.** Le etichette sono gratis (le parole stesse), quindi bastano grandi corpus non annotati; poi transfer learning: fine-tuning con una piccola testa per compito (classificazione da [CLS], multi-label, NLI, NER, QA) o uso diretto come embedding contestuali.
6. **Limiti e collegamenti.** Solo il 15% dei token contribuisce alla loss: pre-training poco efficiente [dal libro, p. 212; non nelle slide]; BERT non è fatto per generare testo (non è causale), al contrario di GPT: confronto nel capitolo 11.

## 7.5 Architetture encoder-decoder

### Prima leggere tutto, poi scrivere

Nella machine translation l'ordine delle parole può cambiare (*The green witch arrived* → *Llegó la bruja verde*) e a volte bisogna aver letto tutta la frase prima di cominciare a tradurre [slide Transformer p. 46]. Si separano i due compiti: un *encoder* bidirezionale legge tutto l'ingresso; un *decoder* causale genera l'uscita parola per parola, consultando a ogni passo l'encoder con la *cross-attention*.

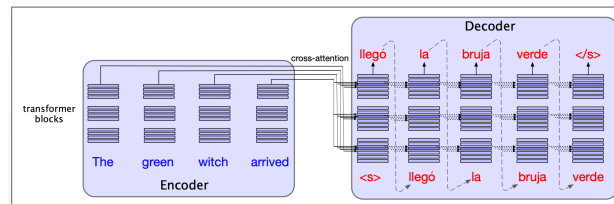


Figura 7.5: Encoder-decoder per la machine translation: il decoder genera un token alla volta, attendendo ai propri token precedenti e, con la cross-attention, a tutti quelli dell'encoder.

Fonte: slide Transformer p. 48

### Encoder-decoder

Obiettivo: la sequenza nella lingua di arrivo che massimizza  $P(y_1, \dots, y_m \mid x_1, \dots, x_n)$  [slide Transformer p. 46]. Dall'ingresso  $x$  si calcola una rappresentazione intermedia  $\mathbf{h}$  (encoding) e da  $\mathbf{h}$  si genera parola per parola la sequenza  $y$  (decoding):  $\mathbf{h} = \text{encoder}(x)$ ,  $y_{i+1} = \text{decoder}(\mathbf{h}, y_1, \dots, y_i)$  per  $i \in [1, \dots, m]$  [slide Transformer p. 47]. L'uscita dell'ultimo layer dell'encoder va al decoder, simile all'encoder ma con in più, in ogni blocco, un **cross-attention layer** che attende ai token dell'encoder [slide Transformer p. 48]. Nell'encoder il self-attention è completo (token precedenti e seguenti), nel decoder è causale (solo i precedenti); la cross-attention prende le query dal decoder, key e value dall'encoder, e calcola il solito CrossAttention( $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ ) =  $\text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$  [slide Transformer p. 49]:

$$\mathbf{Q} = \mathbf{W}^Q \mathbf{H}^{\text{dec}[i-1]}, \quad \mathbf{K} = \mathbf{W}^K \mathbf{H}^{\text{enc}}, \quad \mathbf{V} = \mathbf{W}^V \mathbf{H}^{\text{enc}},$$

dove  $\mathbf{H}^{\text{enc}}$  è l'uscita dell'ultimo layer dell'encoder e  $\mathbf{H}^{\text{dec}[i-1]}$  quella del blocco precedente del decoder (qui  $i$  numera i blocchi, non le parole). **Addestramento** [slide Transformer p. 50]: come per un language model sulla lingua di arrivo: data la coppia  $(x, y)$  si dà  $x$  in ingresso e si calcola su  $y$  la cross-entropy  $L_{\text{CE}}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$ , con teacher forcing. **Predizione**

[slide Transformer p. 51]: un token alla volta, aggiunto all'ingresso del decoder per generare il successivo; con sampling, greedy o beam search.

#### Confusioni tipiche

- **BERT non genera testo:** è un encoder bidirezionale addestrato a riempire buchi; per generare servono modelli causali (GPT) o encoder-decoder. La **maschera** a  $-\infty$  c'è solo nel self-attention causale (language model, decoder), non nell'encoder.
- Il **softmax** dell'attenzione è per riga ( $\sum_j \alpha_{ij} = 1$  per ogni  $i$ ); il **feed-forward layer** lavora su un token alla volta: a mescolare i token è solo il self-attention.
- I **positional embedding** si *sommano* al word embedding, non si concatenano; senza di essi il modello vedrebbe un insieme di parole senza ordine.
- **Greedy  $\neq$  più probabile** («yes yes» 0.20 contro «ok ok» 0.28); la beam search tiene  $k$  ipotesi ma non garantisce l'ottimo.

## Capitolo 8

### POS tagging, Named Entity Recognition ed entity linking

#### 8.1 Parti del discorso e POS tagging

##### L'idea

Le parole si raggruppano in classi grammaticali (nome, verbo, aggettivo...) che dicono come si comportano nella frase. Molte parole stanno in più classi: *book* è verbo in *Book that flight* e nome in *Hand me that book*. Il POS tagging *disambigua*: sceglie la classe giusta nel contesto. È un *sequence labeling* (tagging di sequenze): un tag per parola.

##### Classi, POS tagging e tagset

Le **classi chiuse** (*function words*: determinanti, pronomi, preposizioni, ausiliari) hanno membri fissi: parole brevi e frequenti; le **classi aperte** (*content words*: nomi, verbi, aggettivi, avverbi) accolgono parole nuove (*iPhone, to fax*) [slide POS-NER pp. 4-5]. Il **POS tagging** mappa la sequenza di parole  $x_1, \dots, x_n$  nella sequenza di tag  $y_1, \dots, y_n$ , un tag per parola [slide POS-NER p. 7]. Il **tagset** (l'insieme dei tag) di *Universal Dependencies* ha 17 tag [slide POS-NER p. 8]: 6 aperti (ADJ, ADV, NOUN, VERB, PROPN nome proprio, INTJ interiezione), 8 chiusi (ADP, AUX, CCONJ, SCONJ, DET, NUM, PART particella, PRON) e 3 «altro» (PUNCT, SYM, X). Esempio [slide POS-NER p. 9]: *There/PRON were/VERB 70/NUM children/NOUN there/ADV ./PUNCT*.

**Difficoltà, prestazioni, fonti di informazione.** Solo il 15% circa dei *tipi* di parola è ambiguo (l'85% ha un tag solo, come *Janet*, sempre PROPN), ma sono parole comunissime: circa il 60% dei *token* è ambiguo [slide POS-NER p. 11]; *back* compare con cinque tag (*back/ADJ seat, the back/NOUN, senators back/VERB, buy back/PART, back/ADV then*). L'accuratezza è circa il 97% per l'inglese, uguale per HMM, CRF, BERT e annotatori umani, ma la *most frequent class baseline* (il tag più frequente di ogni parola) fa già il 92% [slide POS-NER p. 12]. Le **fonti di informazione** sono tre [slide POS-NER p. 13]: la probabilità a priori parola/tag (*will*: di solito AUX), le parole vicine (dopo *the* di rado un verbo), morfologia e forma (*un-* → ADJ, *-ly* → ADV, maiuscola → PROPN). Serve a parsing, machine translation, sentiment analysis e text-to-speech [slide POS-NER p. 10].

##### L'idea: gli algoritmi di tagging

Gli algoritmi standard: Hidden Markov Model, Conditional Random Field (CRF) o Maximum Entropy Markov Model (MEMM), reti neurali di sequenza (RNN o transformer), BERT con fine-tuning. Tutti supervisionati (corpus annotato a mano) e tutti al 97%: HMM e CRF con feature scritte a mano, le reti imparando la rappresentazione [slide POS-NER p. 14]. L'HMM ragiona «al contrario»: immagina che i tag, nascosti, generino le parole, e cerca i tag che la spiegano meglio.

##### HMM e Viterbi in breve

Un **Hidden Markov Model** ha due tabelle stimate contando su un corpus annotato: le probabilità di *transizione*  $P(t_i | t_{i-1}) = C(t_{i-1}, t_i) / C(t_{i-1})$  e di *emissione*  $P(w_i | t_i) = C(t_i, w_i) / C(t_i)$ . Nel corpus WSJ il tag MD (verbo modale) compare 13124 volte, 10471 seguito da VB (verbo base) e 4046 con *will*:  $P(VB | MD) = 10471 / 13124 = 0.80$  e  $P(will | MD) = 4046 / 13124 = 0.31$  [dal libro, p. 399; non nelle slide]. Il **decoding** cerca la

sequenza di tag più probabile; con Bayes e due assunzioni (Markov: ogni tag dipende solo dal precedente; ogni parola solo dal suo tag) diventa [dal libro, p. 400; non nelle slide]

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} \prod_{i=1}^n \underbrace{P(w_i | t_i)}_{\text{emissione}} \underbrace{P(t_i | t_{i-1})}_{\text{transizione}}.$$

L'algoritmo di **Viterbi** la trova per programmazione dinamica: in una tabella parole  $\times$  tag,  $v_t(j) = \max_i v_{t-1}(i) a_{ij} b_j(o_t)$  è la probabilità del miglior cammino che arriva al tag  $j$  dopo  $t$  parole ( $a_{ij}$ : transizione da  $i$  a  $j$ ;  $b_j(o_t)$ : emissione di  $o_t$  dal tag  $j$ ; all'inizio  $v_1(j) = \pi_j b_j(o_1)$ , con  $\pi_j$  probabilità iniziale); ogni cella ricorda il tag di provenienza e alla fine si risale dal massimo dell'ultima colonna. Per *Janet will back the bill* [dal libro, pp. 393, 402–404; non nelle slide]: *Janet* è solo NNP, quindi nella prima colonna solo  $v_1(\text{NNP}) = \pi_{\text{NNP}} b_{\text{NNP}}(\text{Janet}) = 0.28 \times 0.000032 \approx 9 \times 10^{-6}$  è non nullo; poi  $v_2(\text{MD}) = 9 \times 10^{-6} \times a_{\text{NNP,MD}} b_{\text{MD}}(\text{will}) = 9 \times 10^{-6} \times 0.01 \times 0.308 = 2.772 \times 10^{-8}$ , e così via fino a NNP MD VB DT NN (tag del Penn Treebank; in UD: PROPN AUX VERB DET NOUN). Il **CRF** è la variante discriminativa: calcola direttamente  $P(Y | X)$  con feature arbitrarie [dal libro, p. 404; non nelle slide].

**Il tagger basato su BERT** [slide POS-NER p. 15]. La frase entra fra [CLS] e [SEP]; l'uscita di ogni token passa in un *linear layer* con softmax sui tag (stessa frase: PROPN AUX VERB DET NOUN); fine-tuning sul corpus annotato (capitolo 7); di una parola spezzata in subword (sotto-parole) si usa solo il primo token. Rispetto all'HMM: niente feature a mano né assunzioni di indipendenza.

## 8.2 Named Entity Recognition e schema BIO

### L'idea

Per il POS tagging *Janet*, *Stanford University* e *Colorado* sono nomi propri, non persona, organizzazione e luogo: la NER aggiunge il livello semantico. È più difficile perché un'entità può occupare più parole: lo schema BIO riduce la ricerca dei confini a un tag per parola.

### Named entity e NER

Una **named entity** è qualunque cosa designata da un nome proprio; i quattro tipi più comuni sono PER (*Marie Curie*), LOC (*New York City*), ORG (*Stanford University*) e GPE, entità geopolitica (*Boulder*, *Colorado*); per estensione date, orari e prezzi [slide POS-NER p. 16]. La **NER** trova gli *span* (segmenti) che sono nomi propri e ne dà il tipo [slide POS-NER pp. 17–18]. Serve a sentiment analysis, question answering e information extraction (estrarre dal testo fatti sulle entità) [slide POS-NER p. 19]. È difficile per la **segmentazione** (i confini) e per l'**ambiguità di tipo**: *Washington* può essere PER, ORG, LOC o GPE [slide POS-NER p. 20].

Nel **BIO tagging** (schema BIO) ogni token riceve **B**-tipo se inizia uno span, **I**-tipo se è dentro uno span, **O** se è fuori da ogni span; con  $n$  tipi i tag sono  $2n + 1$  (9 con quattro tipi) [slide POS-NER p. 23]. Stessa informazione delle parentesi, ma «un tag per token»: la NER diventa sequence labeling e usa gli stessi algoritmi del POS tagging [slide POS-NER pp. 22, 25]. Varianti [slide POS-NER p. 24]: **IO** elimina il B ( $n + 1$  tag, perde informazione); **BIOES** aggiunge **E** per l'ultimo token di uno span e **S** per gli span di una parola ( $4n + 1$  tag); figura 8.1.

| Parola     | IO    | BIO   | BIOES |
|------------|-------|-------|-------|
| Jane       | I-PER | B-PER | B-PER |
| Villanueva | I-PER | I-PER | E-PER |
| of         | O     | O     | O     |
| United     | I-ORG | B-ORG | B-ORG |
| Airlines   | I-ORG | I-ORG | E-ORG |
| Holding    | I-ORG | I-ORG | E-ORG |
| discussed  | O     | O     | O     |
| the        | O     | O     | O     |
| Chicago    | I-LOC | B-LOC | S-LOC |
| route      | O     | O     | O     |
| .          | O     | O     | O     |

Figura 8.1: Esempio BIO e varianti IO, BIOES. Fonte: slide POS-NER pp. 22–24

### 8.3 Entity linking: BLINK, bi-encoder e cross-encoder

#### L'idea

La NER dice che *Washington* è un luogo, non *quale*: l'entity linking sceglie la voce giusta di una base di conoscenza (Wikipedia). BLINK confronta *testi*: codifica con BERT la menzione nel contesto e la descrizione dell'entità, con un passo veloce che seleziona pochi candidati e uno accurato che sceglie fra quelli; un'entità nuova basta descriverla (*zero-shot*).

#### Entity linking e BLINK

**Entity linking** (*named entity disambiguation*): dati un documento e le sue menzioni di entità, associare ogni menzione all'entità corrispondente in una base di conoscenza. Come classificazione sulle entità avrebbe tre problemi: un numero enorme di classi (label), entità mai viste in addestramento, menzioni interdipendenti (*the Iraq war when Bush was president*: 2003 e George W., oppure 1990 e George H. W. Bush) [slide POS-NER p. 28]. **BLINK** (Wu et al., 2020) è un linker zero-shot in due stadi [slide POS-NER p. 29], figura 8.2.

- **Bi-encoder** [slide POS-NER pp. 30–32]: due transformer separati (per esempio BERT) codificano la menzione  $m_i$  (il suo contesto, con la menzione delimitata dai token speciali [Ms] e [Me]) e l'entità  $e_i$ ; le uscite del token [CLS] sono  $\mathbf{y}_m$  e  $\mathbf{y}_e$  e il punteggio è il prodotto scalare  $s(m, e_i) = \mathbf{y}_m \cdot \mathbf{y}_{e_i}$ . Si addestra su coppie corrette  $(m_i, e_i)$  con la loss  $\mathcal{L}(m_i, e_i) = -s(m_i, e_i) + \log \sum_{j=1}^B \exp s(m_i, e_j) = -\log \text{softmax}([s(m_i, e_j)]_j)[i]$ , dove le  $e_j$  sono le  $B$  entità del batch: la corretta e le altre, che fanno da negativi casuali (meglio se difficili, simili a quella giusta: *hard negative sampling*). In inferenza si prendono le  $K$  entità con  $\mathbf{y}_e$  più vicino a  $\mathbf{y}_m$  (vettori pre-calcolati, ricerca approssimata con FAISS) per il secondo stadio.
- **Cross-encoder** [slide POS-NER pp. 33–34]: un solo transformer legge menzione ed entità concatenate in un unico input e produce  $\mathbf{y}_{m,e}$ ; punteggio  $s_{\text{cross}}(m, e) = \mathbf{y}_{m,e} \mathbf{W}$ , stessa softmax loss ma sui  $K$  candidati del bi-encoder; in inferenza vince l' $e_j$  con  $s_{\text{cross}}$  massimo.

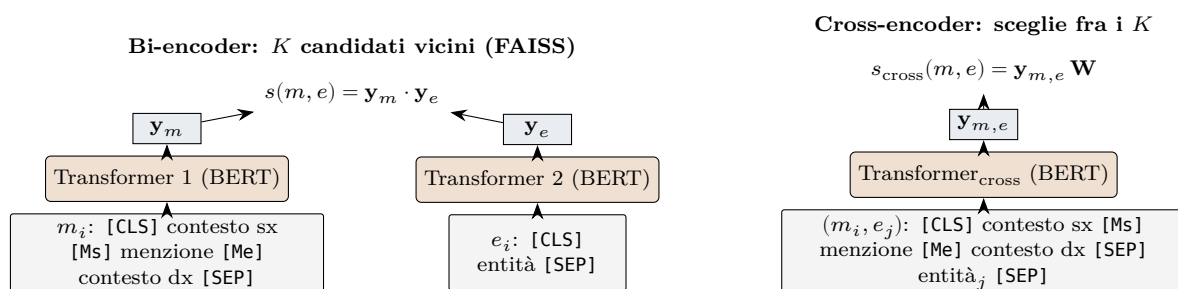


Figura 8.2: BLINK: bi-encoder e cross-encoder. Fonte: slide POS-NER pp. 30, 33

#### Descrivere il POS tagging e il NER come sequence labeling

- (1) Definizione: un tag  $y_i$  per ogni parola  $x_i$ ; nel POS tagging le parti del discorso (tagset UD a 17 tag, classi aperte e chiuse), nella NER il tipo di entità (PER, LOC, ORG, GPE).
- (2) Difficoltà: ambiguità (*back* con cinque tag; 15% dei tipi ma 60% dei token), e per la NER anche segmentazione e ambiguità di tipo (*Washington*).
- (3) BIO tagging: B/I/O per tipo,  $2n + 1$  tag, esempio *Jane Villanueva*, varianti IO e BIOES.
- (4) Algoritmi (comuni ai due compiti): HMM (transizione ed emissione, Viterbi), CRF, reti neurali, BERT con linear layer e softmax.
- (5) Prestazioni: 97% contro il 92% della baseline.

### Confrontare bi-encoder e cross-encoder nell'entity linking

(1) Problema: collegare ogni menzione a un'entità della base di conoscenza. (2) Bi-encoder: due BERT separati, vettori [CLS]  $\mathbf{y}_m$  e  $\mathbf{y}_e$ , punteggio  $\mathbf{y}_m \cdot \mathbf{y}_e$ , loss  $-\log \text{softmax}$  con negativi nel batch; vettori delle entità pre-calcolati e ricerca veloce dei  $K$  vicini (FAISS), ma menzione ed entità non si «vedono». (3) Cross-encoder: un BERT legge la coppia concatenata, punteggio  $\mathbf{y}_{m,e} \mathbf{W}$ ; l'attenzione incrocia le due parti: più preciso, ma va rieseguito per ogni coppia, quindi solo sui  $K$  candidati. (4) BLINK: il bi-encoder filtra, il cross-encoder decide; zero-shot: un'entità nuova basta descriverla.

### Errore tipico da evitare

- PROPEN è un tag POS (proprietà grammaticale), PER/ORG/LOC sono tipi di entità (proprietà semantica): *Colorado* è PROPEN per il tagger e LOC o GPE per la NER.
- In BIO un I-X può seguire solo B-X o I-X dello stesso tipo (*O I-PER* non è valido); i tag sono  $2n + 1$ , non  $2n$ . E  $P(\textit{will} \mid \text{MD})$  è un'emissione, non  $P(\text{MD} \mid \textit{will})$ .

## Capitolo 9

### Parser sintattici: costituenti, CKY e dipendenze

#### 9.1 Perché la sintassi: grammatiche a costituenti

##### L'idea

Il POS tagging (capitolo 8) dice che cos'è ogni parola, non come le parole stanno insieme; il *parsing* sintattico (analisi sintattica) ricostruisce la struttura della frase cercando gli alberi che una grammatica, cioè un elenco finito di regole, le assegna. Due modi di descriverla [slide Parser sintattici p. 3]: le **constituency grammar** usano i *costituenti*, gruppi di parole che si comportano come un'unità (*a morning flight* è un *noun phrase*, sintagma nominale, sostituibile con *it*) e si annidano in un albero; le **dependency grammar** usano le *dipendenze*, relazioni binarie dirette fra parole (sezione 9.3).

##### Context-free grammar, lessico, albero di parsing

Una **Context-Free Grammar** (CFG, grammatica libera dal contesto) è un insieme di regole o produzioni  $A \rightarrow \beta$ :  $A$  è un simbolo *non terminale* (una categoria: NP sintagma nominale, VP verbale, PP preposizionale, Det, Noun...),  $\beta$  una sequenza di non terminali e di *terminali*, cioè le parole [slide Parser sintattici p. 4]. Le regole con sole parole a destra sono il **lessico** ( $Det \rightarrow a \mid the$ ) [slide Parser sintattici pp. 4, 6]; il simbolo *iniziale*  $S$  è la frase:  $S \rightarrow NP VP$ ;  $VP \rightarrow Verb NP \mid Verb NP PP \mid Verb PP$ ;  $PP \rightarrow Preposition NP$ ;  $NP \rightarrow Det Nominal \mid ProperNoun$ ;  $Nominal \rightarrow Noun \mid Nominal Noun$  [slide Parser sintattici pp. 4–5]. L'**albero di parsing** (*parse tree*) ha  $S$  alla radice, le parole alle foglie e in ogni nodo interno una regola applicata (figura 9.1); in *bracketed notation* lo stesso albero si scrive con parentesi etichettate:  $[S [NP [Pro I]] [VP [V prefer] [NP [Det a] [Nom [Nom [N morning]] [Noun flight]]]]]$  [slide Parser sintattici p. 7].

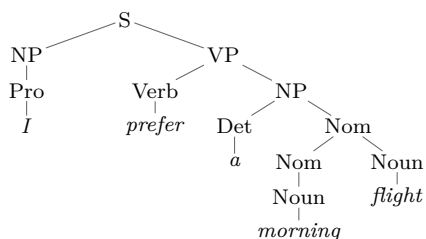


Figura 9.1: Albero di parsing di *I prefer a morning flight*. Fonte: slide Parser sintattici p. 7

**Ambiguità.** La stessa frase può avere più alberi: in *I shot an elephant in my pajamas* il PP *in my pajamas* si attacca al *Nominal elephant* (l'elefante in pigiama) oppure al *VP shot* (io in pigiama), e la grammatica genera tutti e due gli alberi [slide Parser sintattici p. 8]. Le frasi reali hanno molti alberi grammaticali ma senza senso [dal libro, p. 425; non nelle slide]; per non rianalizzare gli stessi pezzi in ogni combinazione serve un algoritmo che costruisca ogni costituente una volta sola: CKY, basato sulla programmazione dinamica.

#### 9.2 Forma normale di Chomsky e algoritmo CKY

##### L'idea

CKY compila una tabella: per ogni segmento della frase scrive quali categorie lo coprono, partendo dalle singole parole (il lessico) e salendo verso segmenti più lunghi, ciascuno spezzato in due segmenti più corti già calcolati: così il NP *the flight* si analizza una volta sola e si riusa in ogni albero. Perché «spezzare in due» basti sempre, le regole fra categorie devono avere due simboli a destra: è la forma normale di Chomsky.

### Chomsky Normal Form

Una CFG è in **Chomsky Normal Form** (CNF) se ogni regola ha la forma  $A \rightarrow BC$  oppure  $A \rightarrow w$ , con  $A, B, C$  non terminali e  $w$  terminale; le regole vietate si sostituiscono [slide Parser sintattici p. 9]: una regola lunga  $A \rightarrow BCD$  diventa  $A \rightarrow BX$ ,  $X \rightarrow CD$  con un nuovo simbolo  $X$ ; una *produzione unitaria*  $A \rightarrow B$  con  $B \rightarrow w$  diventa  $A \rightarrow w$  ( $A$  eredita i lati destri non unitari di  $B$  [dal libro, p. 426; non nelle slide]). La grammatica convertita genera le stesse frasi [dal libro, p. 425; non nelle slide]. Nella grammatica  $\mathcal{L}_1$  delle slide [slide Parser sintattici pp. 10–11]:  $S \rightarrow Aux NP VP$  diventa  $S \rightarrow X1 VP$ ,  $X1 \rightarrow Aux NP$ ;  $VP \rightarrow Verb NP PP$  diventa  $VP \rightarrow X2 PP$ ,  $X2 \rightarrow Verb NP$ ;  $S \rightarrow VP$  sparisce e  $S$  eredita  $S \rightarrow Verb NP \mid X2 PP \mid Verb PP \mid VP PP \mid book \mid include \mid prefer$ .

### L'algoritmo CKY (Cocke-Kasami-Younger)

Programmazione dinamica su una grammatica in CNF [slide Parser sintattici p. 12]. Frase di  $n$  parole e posizioni  $0, \dots, n$  fra le parole ( $_0$  Book<sub>1</sub> the<sub>2</sub> flight<sub>3</sub>); la cella  $[i, j]$  di una tabella triangolare contiene i non terminali che coprono le parole da  $i + 1$  a  $j$  [dal libro, p. 426; non nelle slide]. Passi [slide Parser sintattici p. 15]:

1. per ogni parola  $j = 1, \dots, n$  metti in  $[j - 1, j]$  tutti gli  $A$  con  $A \rightarrow words[j]$  (la diagonale);
2. nella stessa colonna  $j$  sali: per  $i = j - 2, \dots, 0$  e per ogni punto di taglio  $k$ ,  $i < k < j$ , se  $B \in [i, k]$ ,  $C \in [k, j]$  e  $A \rightarrow BC$  è una regola, aggiungi  $A$  a  $[i, j]$  (colonne da sinistra a destra e ciascuna dal basso verso l'alto, così  $[i, k]$  sulla riga e  $[k, j]$  nella colonna sono già pronte [dal libro, p. 427; non nelle slide]);
3. la frase è generata se  $S \in [0, n]$ ; per avere gli alberi, e non solo il sì/no, ogni  $A$  ricorda da quali celle e regola deriva, e seguendo i puntatori da  $[0, n]$  si ricostruiscono tutti gli alberi [slide Parser sintattici p. 14].

|   | <i>Book</i><br>$j = 1$         | <i>the</i><br>$j = 2$ | <i>flight</i><br>$j = 3$ | <i>through</i><br>$j = 4$ | <i>Houston</i><br>$j = 5$ |
|---|--------------------------------|-----------------------|--------------------------|---------------------------|---------------------------|
| 0 | S, VP,<br>Verb<br>Nom,<br>Noun |                       | S, VP, X2                |                           | S, VP, X2                 |
|   |                                | Det                   | NP                       |                           | NP                        |
|   |                                |                       | Nom,<br>Noun             |                           | Nom                       |
|   |                                |                       |                          | Prep                      | PP                        |
|   |                                |                       |                          |                           | NP,<br>Proper-<br>Noun    |

Figura 9.2: Tabella CKY di *Book the flight through Houston*; Nom = Nominal, Prep = Preposition. Fonte: slide Parser sintattici p. 13

**L'esempio passo per passo** ( $\mathcal{L}_1$  in CNF). *Diagonale*: *Book* è *Verb* e *Noun*, e per le regole ereditate anche *VP*, *S* e *Nom*; le altre quattro celle sono in figura. *Colonna 3*: *Det*  $[1, 2]$  + *Nom*  $[2, 3]$  dà *NP*  $\in [1, 3]$ ; *Verb*  $[0, 1]$  + *NP*  $[1, 3]$  dà *S, VP, X2*  $\in [0, 3]$ ;  $[0, 2]$  resta vuota (nessuna regola combina *Verb* e *Det*), come  $[0, 4]$ ,  $[1, 4]$  e  $[2, 4]$  (nessuna regola finisce con *Prep*). *Colonna 5*: *Prep*  $[3, 4]$  + *NP*  $[4, 5]$  dà *PP*  $\in [3, 5]$ ; *Nom*  $[2, 3]$  + *PP*  $[3, 5]$  dà *Nom*  $\in [2, 5]$ ; *Det*  $[1, 2]$  + *Nom*  $[2, 5]$  dà *NP*  $\in [1, 5]$ ; infine  $[0, 5]$  riceve *S, VP, X2* da due tagli: *Verb*  $[0, 1]$  + *NP*  $[1, 5]$  (il PP modifica *flight*) e *VP*  $[0, 3]$  o *X2*  $[0, 3]$  + *PP*  $[3, 5]$  (il PP modifica il verbo).  $S \in [0, 5]$ : frase generata; i puntatori danno tre alberi distinti [dal libro, p. 428; non nelle slide].

## 9.3 Dependency parsing

### L'idea

Invece di raggruppare le parole in sintagmi, si collega ogni parola alla parola da cui «dipende»: *the* e *morning* dipendono da *flight*, *flight* e *I* dal verbo *prefer*. Niente nodi NP o VP: solo parole e archi etichettati con la funzione grammaticale (soggetto, oggetto, determinante), più vicini al significato.

### Dependency tree, relazioni, proiettività

Il *dependency parsing* (analisi a dipendenze) produce un **dependency tree**: i nodi sono le parole, ogni arco va da un genitore, la **head** (testa), a un figlio, il **dependent**, e porta un'etichetta, il **tipo di relazione**: la funzione grammaticale del dipendente [slide Parser sintattici p. 17]. Le etichette standard sono di **Universal Dependencies** [slide Parser sintattici p. 19]: relazioni *clausali*, ruolo sintattico rispetto a un predicato (NSUBJ soggetto nominale, DOBJ oggetto diretto, IOBJ oggetto indiretto), e relazioni di *modificazione*, come una parola modifica la testa (NMOD nominale, AMOD aggettivale, DET, CASE), più CONJ e CC [slide Parser sintattici pp. 19–21]. Il grafo è un albero [slide Parser sintattici p. 22]: (1) una sola radice, il nodo fittizio ROOT, senza archi entranti; (2) ogni altro nodo ha un solo arco entrante (una sola testa); (3) da ROOT c'è un unico cammino a ogni nodo. Un arco è **proiettivo** se dalla testa si raggiunge, seguendo gli archi, ogni parola fra testa e dipendente; l'albero è proiettivo se lo sono tutti gli archi (disegnato sopra la frase, non ha incroci) [slide Parser sintattici pp. 22–23]. Non lo è *JetBlue canceled our flight this morning which was already late: flight* → *was* scavalca *this morning*, che non dipende da *flight* [slide Parser sintattici p. 23]. Quasi tutte le frasi inglesi sono proiettive: su questo si basa il transition-based parsing.

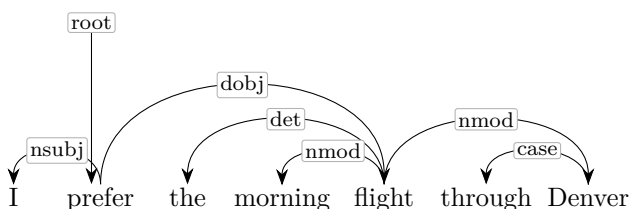


Figura 9.3: Il dependency tree (proiettivo) di *I prefer the morning flight through Denver*: archi dalla testa al dipendente. Fonte: slide Parser sintattici pp. 17, 20

**Confronto con i costituenti** [slide Parser sintattici pp. 18, 44]. Nell'albero a costituenti *prefer* è lontano dai suoi argomenti; qui *I* e *flight* sono suoi figli diretti. Vantaggi: indipendenza dall'ordine delle parole (lingue a ordine libero [dal libro, p. 439; non nelle slide]) e predicati collegati direttamente agli argomenti, utile per coreference resolution (trovare a chi si riferisce un pronome), question answering e information extraction (estrarre fatti dal testo).

## 9.4 Transition-based parsing

### L'idea

Si legge la frase da sinistra a destra una volta sola: le parole lette ma non ancora sistemate aspettano in una pila (*stack*); a ogni passo si guardano le due parole in cima e si decide se una è la testa dell'altra (si crea l'arco e la dipendente, ormai sistemata, esce dalla pila) oppure se leggere un'altra parola. Decide l'*oracle* («oracolo»), oggi un classificatore addestrato su alberi annotati: una scelta per passo, senza ripensamenti, tempo lineare.

### Configurazione, transizioni e algoritmo

La **configurazione** è lo stato del parser: lo **stack** (in cima  $s_1$ , sotto  $s_2$ ), l'**input buffer** con le parole da leggere ( $w_1$  la prima) e le dipendenze trovate; a ogni passo l'**oracle** sceglie fra tre transizioni [slide Parser sintattici p. 24]:

- **LeftArc**: crea la dipendenza  $s_1 \rightarrow s_2$  (la cima è testa della seconda) e toglie  $s_2$ ;
- **RightArc**: crea  $s_2 \rightarrow s_1$  (la seconda è testa della cima) e toglie  $s_1$ ;
- **Shift**: sposta  $w_1$  dal buffer in cima allo stack.

Algoritmo [slide Parser sintattici p. 36]: si parte da stack [ROOT], buffer con tutta la frase, nessuna dipendenza; si ripete  $t \leftarrow \text{ORACLE}(\text{stato})$ ,  $\text{stato} \leftarrow \text{APPLY}(t, \text{stato})$  fino allo stato finale (buffer vuoto, stack [ROOT]): le dipendenze raccolte sono l'albero [dal libro, p. 445; non nelle slide]. Costo  $O(n)$ : ogni parola entra nello stack una volta (Shift) ed esce una volta (un arco),  $2n$  passi, e ORACLE e APPLY costano tempo costante [slide Parser sintattici p. 36]. Con  $k$

tipi di relazione le transizioni  $\text{LeftArc}(r)$ ,  $\text{RightArc}(r)$ ,  $\text{Shift}$  sono  $2k + 1$  [slide Parser sintattici p. 37].

| Passo | Stack (cima a destra)              | Buffer                           | Transizione | Dipendenza aggiunta          |
|-------|------------------------------------|----------------------------------|-------------|------------------------------|
| 0     | [ROOT]                             | [Book, me, the, morning, flight] | Shift       |                              |
| 1     | [ROOT, Book]                       | [me, the, morning, flight]       | Shift       |                              |
| 2     | [ROOT, Book, me]                   | [the, morning, flight]           | RightArc    | Book $\rightarrow$ me        |
| 3     | [ROOT, Book]                       | [the, morning, flight]           | Shift       |                              |
| 4     | [ROOT, Book, the]                  | [morning, flight]                | Shift       |                              |
| 5     | [ROOT, Book, the, morning]         | [flight]                         | Shift       |                              |
| 6     | [ROOT, Book, the, morning, flight] | []                               | LeftArc     | flight $\rightarrow$ morning |
| 7     | [ROOT, Book, the, flight]          | []                               | LeftArc     | flight $\rightarrow$ the     |
| 8     | [ROOT, Book, flight]               | []                               | RightArc    | Book $\rightarrow$ flight    |
| 9     | [ROOT, Book]                       | []                               | RightArc    | ROOT $\rightarrow$ Book      |
| 10    | [ROOT]                             | []                               | Done        |                              |

Figura 9.4: Il parsing di *Book me the morning flight*:  $n = 5$  parole,  $2n = 10$  transizioni; le cinque dipendenze finali sono l'albero. Fonte: slide Parser sintattici pp. 25–35

**L'oracle e il suo addestramento.** Prima regole scritte a mano, oggi un **classificatore** che dalla configurazione (cima dello stack, inizio del buffer, dipendenze trovate) predice la transizione, addestrato su un **dependency treebank**, un corpus di frasi con l'albero annotato a mano, come il Penn Treebank e OntoNotes [slide Parser sintattici pp. 38–40]. Nella versione neurale [slide Parser sintattici p. 41] un encoder (capitolo 7) produce gli embedding delle parole; quelli di  $w_1$ ,  $s_1$ ,  $s_2$  entrano in una feed-forward network con softmax sulle  $2k + 1$  azioni. Il treebank però dà alberi, non coppie (configurazione, transizione): gli esempi si generano simulando il parser con un **training oracle** [dal libro, p. 447; non nelle slide] che confronta l'albero *gold*  $R_p$  (quello corretto) con le dipendenze correnti  $R_c$  [slide Parser sintattici p. 42]: **LeftArc**( $r$ ) se  $(s_1, r, s_2) \in R_p$ ; **RightArc**( $r$ ) se  $(s_2, r, s_1) \in R_p$  e nessun  $(s_1, r', w) \in R_p$  manca in  $R_c$  ( $s_1$  ha già tutti i dipendenti); **Shift** altrimenti. La condizione su RightArc serve perché  $s_1$  esce dallo stack per sempre e non riceve più dipendenti [slide Parser sintattici p. 43]: in *Book the flight through Houston* (gold: ROOT  $\rightarrow$  Book, Book  $\rightarrow$  flight, flight  $\rightarrow$  the, flight  $\rightarrow$  Houston, Houston  $\rightarrow$  through), con stack [ROOT, Book, flight], buffer [through, Houston] e  $R_c = \{\text{flight} \rightarrow \text{the}\}$ , Book  $\rightarrow$  flight è nel gold ma a *flight* manca ancora *Houston*: l'oracle dice Shift (così al passo 1 della figura 9.4 non chiude subito ROOT  $\rightarrow$  Book [dal libro, p. 448; non nelle slide]).

#### Descrivere il parsing a costituenti e l'algoritmo CKY

(1) Definizioni: CFG come regole  $A \rightarrow \beta$  su terminali e non terminali, lessico, simbolo iniziale  $S$ ; albero di parsing e bracketed notation (figura 9.1). (2) Ambiguità strutturale (*I shot an elephant in my pajamas*): molti alberi, e rianalizzare gli stessi pezzi per ciascuno costa troppo. (3) CNF: solo  $A \rightarrow BC$  e  $A \rightarrow w$ ; nuovi simboli per le regole lunghe ( $X1$ ,  $X2$ ), eliminazione delle produzioni unitarie, stesso linguaggio. (4) CKY: cella  $[i, j]$  = categorie che coprono le parole da  $i + 1$  a  $j$ ; diagonale dal lessico, celle superiori combinando  $[i, k]$  e  $[k, j]$  con una regola  $A \rightarrow BC$ ; colonne da sinistra a destra, ciascuna dal basso; frase generata se  $S \in [0, n]$ ; puntatori per gli alberi. (5) Esempio *Book the flight through Houston* (figura 9.2):  $[0, 5] = \{S, VP, X2\}$  da due tagli, tre alberi. (6) Limiti: trova tutti gli alberi ma non dice qual è quello giusto (serve la disambiguazione, oggi neurale [dal libro, p. 425; non nelle slide]).

#### Confrontare costituenti e dipendenze; descrivere il transition-based parser

(1) Costituenti: sintagmi annidati generati da una CFG, albero con nodi NP, VP...; dipendenze: parole come nodi, archi head  $\rightarrow$  dependent etichettati con la funzione grammaticale (Universal Dependencies: NSUBJ, DOBJ, DET...). (2) Vantaggi: indipendenza dall'ordine delle parole, predicati collegati direttamente agli argomenti. (3) Albero: radice ROOT, una testa per parola,

cammino unico; proiettività (nessun incrocio). (4) Transition-based parser: configurazione (stack, buffer, dipendenze), transizioni LeftArc, RightArc, Shift;  $2n$  passi,  $O(n)$  (figura 9.4). (5) Oracle: classificatore (encoder, embedding di  $w_1, s_1, s_2$ , feed-forward network, softmax su  $2k + 1$  azioni) addestrato su un treebank col training oracle: LeftArc se  $s_1 \rightarrow s_2$  è nel gold, RightArc se  $s_2 \rightarrow s_1$  è nel gold e  $s_1$  ha già tutti i dipendenti, altrimenti Shift. (6) Limiti: greedy (nessun ritorno indietro), gli errori non si correggono [dal libro, p. 446; non nelle slide]; solo alberi proiettivi [slide Parser sintattici p. 44].

#### Errore tipico da evitare

- In LeftArc e RightArc a uscire dallo stack è sempre il *dipendente*, mai la testa (che può ricevere altri dipendenti); il nome indica la direzione dell'arco: in LeftArc la cima  $s_1$  è testa di  $s_2$ , alla sua sinistra.
- La cella CKY  $[i, j]$  copre le parole da  $i + 1$  a  $j$  (gli indici stanno fra le parole). Le etichette delle dipendenze sono funzioni grammaticali, non parti del discorso.

## Capitolo 10

### Parser semantici e rappresentazioni del significato

#### 10.1 Rappresentare il significato

##### L'idea

Un parser sintattico (capitolo 9) dice *com'è fatta* una frase, non che cosa *significa*. Per rispondere a una domanda bisogna tradurla in una struttura formale confrontabile con ciò che il programma sa del mondo: il semantic parsing è questa traduzione, in formule logiche, frame di ruoli o grafi.

##### Meaning representation, semantic parsing e desiderata

Una **meaning representation** (rappresentazione del significato) è una struttura formale che cattura il significato di un'espressione linguistica; costruirla e assegnarla all'input è il **semantic parsing** (analisi semantica) [slide Parser semantici p. 3]. *I have a car* in tre forme [slide Parser semantici p. 4]: in logica del primo ordine  $\exists e, y \text{ Having}(e) \wedge \text{Haver}(e, \text{Speaker}) \wedge \text{HadThing}(e, y) \wedge \text{Car}(y)$ ; come grafo AMR (h / have-01 :ARG0 (i / i) :ARG1 (c / car)); come frame, *Having* con *Haver* = Speaker e *HadThing* = Car. Le quattro proprietà richieste (*desiderata*) [slide Parser semantici p. 5]:

- **verificabilità**: confrontare lo stato di cose descritto con una base di conoscenza: *Does Maharani serve vegetarian food?* diventa *Serves(Maharani, VegetarianFood)*, da cercare in un database di predicati nella stessa forma [slide Parser semantici p. 6];
- **non ambiguità**: un solo significato per rappresentazione. *I wanna eat someplace that's close to ICSI* ha due letture (mangiare *in* un posto o mangiare *un* posto), quindi due rappresentazioni; la **vaghezza** è ammessa: *I want to eat Italian food* non dice quale piatto, ma ha una sola rappresentazione [slide Parser semantici p. 7];
- **forma canonica**: parafrasi (*Does Maharani have vegetarian dishes?*, *Does Maharani serve vegetarian fare?*) e attivo/passivo (*Vegetarian dishes are served by Maharani*) devono dare la stessa rappresentazione, altrimenti la base di conoscenza non troverebbe la maggior parte delle forme, o sarebbe difficile tenerla coerente [slide Parser semantici p. 8];
- **espressività**: coprire un'ampia gamma di argomenti, idealmente ogni frase sensata.

#### 10.2 Semantica model-theoretic e logica del primo ordine

##### L'idea

Che cosa vuol dire che una formula è «vera»? Si costruisce un *modello*, un piccolo mondo di oggetti, insiemi di oggetti (proprietà) e insiemi di coppie (relazioni); ogni simbolo punta a un pezzo di questo mondo e la formula è vera se, seguendo i puntatori, trova ciò che afferma (*Matthew likes Frasca*: cercare una coppia in una tabella).

##### Modello, interpretazione e logica del primo ordine (FOL)

Un **modello** è un costrutto formale che sta per uno stato di cose del mondo: contiene **oggetti**, **proprietà** (insiemi di oggetti: *red* denota l'insieme delle cose rosse) e **relazioni** (insiemi di tuple ordinate), definiti per *estensione* [slide Parser semantici pp. 9–10]. Il **vocabolario non logico** (nomi di oggetti, proprietà e relazioni: insieme aperto, ognuno con una *denotazione*,

cioè l'elemento del modello che indica) si distingue da quello **logico** (connettivi, quantificatori, parentesi: insieme chiuso) [slide Parser semantici p. 9]; l'**interpretazione** mappa i simboli sulle denotazioni [slide Parser semantici p. 11]. *Esempio* [slide Parser semantici p. 12]: dominio  $\mathcal{D} = \{a, \dots, j\}$  con Matthew, Franco, Katie, Caroline  $\mapsto a, b, c, d$ ; Frasca, Med, Rio  $\mapsto e, f, g$ ; Italian, Mexican, Eclectic  $\mapsto h, i, j$ ;  $Noisy = \{e, f, g\}$ ;  $Likes = \{\langle a, f \rangle, \langle c, f \rangle, \langle c, g \rangle, \langle b, e \rangle, \langle d, f \rangle, \langle d, g \rangle\}$ ;  $Serves = \{\langle f, j \rangle, \langle g, i \rangle, \langle e, h \rangle\}$ . *Matthew likes Frasca* è vera se  $\langle a, e \rangle \in Likes$  (*Matthew*  $\mapsto a$ , *Frasca*  $\mapsto e$ ): non c'è, è falsa.

**First-Order Logic (FOL)**: esprime enunciati più complessi di un singolo predicato (*Katie and Caroline like the same restaurants, Not everybody likes Frasca*) [slide Parser semantici p. 13]. Grammatica [slide Parser semantici p. 14]:

$$\begin{aligned} Formula &\rightarrow AtomicFormula \mid Formula \text{ Connective } Formula \mid \neg Formula \\ &\mid (Formula) \mid Quantifier \ Variable, \dots Formula \\ AtomicFormula &\rightarrow Predicate(Term, \dots) \quad Term \rightarrow Function(Term, \dots) \mid Constant \mid Variable \\ Connective &\rightarrow \wedge \mid \vee \mid \Rightarrow \quad Quantifier \rightarrow \forall \mid \exists \quad Function \rightarrow LocationOf \mid CuisineOf \mid \dots \\ Constant &\rightarrow VegetarianFood \mid Maharani \mid \dots \quad Predicate \rightarrow Serves \mid \dots \quad Variable \rightarrow x \mid y \mid \dots \end{aligned}$$

*Esempio* [slide Parser semantici p. 15], *a restaurant that serves Mexican food near ICSI* («esiste un  $x$  che è un ristorante, serve cibo messicano ed è vicino a ICSI»), con la variabile  $x$ , la costante *ICSI*, la funzione *LocationOf*( $x$ ) e il quantificatore  $\exists$  («esiste»;  $\forall$  è «per ogni»):

$$\exists x \text{ Restaurant}(x) \wedge Serves(x, MexicanFood) \wedge Near(LocationOf(x), LocationOf(ICSI)).$$

**Rappresentazione neo-davidsoniana** [slide Parser semantici p. 16]: un verbo ha un numero variabile di argomenti (*I ate; I ate a turkey sandwich for lunch at my desk*); si introduce una variabile  $e$  per l'evento e un predicato binario per ogni ruolo, un congiunto per ogni argomento presente (per *I ate* restano solo i primi due):

$$\exists e \text{ Eating}(e) \wedge Eater(e, Speaker) \wedge Eaten(e, TurkeySandwich) \wedge Meal(e, Lunch) \wedge Location(e, Desk).$$

### 10.3 Ruoli semantici: PropBank e FrameNet

#### L'idea

I ruoli *Eater* ed *Eaten* valgono solo per *eat*: se ogni verbo ha i suoi, non si generalizza. Serve un inventario condiviso (chi, che cosa, dove, quando, con che cosa, perché, come) che dica chi ha fatto che cosa a chi, qualunque sia la sintassi.

#### Ruoli tematici, SRL, PropBank e FrameNet

In *Sasha broke the window* e *Pat opened the door* il soggetto è chi agisce (*Breaker, Opener*) e l'oggetto ciò che subisce (*BrokenThing, OpenedThing*): **agent** (chi) e **theme** (che cosa) sono **ruoli tematici**, nomi standard validi per ogni predicato; i **ruoli semantici** sono un approccio simile ma più flessibile; assegnare i ruoli a un predicato in una frase è il **Semantic Role Labeling (SRL)** [slide Parser semantici p. 17]. **PropBank** (Proposition Bank) è una risorsa di frasi annotate con ruoli semantici (tutto il Penn Treebank); i ruoli, numerati Arg0, Arg1, Arg2, ..., dipendono dal predicato, ma in generale **Arg0** è il *proto-agent* (agisce di proposito, causa l'evento), **Arg1** il *proto-patient* (subisce un cambiamento di stato); Arg2 è spesso beneficiario, strumento, attributo o stato finale, Arg3 il punto di partenza, Arg4 quello di arrivo [slide Parser semantici p. 18] [dal libro, p. 492; non nelle slide]. Il significato concreto sta nel **frame file** di ogni senso del verbo [slide Parser semantici p. 19]: **agree.01** ha Arg0 Agreeer, Arg1 Proposition, Arg2 Other entity agreeing: [Arg0 The group] *agreed* [Arg1 it wouldn't make an offer]; [ArgM-TMP Usually] [Arg0 John] *agrees* [Arg2 with Mary] [Arg1 on everything]. **fall.01** non ha Arg0, perché il soggetto di *fall* subisce [dal libro, p. 493; non nelle slide]. Gli argomenti non numerati sono **ArgM-...** (ArgM-TMP quando, ArgM-LOC dove); **NomBank** fa lo

stesso per i predicati nominali (*Apple's agreement with IBM*: Apple Arg0, IBM Arg2) [slide Parser semantici p. 20].

**FrameNet**: approccio cognitivo, la conoscenza di sfondo è organizzata in **frame**, strutture che definiscono un insieme di ruoli semantici propri del frame, i **frame element**, e includono i predicati che li usano; ogni parola *evoca* un frame e ne mette a fuoco alcuni aspetti. I ruoli sono **core** (specifici del frame) o **non-core** (generali: tempo, luogo..., come gli ArgM di PropBank [dal libro, p. 495; non nelle slide]) [slide Parser semantici p. 21].

*Esempio.* Il frame **change\_position\_on\_a\_scale** raccoglie le parole che indicano il cambiamento di posizione di un ITEM su una scala (l'ATTRIBUTE) da un valore iniziale a uno finale [dal libro, p. 495; non nelle slide]; i frame element sono in figura 10.1 [slide Parser semantici p. 23]: [ITEM Oil] *rose* [ATTRIBUTE in price] [DIFFERENCE by 2%]; [ITEM Microsoft shares] *fell* [FINAL\_VALUE to 7 5/8]; [ITEM Colon cancer incidence] *fell* [DIFFERENCE by 50%] [GROUP among men]. I ruoli sono del frame, non del verbo (*rise, fall, increase*).

| Core Roles          |                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------|
| ATTRIBUTE           | The ATTRIBUTE is a scalar property that the ITEM possesses.                                                            |
| DIFFERENCE          | The distance by which an ITEM changes its position on the scale.                                                       |
| FINAL_STATE         | A description that presents the ITEM's state after the change in the ATTRIBUTE's value as an independent predication.  |
| FINAL_VALUE         | The position on the scale where the ITEM ends up.                                                                      |
| INITIAL_STATE       | A description that presents the ITEM's state before the change in the ATTRIBUTE's value as an independent predication. |
| INITIAL_VALUE       | The initial position on the scale from which the ITEM moves away.                                                      |
| ITEM                | The entity that has a position on the scale.                                                                           |
| VALUE_RANGE         | A portion of the scale, typically identified by its end points, along which the values of the ATTRIBUTE fluctuate.     |
| Some Non-Core Roles |                                                                                                                        |
| DURATION            | The length of time over which the change takes place.                                                                  |
| SPEED               | The rate of change of the VALUE.                                                                                       |
| GROUP               | The GROUP in which an ITEM changes the value of an ATTRIBUTE in a specified way.                                       |

Figura 10.1: Frame element del frame.

Fonte: slide Parser semantici p. 22

### Semantic Role Labeling: l'algoritmo feature-based e quello neurale

Il SRL trova i ruoli di ogni argomento di ogni predicato della frase, con apprendimento supervisionato su PropBank o FrameNet [dal libro, p. 496; non nelle slide]. **Feature-based** [dal libro, pp. 496–499; non nelle slide]: (1) parsing sintattico (capitolo 9) e ricerca dei predicati; (2) per ogni predicato, ogni costituente dell'albero riceve un vettore di feature: predicato, tipo di sintagma (NP, PP), head word (la parola testa) e suo POS, *path* nell'albero fino al predicato (NP↑S↓VP↓VBD per il soggetto di *issued*), voce attiva/passiva, posizione prima/dopo il predicato; (3) un classificatore 1-di-*N* gli assegna un ruolo o NONE, con vincoli globali (argomenti non sovrapposti, un solo Arg0). **Neurale** [dal libro, p. 499; non nelle slide]: dato il predicato, è sequence labeling con tag BIO come la NER (capitolo 8): frase e predicato (dopo [SEP]) entrano in un encoder e ogni token, concatenato con l'indicatore del predicato, passa per un feed-forward con softmax: *the cats love hats*, predicato *love* → B-ARG0 I-ARG0 B-PRED B-ARG1.

## 10.4 Rappresentazioni a grafo: FRED e AMR

### L'idea

Per molte applicazioni la FOL è troppo complessa: si rinuncia a un po' di espressività per la semplicità di un grafo [slide Parser semantici p. 24]: concetti come nodi, ruoli come archi etichettati. **FRED** [slide Parser semantici pp. 25–26] lo fa in RDF (Resource Description Framework), lo standard dei *Linked Data* (dati pubblicati sul web come triple soggetto–relazione–oggetto, collegate fra basi di dati diverse da identificatori condivisi, gli URI): il grafo, con un nodo per l'evento e un arco per ruolo (neo-davidsoniano), si aggancia così a risorse esistenti come WordNet e *DBpedia* (Wikipedia in triple RDF: *Franz Ferdinand* e *Sarajevo* puntano alle loro voci, entity linking).

### Abstract Meaning Representation (AMR)

AMR (Banarescu et al., 2013) [slide Parser semantici p. 27]: (1) **grafo radicato ed etichettato**; (2) **astrae dalla sintassi**: frasi diverse con lo stesso significato hanno la stessa AMR;

(3) **basata su PropBank**: i ruoli sono quelli dei *frameset* di PropBank (un senso di predicato con i suoi ruoli numerati, come *agree.01*), anche senza un verbo esplicito (*bond investor* → *invest-01*); (4) **agnostica sulla derivazione**: non prescrive come ottenerla dal testo né come allinearla alle parole. I **nodi** sono concetti [slide Parser semantici p. 29]: parole inglesi (*boy*), frameset PropBank (*want-01*), keyword (tipi speciali di entità, quantità, congiunzioni); gli **archi** relazioni: argomenti dei frame (:ARG0...:ARG4), relazioni generali (:beneficiary, :cause...), per quantità, date e liste (:quant, :day, :op1...). Ogni nodo ha una *variabile* e un concetto (*w* / *want-01*); la notazione **PENMAN** annida le parentesi dalla radice e cita con la sola variabile un nodo raggiunto da due archi (*rientranza*).

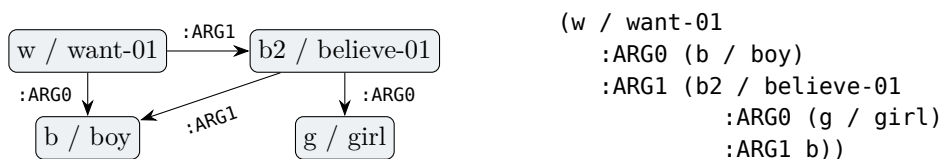


Figura 10.2: L'AMR di *The boy wants the girl to believe him* [non nelle fonti del corso] come grafo e in PENMAN: il volere *w* ha :ARG0 (chi vuole) il ragazzo *b* e :ARG1 (cosa voluta) il credere *b2*, che ha :ARG0 la ragazza *g* e :ARG1 *lo stesso b*: due archi entranti, grafo e non albero. Fonte: slide Parser semantici p. 28

## 10.5 SPRING: da testo ad AMR con un transformer

### L'idea

Un grafo AMR in PENMAN è una stringa di token: tradurre una frase nel suo grafo diventa una traduzione fra sequenze, come la machine translation, e la può fare un encoder-decoder pre-addestrato, purché impari i simboli nuovi e produca sempre un grafo valido.

### SPRING

SPRING (Bevilacqua et al., 2021) traduce da testo ad AMR e viceversa con un solo encoder-decoder pre-addestrato, ed è lo stato dell'arte del parsing basato su AMR [slide Parser semantici p. 30]. Il modello è **BART** (figura 10.3): encoder bidirezionale e decoder autoregressivo (capitolo 7), pre-addestrato a ricostruire il testo originale da una copia corrotta in cinque modi [slide Parser semantici p. 31].

I cinque tipi di rumore [slide Parser semantici p. 31] (glosse dall'articolo di BART [non nelle fonti del corso]): **token masking**, token sostituiti da [MASK] come in BERT; **token deletion**, token cancellati senza segnaposto; **text infilling**, uno span (anche vuoto) sostituito da un solo [MASK]; **sentence permutation**, frasi in ordine casuale; **document rotation**, il testo riparte da un token a caso. Tre sfide [slide Parser semantici p. 32] e le soluzioni:

- **rappresentare il grafo** [slide Parser semantici p. 33]: PENMAN, breadth-first o depth-first search (DFS, leggermente migliore); i riferimenti a un nodo già visto (le rientranze) sono variabili in PENMAN e token speciali <R0>, <R1>, ... in BFS e DFS;
- **vocabolario nuovo** [slide Parser semantici p. 34]: relazioni e frame con almeno 5 occorrenze diventano token, allargando le matrici di embedding; ogni token nuovo parte dalla media degli embedding dei suoi subword;

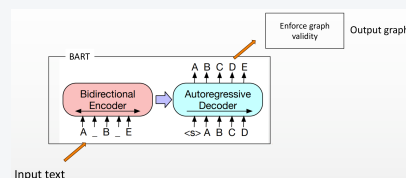


Figura 10.3: SPRING (text-to-AMR) con BART. Fonte: slide Parser semantici p. 31

- **validità del grafo** [slide Parser semantici p. 35]: sull'output si tolgono brevi sequenze di token, si ripristina la parità delle parentesi e si scarta ogni token che non può seguire quello che lo precede.

### Descrivere le rappresentazioni del significato e il semantic parsing

(1) Definizione: struttura formale che cattura il significato; il semantic parsing la costruisce dal testo. (2) Desiderata: verificabilità, non ambiguità (vaghezza ammessa), forma canonica, espressività. (3) Semantica model-theoretic: modello, vocabolario logico e non logico, interpretazione (ristoranti). (4) FOL: grammatica, esempio di ICSI, rappresentazione neo-davidsoniana con la variabile evento. (5) Alternative più semplici: ruoli semantici (PropBank, FrameNet, SRL) e grafi (FRED in RDF; AMR: grafo radicato basato su PropBank, astrae dalla sintassi; *want-01/believe-01*). (6) Parser: SPRING, grafo come sequenza di token, con BART.

### Confrontare PropBank e FrameNet

(1) Entrambi annotano ruoli semantici e sono i dati del SRL. (2) PropBank: ruoli numerati per senso del verbo (Arg0 proto-agent, Arg1 proto-patient), significato nel frame file, ArgM per i modificatori, NomBank per i nomi. (3) FrameNet: frame element definiti per *frame*, core e non-core ( $\approx$  ArgM); *rise*, *fall*, *increase* evocano lo stesso frame. (4) PropBank: ruoli per singolo senso del verbo; FrameNet: condivisi da tutte le parole (verbi e nomi) che evocano il frame, quindi generali fra predicati diversi [dal libro, p. 494; non nelle slide]. (5) AMR usa i frameset di PropBank come concetti.

### Che cos'è il SRL e come si realizza

(1) Task: per ogni predicato trovare gli argomenti e il ruolo di ciascuno (Arg0, Arg1, ArgM-TMP... o frame element); supervisionato su PropBank o FrameNet. (2) Feature-based: parsing, poi per ogni costituente un classificatore sceglie il ruolo o NONE da feature come predicato, tipo di sintagma, head word, path nell'albero, voce, posizione. (3) Neurale: dato il predicato, sequence labeling con tag BIO (B-ARG0, I-ARG0, O) da encoder, feed-forward e softmax. (4) I ruoli non dipendono dalla sintassi (passivo); AMR riusa gli stessi frameset.

### Errore tipico da evitare

- Ambiguità (due significati: da eliminare) non è vaghezza (poco specificato: ammessa).
- Arg0/Arg1 non sono soggetto/oggetto: nel passivo il soggetto è Arg1 ([<sub>Arg1</sub> The price of bananas] *was increased* [<sub>Arg0</sub> by Big Fruit Co.] [slide Parser semantici p. 19]).
- Un'AMR è un grafo, non un albero: *boy* ha due archi entranti e si cita con la variabile.

## Capitolo 11

### GPT e Large Language Models

#### 11.1 GPT: un decoder pre-addestrato, poi adattato al compito

L'idea: un language model che impara da solo

Un transformer con maschera causale fa da *language model*: predice il token successivo (sez. 7.3) [slide Transformer p. 21]. GPT (*Generative Pretrained Transformer*, 2018) tiene solo il decoder e lo addestra su un corpus enorme di testo *non annotato* (**pre-training**): l'etichetta di ogni posizione è gratis: la parola che segue. Poi lo adatta al compito con pochi dati etichettati (**fine-tuning**): un linear layer in cima e l'ingresso *trasformato* in una sequenza con token delimitatori (fig. 11.1). Vocabolario [slide GPT p. 3]: **zero-shot** = predire etichette mai viste in addestramento, senza esempi; **one-shot** = un esempio del compito; **few-shot** = pochi; di norma senza aggiornare i pesi.

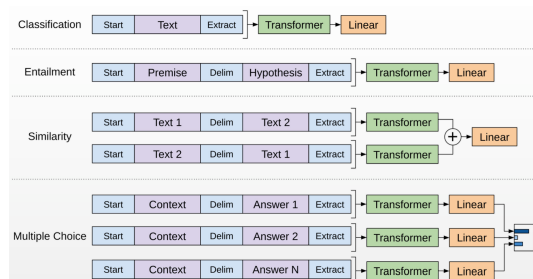


Figura 11.1: Fine-tuning di GPT: ogni compito diventa una sequenza *Start ... Delim ... Extract* letta dal linear layer (similarity: somma dei due ordini; multiple choice: softmax dei punteggi).

Fonte: slide GPT p. 8

#### Architettura, pre-training e fine-tuning di GPT

Solo decoder, senza cross-attention, self-attention da sinistra a destra (mascherata): 12 transformer block (fig. 7.2) [slide GPT p. 4]. Dato il contesto  $U = (u_{-k}, \dots, u_{-1})^T$  dei  $k$  token precedenti [slide GPT p. 5]:

$$h_0 = U W_e + W_p, \quad h_l = \text{transformer\_block}(h_{l-1}), \quad P(u) = \text{softmax}(h_n W_e^T),$$

per  $l = 1, \dots, n$ , con  $W_e$  matrice dei token embedding (riusata, trasposta, in uscita) e  $W_p$  dei position embedding. **Pre-training non supervisionato** [slide GPT p. 6]: su un corpus di token  $\mathcal{U} = \{u_1, \dots, u_n\}$  si massimizza  $L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$ , cioè la cross-entropy del token successivo (def. 7.3); BookCorpus (tab. 11.1), 12 attention head,  $D = 768$ , 100 epoche (passate sul training set, cap. 6) con Adam [slide GPT p. 9]. **Fine-tuning supervisionato** [slide GPT p. 7]: un linear layer  $W_y$  mappa  $h_l^m$ , l'attivazione dell'ultimo block sull'ultimo token, nelle classi; sul dataset etichettato  $\mathcal{C}$  si massimizza  $L_2$  più il language model sul testo del compito come termine ausiliario ( $\lambda = 0.5$ , 3 epoche [slide GPT p. 9]):

$$P(y | x^1, \dots, x^m) = \text{softmax}(h_l^m W_y), \quad L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y | x^1, \dots, x^m), \quad L_3 = L_2 + \lambda L_1,$$

stato dell'arte su quattro compiti [slide GPT p. 10]: *natural language inference* (NLI: dire se una frase segue da un'altra, la contraddice o è neutra [dal libro, p. 220; non nelle slide]), QA e

*commonsense reasoning* (senso comune), *semantic similarity*, classificazione (grammaticalità e sentiment).

## 11.2 GPT-2 e GPT-3: la scala, zero-shot e in-context learning

### Il compito è già scritto nel testo

Ipotesi di GPT-2 [slide GPT p. 11]: «un language model con capacità sufficiente imparerà a inferire ed eseguire i compiti dimostrati nelle sequenze di linguaggio naturale, per predirle meglio»: il web contiene traduzioni e riassunti, e per predirli bene bisogna saper tradurre e riassumere. Allora il compito si chiede nel prompt, *senza fine-tuning*, con o senza esempi; cambia il contesto, non i pesi: **in-context learning** [dal libro, p. 161; non nelle slide].

| modello      | parametri               | layer | $d_{\text{model}}$ | contesto | dati di pre-training                                |
|--------------|-------------------------|-------|--------------------|----------|-----------------------------------------------------|
| GPT (2018)   | $\approx 117\text{M}^*$ | 12    | 768                | –        | BookCorpus: 7 000 libri, $\approx 1\text{B}$ parole |
| GPT-2 (2019) | 117M–1542M              | 12–48 | 768–1600           | 1024     | WebText: 8M documenti, 40 GB                        |
| GPT-3 (2020) | 125M–175B               | 12–96 | 768–12 288         | 2048     | 300B token, 60% Common Crawl filtrato               |

Tabella 11.1: Da GPT a GPT-3 [slide GPT pp. 9, 12, 16, 17]; \*non sulle slide: è il GPT-2 minimo.

### GPT-2 e GPT-3

**GPT-2** (*Language Models are Unsupervised Multitask Learners*) [slide GPT p. 12]: dataset WebText, BPE (cap. 1) con vocabolario 50 257, contesto 1024 token; architettura di GPT in quattro taglie (tab. 11.1). Valutazione **zero-shot** col solo prompt [slide GPT p. 13]: reading comprehension (documento + Q:/A:), summarization (riassunto: documento + TL;DR, *too long, didn't read*), translation (coppie «inglese = francese»), question answering; a volte competitivo senza il training set del compito; migliora con la taglia [slide GPT p. 14].

**GPT-3** (*Language Models are Few-Shot Learners*): «scalare i language model migliora molto le prestazioni few-shot indipendenti dal compito, a volte fino a competere con lo stato dell'arte con fine-tuning» [slide GPT p. 15]. Architettura di GPT-2 con *sparse attention* (non tutti i token contro tutti); otto taglie, da 125M a 175B (96 layer e 96 head da 128, batch da 3.2M token) [slide GPT p. 16]. Addestrato su 300B token [slide GPT p. 17]: Common Crawl filtrato (410B token disponibili, 60% della miscela: 180B usati, 0.44 epoche), WebText2 22%, Books1 e Books2 8% ciascuno, Wikipedia 3% (ripetuta 3.4 volte). Common Crawl (archivio aperto del web) è filtrato con un classificatore di qualità e deduplicato [slide GPT p. 18]. Risultati [slide GPT p. 19]: su TriviaQA e CoQA l'accuratezza cresce con i parametri e con gli esempi nel prompt (few-shot > one-shot > zero-shot); a 175B il few-shot supera lo stato dell'arte con fine-tuning su TriviaQA e lo avvicina su CoQA. **Scalabilità ed emergent abilities** [slide Intro p. 3]: la capacità di risolvere compiti senza addestramento specifico cresce con i parametri; oltre una soglia di taglia emergono abilità nuove, per esempio l'aritmetica: l'addizione a due cifre few-shot resta sotto il 15% fino a 6.7B parametri e tocca il 100% a 175B.

**Etica e sicurezza dei LLM** [slide Intro p. 11]: *bias* ereditati dai dati; *privacy* (conversazioni personali; il modello può riemettere dati del training set); *disinformazione* (contenuti falsi o fuorvianti: *allucinazioni*, cioè testo coerente ma falso, e propaganda); *trasparenza* (decisioni difficili da spiegare); *copyright* dei testi raccolti dal web [dal libro, pp. 175–177; non nelle slide].

## 11.3 InstructGPT: allineare il modello con il feedback umano

### Continuare il testo non è obbedire

Un LLM pre-addestrato continua il testo nel modo più *probabile*, non più *utile*: a «Explain the moon landing to a six year old» GPT-3 rispondeva «Explain the theory of gravity to a 6 year old» [dal libro, p. 226; non nelle slide]; genera anche risposte tossiche, false, dannose o che ignorano le istruzioni: non è *allineato* con l'utente [slide GPT p. 21]. InstructGPT (da cui è nato ChatGPT [slide GPT p. 20]) lo allinea con il **RLHF** (*Reinforcement Learning from Human Feedback*: apprendimento per rinforzo dal feedback umano [dal libro, p. 227; non nelle slide]) in tre fasi (fig. 11.2): *mostrare, giudicare, esercitarsi*.

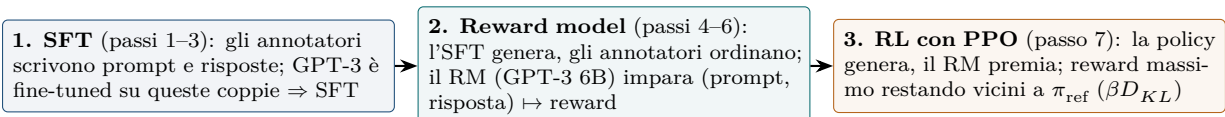


Figura 11.2: InstructGPT in tre fasi; i passi 4–7 si iterano [slide GPT pp. 22–28, schema della guida].

### Le tre fasi di InstructGPT (il protocollo in sette passi)

- SFT, supervised fine-tuning** (passi 1–3, da GPT-3 [slide GPT p. 22]). Annotatori selezionati con quattro criteri [slide GPT p. 23] (accordo nel segnalare linguaggio sensibile e sui ranking, dimostrazioni scritte su temi sensibili, autovalutazione nel riconoscerlo per gruppi diversi) scrivono coppie prompt-risposta («Create a shopping list from this recipe» → gli ingredienti) [slide GPT p. 24]; GPT-3 fine-tuned su queste coppie è il modello **SFT** [slide GPT p. 25]. È l'*instruction tuning*: stessa loss del language model, ma su istruzioni con la risposta corretta [dal libro, p. 227; non nelle slide].
- Reward model** (passi 4–6). L'SFT genera più risposte a un prompt; gli annotatori le ordinano per utilità e tossicità/dannosità, preferendo le innocue [slide GPT p. 26]. Sui ranking si addestra un **Reward Model (RM)**, un GPT-3 da 6B: (prompt, risposta)  $\mapsto$  uno scalare, il *reward* [slide GPT p. 27]. Il RM è un LLM con un linear layer a un'uscita al posto della testa di language model; per ogni coppia (vincente  $o_w$ , perdente  $o_l$ ) la loss di Bradley-Terry è  $L = -\log \sigma(r(x, o_w) - r(x, o_l))$  [dal libro, p. 235; non nelle slide]:  $r_w - r_l = 1 \Rightarrow \sigma(1) = 0.731$ ,  $L = 0.313$ ; punteggi uguali  $\Rightarrow \sigma(0) = 0.5$ .
- Reinforcement learning** (passo 7). Il modello SFT (la *policy*, che genera le risposte  $o$ ) è addestrato con *Proximal Policy Optimization* (PPO) a massimizzare il reward del RM restando vicino al modello iniziale [slide GPT p. 28]:  $\pi^* = \operatorname{argmax}_{\pi_\theta} \mathbb{E}[r(x, o) - \beta D_{KL}(\pi_\theta \| \pi_{\text{ref}})]$ , dove  $D_{KL}$  (divergenza di Kullback-Leibler) misura la distanza fra due distribuzioni di probabilità, qui la policy e il modello di partenza  $\pi_{\text{ref}}$ , e  $\beta$  pesa la penalità; senza di essa il modello dimenticherebbe il pre-training inseguendo il reward [dal libro, pp. 237–238; non nelle slide]. Per non peggiorare sugli altri compiti NLP si aggiunge un termine di pre-training (**PPO-ptx**) [slide GPT p. 22].

**Risultati** [slide GPT p. 29]: InstructGPT 1.3B batte per gli umani GPT-3 175B ( $175/1.3 \approx 135$  volte più grande); più veritiero, meno tossico, regressioni minime sui dataset NLP.

## 11.4 Usare un LLM: prompting e campionamento

### Tutto è generazione condizionata

Si scrive un **prompt** e il modello continua token per token (def. 7.3); il *decoding* regola il compromesso fra prevedibile e vario: il greedy (il token più probabile) è ripetitivo, il random sampling (un token estratto con la sua probabilità) finisce a volte nella coda dei token improbabili [dal libro, pp. 164–165; non nelle slide]; i parametri sotto stanno in mezzo. Alternative libere: Mistral, Llama, Gemma, Phi, eseguibili in locale con Ollama [slide GPT p. 32].

### I parametri dei modelli GPT-like, con un esempio di temperatura

[slide GPT p. 31] **temperature** (1): i  $|V|$  logit (valori prima del softmax) divisi per  $T$ : più netta per  $0 < T < 1$ , più piatta per  $T > 1$ ,  $y = \text{softmax}(u/T)$  [dal libro, p. 165; non nelle slide]. **max\_tokens** (256): token generati al massimo. **top\_k**: si campiona fra i  $k$  token più probabili ( $k$  di solito alto); **top\_p** (1), *nucleus sampling*: i token in testa con probabilità cumulata  $\geq p$ ; poi si rinormalizza [dal libro, pp. 199–200; non nelle slide]. **frequency\_penalty** e **presence\_penalty** (0, in  $[-2, 2]$ ): valori positivi penalizzano i token già comparsi (la prima per frequenza, la seconda per presenza) e spingono verso argomenti nuovi. **n** (1): alternative generate; **stop**: sequenze di arresto.

**Esempio** [dal libro, p. 167; non nelle slide]: logit  $u = (1.2, 0.9, 0.1, -0.5)$  per *all, the, your, that*.

| $T$ | all   | the   | your  | that  |
|-----|-------|-------|-------|-------|
| 1   | 0.443 | 0.328 | 0.148 | 0.081 |
| 0.5 | 0.591 | 0.324 | 0.065 | 0.020 |
| 0.1 | 0.953 | 0.047 | 0.000 | 0.000 |
| 10  | 0.270 | 0.262 | 0.241 | 0.227 |

Per  $T = 0.5$ :  $u/T = (2.4, 1.8, 0.2, -1.0)$  e  $11.02/18.66 = 0.591$ . A  $T = 1$ : top- $k$  ( $k = 2$ ) tiene *all, the*  $\rightarrow (0.574, 0.426)$ ; top- $p$  ( $p = 0.9$ ) tiene 3 token (cumulate 0.443, 0.772, 0.919)  $\rightarrow (0.482, 0.357, 0.161)$ .

## 11.5 GPT contro BERT

### Domanda 7: confrontare GPT (generativo) con BERT (bidirezionale) e i loro ambiti

- Radice comune e architettura** (tab. 11.2). Pile di transformer block pre-addestrate su testo non annotato e poi adattate [slide Transformer p. 3]: GPT tiene il decoder (*auto-regressivo*), con maschera causale: ogni token vede solo i precedenti, niente cross-attention; BERT tiene l'encoder (*auto-encoding*), bidirezionale: vede tutta la frase. Chi vede solo il passato predice il futuro, chi vede tutto solo riempie buchi.
- Pre-training**. GPT massimizza  $\sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1})$ : il token successivo. BERT non può (la risposta sarebbe in ingresso): cloze task (indovinare i token mascherati) sul 15% dei token più next sentence prediction (cap. 7).
- Uscita e uso**. GPT dà la distribuzione del prossimo token e genera per campionamento (temperature, top- $k$ , top- $p$ ); si adatta con fine-tuning e input trasformato, con prompt zero/few-shot a pesi fermi (in-context learning), con instruction tuning e RLHF. BERT dà embedding contestuali: una testa sul [CLS] classifica, una per token fa tagging.
- Ambiti**. GPT *produce* testo: dialogo, summarization, machine translation, QA; BERT *capisce* un testo: classificazione, NER, POS tagging, NLI, similarità, QA estrattivo.
- Limiti e spirito critico**. GPT allucina, costa, va allineato e pone problemi etici, ma su scala il few-shot compete col fine-tuning [slide GPT pp. 15, 19] e InstructGPT 1.3B batte GPT-3 175B per gli umani; BERT è piccolo e preciso ma non genera.

|              | <b>GPT</b> (generativo)                       | <b>BERT</b> (bidirezionale, sez. 7.4)        |
|--------------|-----------------------------------------------|----------------------------------------------|
| architettura | decoder-only, causale: vede solo i precedenti | encoder-only: ogni token vede tutta la frase |
| pre-training | predire il token successivo ( $L_1$ )         | masked LM sul 15% + next sentence prediction |
| uscita e uso | genera; fine-tuning, prompt, RLHF             | non genera; testa sul [CLS] o per token      |
| ambiti       | dialogo, summarization, machine translation   | classificazione, tagging, NLI, QA estrattivo |

Tabella 11.2: GPT e BERT a confronto.

**Confusioni tipiche**

- **La temperatura non riordina i token:**  $T < 1$  più netta,  $T > 1$  più piatta,  $T \rightarrow 0$  greedy. **Il reward model è il giudice, non il modello finale** (la policy dopo PPO).
- **GPT non è bidirezionale e BERT non genera:** decide la maschera causale. **Zero-shot non vuol dire non addestrato**, ma senza fine-tuning per il compito.

## Capitolo 12

### Applicazioni: question answering e RAG, machine translation, summarization

#### 12.1 Question answering e RAG: cercare, poi leggere (o generare)

##### L'idea: un motore di ricerca e un lettore

Per rispondere a una domanda fattuale («quando fu la prima del *Flauto magico*?») bisogna *trovare*, fra milioni di documenti, i pochi passaggi che possono contenere la risposta, poi *leggerli*. I sistemi open domain separano i due passi: un **retriever** (un motore di ricerca) sceglie i  $k$  passaggi più simili alla domanda; un **reader** (un encoder come BERT) legge domanda e passaggio insieme e indica dove la risposta *comincia* e dove *finisce*: non genera testo, estrae uno *span* (un intervallo di token del passaggio). **RAG** cambia il secondo passo: i passaggi trovati entrano *nel prompt* di un LLM, che risponde in base a quelli invece che «a memoria», dai suoi pesi.

##### Open domain QA: retrieve and read

Quasi tutti i sistemi fanno **factoid QA** (risposte brevi e fattuali); esistono anche la **long-form QA** (domande «perché», risposte lunghe) e la **community QA** (domande fra utenti: Quora, StackOverflow) [slide Applicazioni p. 3]. Tre famiglie: **open domain QA** (IR-based: la conoscenza è un grande corpus; *retrieve and read*: retrieval dei passaggi rilevanti, poi reading comprehension); **knowledge-based QA** (la conoscenza è un *knowledge graph*, un grafo di entità e relazioni, o un database strutturato: la domanda diventa una forma logica o una query); **QA by text generation** (un Large Language Model risponde dai propri parametri).

##### Information retrieval: retrieval sparso (dal libro) e denso

**Ad hoc retrieval** [dal libro, p. 245; non nelle slide]: data una *query* (il bisogno informativo dell'utente), restituire in ordine di rilevanza i *documenti* di una *collezione*. **Sparso** [dal libro, p. 249; non nelle slide]: query e documenti sono vettori tf-idf (cap. 5) e  $\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d})$ , una somma sui soli termini di  $q$ ; esempio del libro [dal libro, pp. 249–250; non nelle slide]: per la query «sweet love»,  $d_1$  «Sweet sweet nurse! Love?» ha coseno 0.747 e  $d_2$  «Sweet sorrow» 0.0779. **BM25** [dal libro, pp. 250–251; non nelle slide], variante di tf-idf con due parametri ( $N$  documenti,  $df_t$  quelli che contengono  $t$ ,  $|d_{\text{avg}}|$  la lunghezza media):

$$\text{score}(q, d) = \sum_{t \in q} \log \frac{N}{df_t} \cdot \frac{tf_{t,d}}{k(1-b+b|d|/|d_{\text{avg}}|) + tf_{t,d}} :$$

$k$  bilancia tf e idf ( $k=0$ : conta solo la presenza del termine;  $k$  grande: tf grezza),  $b \in [0, 1]$  regola la normalizzazione per la lunghezza (1: piena, 0: nessuna); tipici  $k \in [1.2, 2]$ ,  $b = 0.75$ . **Inverted index** [dal libro, p. 251; non nelle slide]: dizionario dei termini, ognuno con la *postings list* degli ID dei documenti che lo contengono: dà subito i candidati (un documento senza termini della query ha punteggio 0). **Valutazione** [dal libro, pp. 252–254; non nelle slide]: precision e recall (sez. 3.3) a ogni posizione del ranking (*precision-recall curve*) e **mean average precision** (MAP: media della precision alle posizioni dei documenti rilevanti, poi sulle query).

**Denso** [slide Applicazioni p. 5]: supera il *vocabulary mismatch* dello sparso, che esige le *stesse*

parole in query e documento [dal libro, p. 255; non nelle slide]. Un *bi-encoder* (come per l'entity linking, cap. 8), cioè due encoder separati per domanda e documento:  $\mathbf{h}_q = \text{BERT}_Q(q)[\text{CLS}]$ ,  $\mathbf{h}_d = \text{BERT}_D(d)[\text{CLS}]$ ,  $\text{score}(d, q) = \mathbf{h}_q \cdot \mathbf{h}_d$ . I vettori dei documenti si calcolano una volta sola e si indicizzano [dal libro, p. 255; non nelle slide]; i  $k$  con punteggio più alto si trovano con una *nearest neighbor search* (ricerca dei vettori più vicini) approssimata da FAISS.

### Reading comprehension: dataset e valutazione del factoid QA

**Reading comprehension** [slide Applicazioni p. 6]: dati un passaggio e una domanda, predire lo span della risposta nel passaggio. **Dataset** [slide Applicazioni p. 7]: SQuAD (circa 150K domande su passaggi di Wikipedia, risposta = span); HotpotQA (ragionamento su più documenti); TriviaQA (94K domande scritte senza leggere i passaggi); Natural Questions (query reali a Google, con long answer e short answer); TyDi QA (204K coppie in 11 lingue). **Metriche** [slide Applicazioni p. 10]. **Mean Reciprocal Rank**:  $\text{MRR} = \frac{1}{N} \sum_{i: \text{rank}_i \neq 0} 1/\text{rank}_i$ , con  $\text{rank}_i$  la posizione della risposta corretta nella lista prodotta per la domanda  $i$  (0 se non compare: il termine si salta) e  $N$  il numero di coppie. **Exact match (EM)**: percentuale di risposte predette identiche alla risposta *gold* (quella corretta, annotata a mano). **F1** (sez. 3.3) sui token: per ogni domanda,  $\text{precision} = \text{quota dei token predetti che stanno nella risposta gold}$ ,  $\text{recall} = \text{quota dei token gold ritrovati}$ ; poi la media sulle domande.

**Esempio** (costruito per la guida sulle tre domande della slide 6 [non nelle fonti del corso]; token = parole). Gold *Houston, Texas / singing and dancing / 2003*; predette *Houston, Texas / singing / in 2003*: EM 1,0,0 (33.3%); F1 1, poi 0.5 (precision 1, recall 1/3), poi 0.667 (precision 1/2, recall 1); media 0.722: la F1 premia in parte le risposte «quasi giuste».

### Il reader: estrarre lo span con BERT

**Extractive QA** [slide Applicazioni p. 8]: la risposta è uno span  $a$  del passaggio  $p$ , con inizio  $a_s$  e fine  $a_e$ :  $P(a | q, p) = P_{\text{start}}(a_s | q, p) P_{\text{end}}(a_e | q, p)$ . Con BERT [slide Applicazioni p. 9]: ingresso [CLS]  $q_1 \dots q_n$  [SEP]  $p_1 \dots p_m$ ; per ogni token del passaggio l'encoder dà un vettore  $\mathbf{p}'_i$ ; due vettori di parametri  $\mathbf{S}$  ed  $\mathbf{E}$  (appresi nel fine-tuning) danno due distribuzioni sui token del passaggio:

$$P_{\text{start}_i} = \frac{\exp(\mathbf{S} \cdot \mathbf{p}'_i)}{\sum_j \exp(\mathbf{S} \cdot \mathbf{p}'_j)}, \quad P_{\text{end}_i} = \frac{\exp(\mathbf{E} \cdot \mathbf{p}'_i)}{\sum_j \exp(\mathbf{E} \cdot \mathbf{p}'_j)}.$$

Si sceglie lo span  $(i, j)$ ,  $i \leq j$ , che massimizza  $\mathbf{S} \cdot \mathbf{p}'_i + \mathbf{E} \cdot \mathbf{p}'_j$  (equivale a massimizzare  $P_{\text{start}_i} P_{\text{end}_j}$ ). Loss, con  $i^*, j^*$  le posizioni vere:  $L = -\log P_{\text{start}_{i^*}} - \log P_{\text{end}_{j^*}}$ . Risposta assente dal passaggio: si predice lo span sul token [CLS].

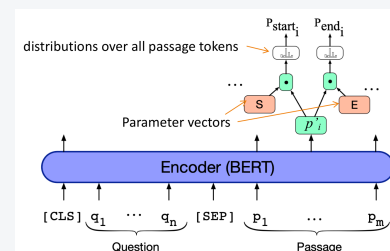


Figura 12.1: Il reader: dai vettori  $\mathbf{p}'_i$  del passaggio, con  $\mathbf{S}$  ed  $\mathbf{E}$ , escono le distribuzioni di inizio e fine.

Fonte: slide Applicazioni p. 9

### Retrieval-Augmented Generation (dal libro)

Tre problemi del prompting puro [dal libro, p. 244; non nelle slide]: *hallucination* (detta con sicurezza: i modelli sono poco calibrati), dati proprietari inaccessibili, conoscenza statica. Un sistema RAG ha due componenti, il **retriever** e il **generator** (o *reader*) [dal libro, p. 258; non nelle slide]. Algoritmo base, data una collezione  $D$  e una query  $q$  [dal libro, pp. 258–259; non nelle slide]: (1) il retriever restituisce  $R(q) = d_1 \dots d_k$ , i  $k$  passaggi più rilevanti di  $D$ ; (2) si costruisce un prompt con i passaggi e la domanda («Based on these texts, answer this question:  $q$ »); (3) l'LLM genera la risposta un token alla volta:  $p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i |$

$R(q)$ ; prompt;  $q$ ;  $x_{<i}$ ). Varianti [dal libro, p. 259; non nelle slide]: *reranker* (riordina i passaggi recuperati), ricerca *multi-hop* (i passaggi trovati si aggiungono alla query per cercare ancora), *agent-based RAG* (il sistema decide quando e dove cercare); *knowledge citation* (riferimenti alle fonti) chieste nel prompt [dal libro, p. 260; non nelle slide]. Con i documenti: **open book QA**; senza: **closed book QA** [dal libro, p. 262; non nelle slide].

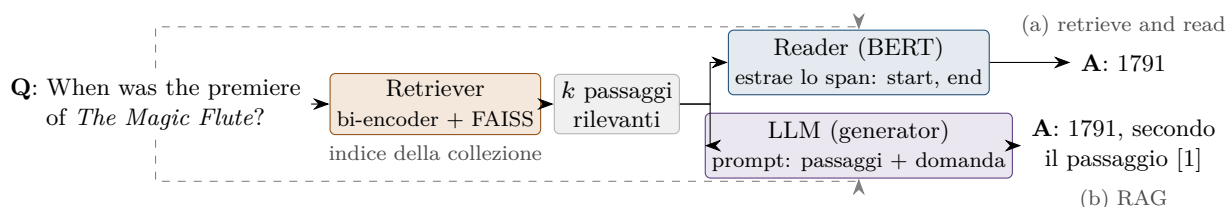


Figura 12.2: (a) Retrieve and read: il reader BERT estrae lo span dai  $k$  passaggi; (b) RAG: i passaggi entrano nel prompt di un LLM che genera; la domanda (tratteggiata) arriva a entrambi. Fonte: schema della guida (slide Applicazioni p. 4, libro p. 258).

## 12.2 Machine translation e text summarization

L'idea: generare, poi misurare la somiglianza con una traduzione umana

Tradurre è generare la frase di arrivo condizionando su *tutta* la frase di partenza. Il difficile è *valutare*: le traduzioni giuste sono molte. I giudici umani sono il riferimento ma costano; le metriche automatiche confrontano l'uscita con una traduzione di riferimento: n-gram in comune (BLEU), embedding vicini (BERTScore), imitazione dei giudizi umani (BLEURT).

Il compito, i dati, la valutazione umana

**MT** [slide Applicazioni p. 11]: data una sequenza di token in una lingua, predire la sequenza nella lingua di arrivo che massimizza  $P(y_1, \dots, y_m \mid x_1, \dots, x_n)$ ; i modelli moderni sono encoder-decoder (sez. 7.5). **Corpora** [slide Applicazioni p. 12]: un *parallel corpus* o *bixtext*, lo stesso testo in due o più lingue (Europarl, United Nations Parallel Corpus, OpenSubtitles, ParaCrawl). **Due dimensioni** [slide Applicazioni p. 13]: *adequacy* (fedeltà: quanto del significato della sorgente è conservato) e *fluency* (quanto la traduzione è grammaticale, chiara, naturale). **Valutazione umana** [slide Applicazioni p. 14]: crowdworker giudicano la fluency su una scala (da 1, incomprensibile, a 5) e l'adequacy confrontando la traduzione con la sorgente (giudici bilingui) o con una traduzione umana (monolingui); più efficace, ma costosa. **Automatica** [slide Applicazioni p. 15]: confronto con una reference: BLEU, BLEURT, BERTScore.

BLEU (BiLingual Evaluation Understudy)

Una buona traduzione condivide molti n-gram con la reference [slide Applicazioni p. 16]. Per  $n = 1, \dots, 4$  si calcola la precisione degli n-gram su tutto il corpus di test, poi si combinano le quattro precisioni con la media geometrica e con la *brevity penalty* BP [slide Applicazioni p. 17]:

$$\text{prec}_n = \frac{\sum_{C \in \{Candidates\}} \sum_{n\text{-gram} \in C} \text{Count}_{\text{match}}(n\text{-gram})}{\sum_{C' \in \{Candidates\}} \sum_{n\text{-gram}' \in C'} \text{Count}(n\text{-gram}')}, \quad \text{BLEU} = \text{BP} \times (\prod_{n=1}^4 \text{prec}_n)^{1/4},$$

con  $\text{BP} = \min(1, \exp(1 - \text{ref\_len}/\text{sys\_len}))$ , dove *Candidates* sono le frasi prodotte dal sistema e  $\text{Count}_{\text{match}}$  conta le occorrenze dell'n-gram del candidato presenti anche nella reference, al massimo tante quante nella reference (conteggi *clipped* [dal libro, p. 286; non nelle slide]; regola ed esempio [non nelle fonti del corso]: «the the the the the the» ha precisione

unigram 2/6, non 6/6). BP punisce le traduzioni più corte della reference, dove la precisione è alta a buon mercato. **Limiti** [slide Applicazioni p. 18]: BLEU è *locale* (spostare un'intera frase può non cambiarlo), non vede la coerenza fra frasi e confronta male sistemi diversi (umano assistito contro automatico, architetture diverse).

| $n$ | $n$ -gram del candidato A                    | $\text{prec}_n$ |
|-----|----------------------------------------------|-----------------|
| 1   | the cat is on the rug                        | 5/6             |
| 2   | the cat, cat is, is on, on the, the rug      | 4/5             |
| 3   | the cat is, cat is on, is on the, on the rug | 3/4             |
| 4   | the cat is on, cat is on the, is on the rug  | 2/3             |

Tabella 12.1: N-gram di A «the cat is on the rug»; in blu quelli nella reference «the cat is on the mat».

**Esempio** (costruito per la guida [non nelle fonti del corso]). Per il candidato A (tab. 12.1)  $\text{sys\_len} = \text{ref\_len}$ , quindi  $\text{BP} = 1$  e  $\text{BLEU}_A = (\frac{5}{6} \cdot \frac{4}{5} \cdot \frac{3}{4} \cdot \frac{2}{3})^{1/4} = (\frac{1}{3})^{1/4} = 0.760$ . Il candidato B «the cat is on» ha tutte le precisioni uguali a 1: senza correzione avrebbe  $\text{BLEU} = 1$ , più di A; con  $\text{ref\_len} = 6$ ,  $\text{sys\_len} = 4$  e  $\text{BP} = e^{1-6/4} = 0.607$  scende a  $\text{BLEU}_B = 0.607 < \text{BLEU}_A$ .

### Metriche neurali: BLEURT e BERTScore

**BLEURT** [slide Applicazioni p. 19]: da triple  $(x, \tilde{x}, r)$  (traduzione di riferimento  $x$ , candidata  $\tilde{x}$ , giudizio umano  $r \in \mathbb{R}$ ) si addestra un predittore basato su BERT, fine-tuned a predire  $r$ , che sostituisce i giudici. **BERTScore** [slide Applicazioni p. 20]:  $x$  e  $\tilde{x}$  passano per BERT, che dà un embedding contestuale per token; ogni token è abbinato in modo greedy al token più simile dell'altra frase (cosine similarity, def. 5.3: prodotto scalare di vettori normalizzati): recall sulla reference  $R_{\text{BERT}} = \frac{1}{|\tilde{x}|} \sum_{x_i \in x} \max_{\tilde{x}_j \in \tilde{x}} x_i \cdot \tilde{x}_j$ , precision sul candidato  $P_{\text{BERT}} = \frac{1}{|\tilde{x}|} \sum_{\tilde{x}_j \in \tilde{x}} \max_{x_i \in x} x_i \cdot \tilde{x}_j$ , e la loro F1. A differenza di BLEU premia sinonimi e parafrasi.

### Text summarization e ROUGE

**Text summarization** [slide Applicazioni p. 21]: generare una versione concisa e coerente di un documento più lungo che ne conservi la maggior parte del significato; **extractive** (si selezionano frasi o paragrafi del testo originale) o **abstractive** (si generano frasi nuove). **Dataset** [slide Applicazioni p. 22]: CNN/Daily Mail (articoli con gli *highlights* come riassunti), PubMed (articoli biomedici con l'abstract). **Metriche** [slide Applicazioni p. 23]: quelle della MT (BLEU, BLEURT, BERTScore) o le **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation) [slide Applicazioni p. 24]. **ROUGE-N**: percentuale degli  $n$ -gram del riassunto di riferimento (umano) che compaiono nel riassunto generato (un recall, al contrario di BLEU); ROUGE-1 (unigram) e ROUGE-2 (bigram). **ROUGE-L**: basata sulla *longest common subsequence* (LCS: la più lunga sequenza di parole comune alle due frasi, nello stesso ordine ma non necessariamente contigue); con  $m$  la lunghezza della reference  $X$  e  $n$  quella del candidato  $Y$ :  $R_{\text{lcs}} = \text{LCS}(X, Y)/m$ ,  $P_{\text{lcs}} = \text{LCS}(X, Y)/n$ ,  $\text{ROUGE}_L(X, Y) = (1 + \beta^2) R_{\text{lcs}} P_{\text{lcs}} / (R_{\text{lcs}} + \beta^2 P_{\text{lcs}})$ .

**Esempio** (costruito per la guida [non nelle fonti del corso]). Reference «police killed the gunman»; riassunti A «police kill the gunman», B «the gunman kill police». ROUGE-1: entrambi 3/4 (*police, the, gunman*); ROUGE-2: entrambi 1/3 (solo *the gunman*). ROUGE-L: la LCS di A è «police the gunman», lunga 3,  $R_{\text{lcs}} = P_{\text{lcs}} = 3/4$  e  $\text{ROUGE}_L = 0.75$ ; quella di B è «the gunman», lunga 2,  $\text{ROUGE}_L = 0.50$  (con  $R_{\text{lcs}} = P_{\text{lcs}}$  il valore di  $\beta$  non conta): solo ROUGE-L si accorge che B ha stravolto l'ordine.

### Descrivere l'information retrieval e il suo rapporto con RAG (e con il QA open domain)

1. **IR.** Da una query, i documenti in ordine di rilevanza. Sparso: tf-idf e coseno (0.747 contro 0.0779) o BM25 ( $k$ : tf contro idf;  $b$ : lunghezza), inverted index; denso: bi-encoder,  $\text{score}(d, q) = \mathbf{h}_q \cdot \mathbf{h}_d$ , nearest neighbor search approssimata (FAISS), robusto al vocabulary mismatch. Valutazione del ranking: precision e recall a ogni posizione, MAP (nel QA anche MRR).
2. **QA open domain = retrieve and read.** Factoid QA (SQuAD): la risposta è uno span. Reader BERT su [CLS]  $q$  [SEP]  $p$ : con  $\mathbf{S}$  ed  $\mathbf{E}$  softmax di inizio e fine, span  $(i, j)$  con  $\mathbf{S} \cdot \mathbf{p}'_i + \mathbf{E} \cdot \mathbf{p}'_j$  massimo, loss  $-\log P_{\text{start}_{i^*}} - \log P_{\text{end}_{j^*}}$ . Valutazione: exact match e F1 sui token (33.3% e 0.722).
3. **RAG.** Stesso retriever, ma i passaggi entrano nel prompt di un LLM che genera:  $\prod_i p(x_i | R(q); \text{prompt}; q; x_{<i})$ . Motivi: hallucination, dati privati, conoscenza statica; varianti: reranker, multi-hop, citazioni. **Limiti e collegamenti.** Il reader estrae soltanto; RAG dipende dal retrieval e l'LLM può ignorare i passaggi. tf-idf (cap. 5), BERT (cap. 7), prompting (cap. 11).

### Confrontare le metriche automatiche: BLEU, ROUGE, BERTScore

1. **BLEU:** precisione degli n-gram ( $n = 1..4$ , conteggi clipped), media geometrica, brevity penalty contro le uscite corte (esempio: A 0.760, B 0.607); locale, cieco alla coerenza fra frasi, confronta male sistemi diversi.
2. **ROUGE:** recall della reference (quanto del riassunto umano è recuperato): ROUGE-1/2 sugli n-gram, ROUGE-L sulla LCS, che sente l'ordine (0.75 contro 0.50).
3. **BERTScore e BLEURT:** cosine similarity fra embedding contestuali con abbinamento greedy (recall, precision, F1), accetta sinonimi e parafrasi; BLEURT è BERT fine-tuned a predire i giudizi umani. Più vicini al giudizio umano ma dipendenti da un modello; BLEU e ROUGE sono semplici e riproducibili ma rigidi.

### Confusioni tipiche

- **BLEU è una precisione, ROUGE un recall:** BLEU divide per gli n-gram del *candidato*, ROUGE-N per quelli della *reference*; per questo BLEU ha bisogno della brevity penalty; e vale 0 se una sola  $\text{prec}_n$  è 0 (media geometrica).
- **Il reader non genera:** BERT sceglie due posizioni nel passaggio; se la risposta non c'è, l'extractive QA non può produrla. Genera RAG, con un LLM; ma **RAG non riaddestra il modello:** la conoscenza nuova entra nel prompt, non nei pesi.

## Capitolo 13

### Le domande d'esame

Lo scritto è a domande aperte di teoria [slide Intro p. 16]: qui la scaletta di una risposta (§13.1), le sette domande del programma risolte (§13.2) e le altre domande ovvie di ogni capitolo (§13.3).

#### 13.1 Come si risponde a una domanda aperta

I criteri del voto [programma del corso]: «padronanza dei concetti», «esposizione e collegamento», «linguaggio tecnico» e, per il 28–30, «spirito critico»; la scaletta sotto li soddisfa (fig. 13.1).

##### La scaletta di una risposta, in sei passi

1. **Definizione** precisa, con i termini inglesi delle slide, e il problema che il metodo risolve.
2. **Idea**: perché funziona, a parole, in due o tre frasi.
3. **Formula o schema** chiave, con ogni simbolo spiegato (che cos'è, origine, dimensioni).
4. **Esempio** piccolo con i numeri: quello delle slide.
5. **Limiti**: vantaggi, svantaggi, alternative (è lo spirito critico).
6. **Collegamento** a un altro argomento: che cosa cambia da un metodo all'altro.

Allo scritto: un titolo per passo, una formula per concetto, niente elenchi di parole chiave; all'orale: si parte dalla definizione, i dettagli li chiede il docente.

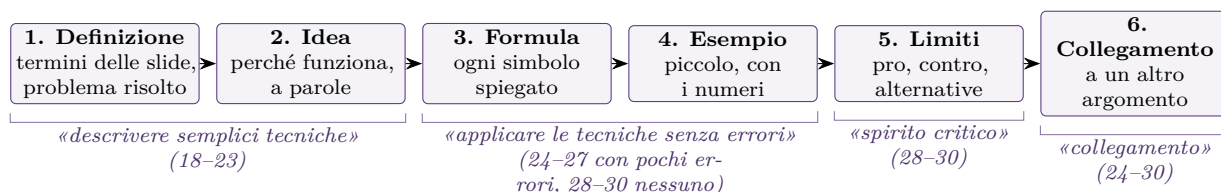


Figura 13.1: I sei passi della risposta e i criteri di voto che soddisfano. Fonte: programma del corso.

#### 13.2 Le sette domande del programma, risolte

1. Descrivere le tecniche di smoothing dei modelli di linguaggio basati su N-grams, evidenziando vantaggi e limiti dei vari metodi.

**Definizione e problema.** Un language model n-gram stima la probabilità della parola successiva dalle ultime  $k-1$  parole (assunzione di Markov) con la MLE  $P(w_i | w_{i-1}) = C(w_{i-1}w_i)/C(w_{i-1})$  [slide N-gram pp. 10, 17]. Gli n-gram del test mai visti nel training (gli *zeri*) azzerano la probabilità della frase e rendono la perplexity incalcolabile [pp. 43–44]: su Shakespeare il 99.96% dei bigram possibili non compare mai [p. 41]. **Idea.** Lo smoothing toglie massa di probabilità agli eventi visti e la dà a quelli mai visti [p. 46]; i metodi differiscono per *quanto* togliere e *a chi* darlo. **Add-one (Laplace):**  $P_{\text{Add-1}} = \frac{C(w_{i-1}w_i)+1}{C(w_{i-1})+|V|}$  [p. 47]. Semplice e senza zeri, ma  $|V|$  è grande rispetto a  $C(w_{i-1})$  e sposta troppa massa: nel Berkeley Restaurant Project  $P(\text{want} | i)$  scende da 0.33 a 0.21 e il conteggio ricostruito di «want to» da 608 a 238 [pp. 48–51]; va bene per la classificazione di testi, non per i LM [p. 52]. **Backoff e interpolazione:** con poca evidenza si usa meno contesto; il backoff scende di ordine solo se il conteggio è zero, l'interpolazione mescola sempre:  $\hat{P} = \lambda_1 P(w_i | w_{i-2}w_{i-1}) + \lambda_2 P(w_i | w_{i-1}) + \lambda_3 P(w_i)$ ,  $\sum_j \lambda_j = 1$ , con  $\lambda$  stimati su un corpus *held-out* (EM) [pp. 53–55]; parole sconosciute con <UNK> [p. 56]. L'interpolazione batte il backoff, ma serve l'*held-out*. **Stupid backoff** (web-scale): frequenze relative e, se l'n-gram manca,  $0.4 \cdot S$  dell'ordine inferiore [pp. 58–59]: scala a miliardi di n-gram, ma dà punteggi, non probabilità. **Absolute discounting:** Church e Gale mostrano che un n-gram

contato  $c$  volte nel training vale circa  $c - 0.75$  nell'held-out [p. 60]:  $P_{AD} = \frac{C(w_{i-1}w_i) - d}{C(w_{i-1})} + \lambda(w_{i-1}) P(w_i)$  [p. 61]. Sconto motivato dai dati, ma l'unigram è la stima sbagliata: «Kong» è frequente eppure segue solo «Hong». **Kneser-Ney**: stessa forma, ma al posto dell'unigram la continuation probability  $P_{CONT}(w_i) = |\{w_{i-1} : C(w_{i-1}w_i) > 0\}| / n$ . di tipi di bigram (quante parole *diverse* precedono  $w_i$ ), con  $\lambda(w_{i-1}) = \frac{d}{C(w_{i-1})} |\{w : C(w_{i-1}w) > 0\}|$  la massa scontata; ricorsivo con i continuation count [pp. 62–66]. È il metodo più usato [p. 67], ma più complesso. **Collegamenti**. La perplexity misura il guadagno; add-one torna in Naive Bayes; i LM neurali superano gli zeri con gli embedding. *Nella guida*: §2.4, pp. 12–14.

## 2. Discutere l'importanza dei word embeddings nella comprensione del linguaggio naturale. Illustrare il funzionamento del modello Word2Vec.

**Definizione.** Un word embedding è un vettore denso e corto (50–1000 componenti) che rappresenta il significato di una parola, imparato dai testi secondo l'ipotesi distribuzionale (il significato è l'uso: le parole con cui una parola compare) [slide Semantica vettoriale pp. 17–24]; parole simili hanno vettori vicini (cosine similarity). **Perché contano.** Con stringhe o indici ogni parola è ugualmente diversa da ogni altra; con gli embedding la similarità è calcolabile e un modello *generalizza*: ciò che ha imparato su «cat» vale per «dog», mai visto, perché i vettori sono vicini. Catturano relazioni (king – man + woman  $\approx$  queen), campi semantici e connotazione [pp. 63–67] e sono l'ingresso di ogni modello neurale di NLP; rispetto ai vettori sparsi di tf-idf sono densi: meno pesi, generalizzano meglio, catturano la sinonimia [pp. 42–43]. **Word2vec (skip-gram with negative sampling).** Predire invece di contare: un classificatore binario « $c$  compare vicino a  $w$ ?», auto-supervisionato dal corpus [pp. 45–46]. (a) Positivi:  $w$  con ogni  $c$  della finestra ( $\pm 2$ ); negativi: per ogni positivo  $k$  parole pescate a caso secondo la frequenza [pp. 48, 52]. (b) Due embedding per parola, le matrici  $W$  (target) e  $C$  (contesto), casuali all'inizio [p. 51]. (c)  $P(+ | w, c) = \sigma(\mathbf{c} \cdot \mathbf{w})$ ,  $P(- | w, c) = \sigma(-\mathbf{c} \cdot \mathbf{w})$  [pp. 49–50]. (d) Loss  $L_{CE} = -[\log \sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) + \sum_{i=1}^k \log \sigma(-\mathbf{c}_{neg_i} \cdot \mathbf{w})]$  minimizzata con SGD:  $\mathbf{c}_{pos}$  si avvicina a  $\mathbf{w}$ , i  $\mathbf{c}_{neg_i}$  se ne allontanano [pp. 55–60]. (e) Si butta il classificatore e si tengono gli embedding ( $\mathbf{w}_i + \mathbf{c}_i$ ) [p. 61]. *Esempio*:  $\mathbf{w} = (1.0, 0.5)$ ,  $\mathbf{c}_{pos} = (0.8, 0.6)$ :  $P(+ ) = \sigma(1.10) = 0.750$ ; dopo un passo con  $\eta = 0.5$  sale a 0.809 e la loss scende da 1.56 a 1.18 [non nelle fonti del corso]. **Limiti e collegamenti.** Embedding *statici*: un vettore per parola, niente polisemia (*mouse*); analogie solo per parole frequenti; ereditano i bias del corpus (man : programmer :: woman : homemaker) [pp. 68–70]. BERT li rende contestuali. *Nella guida*: §5.5, p. 31.

## 3. Illustrare la rappresentazione delle parole mediante vettori sparsi basata su tf-idf e cosine similarity. Discuterne brevemente i limiti.

**Definizione.** Vettori *sparsi* costruiti contando: nella matrice termine-documento la cella  $(t, d)$  conta le occorrenze di  $t$  in  $d$ ; nella matrice parola-parola ( $|V| \times |V|$ ) la cella  $(w, c)$  conta quante volte  $c$  compare in una finestra attorno a  $w$  [slide Semantica vettoriale pp. 26–30]. Ogni parola è la sua riga:  $|V|$  dimensioni (20 000–50 000), quasi tutte zero. **Idea**: ipotesi distribuzionale, contesti simili danno significati simili [p. 17]. **tf-idf**. I conteggi grezzi premiano parole frequenti ovunque e poco informative (*the*, *good*) [p. 38]. Si pesa  $w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$ , con  $\text{tf}_{t,d} = \log_{10}(\text{count}(t, d) + 1)$  e  $\text{idf}_t = \log_{10}(N/\text{df}_t)$  ( $N$  documenti,  $\text{df}_t$  quelli che contengono  $t$ ) [pp. 39–41]: il tf cresce lentamente con la frequenza, l'idf è massimo per una parola in un solo documento e vale 0 per una in tutti. *Esempio*: su  $N = 37$  opere di Shakespeare *good* compare in tutte,  $\text{idf} = 0$ ; *battle* in 21,  $\text{idf} = \log_{10}(37/21) = 0.246$ , e in *As You Like It* (una occorrenza) pesa  $0.301 \times 0.246 = 0.074$  [p. 41]. **Cosine similarity**. Il prodotto scalare  $\mathbf{v} \cdot \mathbf{w} = \sum_i v_i w_i$  è alto se i vettori hanno valori grandi nelle stesse dimensioni, ma favorisce i vettori lunghi (parole frequenti); si normalizza:  $\cos(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} / (|\mathbf{v}| |\mathbf{w}|)$ , il coseno dell'angolo, fra 0 e 1 con conteggi [pp. 32–35]. *Esempio* (dimensioni pie, data, computer):  $\cos(\text{cherry}, \text{information}) = 0.018$ ,  $\cos(\text{digital}, \text{information}) = 0.996$  [p. 36]. Vale per parole, documenti e query (information retrieval). **Limiti**. Vettori lunghi e sparsi: molti pesi se usati come feature e generalizzano peggio; i sinonimi (*car/automobile*) sono dimensioni distinte, quindi la sinonimia sfugge; conteggi, non rappresentazioni apprese; un vettore per parola [pp. 42–43]. **Collegamento**: embedding densi (word2vec), poi contestuali (BERT). *Nella guida*: §5.4, pp. 29–30.

#### 4. Descrivere il modello BERT illustrandone la strategia di pre-training.

**Definizione.** BERT (Bidirectional Encoder Representations from Transformers) è un *encoder transformer* pre-addestrato:  $L = 12/24$  block,  $A = 12/16$  attention head,  $H = 768/1024$  [slide Transformer p. 31]. Il self-attention è bidirezionale, senza maschera:  $\text{softmax}(\mathbf{Q}\mathbf{K}^T/\sqrt{d_k})\mathbf{V}$  su tutta la matrice  $N \times N$ , ogni token vede i precedenti e i seguenti [p. 32]; l'uscita è un embedding *contestuale* per token. **Idea.** Per capire un testo conviene vedere tutta la frase; ma allora predire il token successivo è banale (è già in ingresso): serve un compito che non riveli la risposta, imparato da testo non annotato (le etichette sono le parole stesse). **Pre-training.** (1) *Cloze task*, o masked language model: si sceglie il 15% dei token; di questi l'80% diventa [MASK], il 10% un token casuale, il 10% resta invariato; nelle posizioni scelte un linear layer  $W_V$  + softmax sul vocabolario predice il token originale, con cross-entropy  $-\log y_{\text{parola}}$  solo lì [pp. 33–35]. (2) *Next sentence prediction*: coppie di frasi, 50% consecutive (IsNext) e 50% casuali (NotNext); [CLS] in testa e [SEP] fra le frasi; il vettore d'uscita di [CLS] passa in  $W_{\text{NSP}}$  + softmax a due classi [pp. 36–37]. Ingresso = token + segment ( $E_A/E_B$ ) + position embedding; le due loss si addestrano insieme [pp. 38–39]. **Uso.** Embedding contestuali al posto di quelli statici; *transfer learning* con il *fine-tuning* di una piccola testa: classificatore (linear layer + softmax) sul vettore di [CLS], multi-label con una sigmoide per classe, un tag per token (NER, POS tagging), natural language inference (NLI: una frase implica, contraddice o è neutra rispetto a un'altra [dal libro, p. 220; non nelle slide]), QA estrattivo (SQuAD) [pp. 40–44]; o come feature senza fine-tuning (NER: F1 94.9 con l'ultimo layer, 96.1 con gli ultimi quattro concatenati) [p. 45]. **Limiti e collegamenti.** Solo il 15% dei token dà loss: pre-training poco efficiente; BERT non genera testo, perché non è causale: per generare servono GPT (decoder) o un encoder-decoder (domanda 7). *Nella guida:* §7.4, p. 45.

#### 5. Descrivere l'utilizzo di una feed-forward network con un hidden layer per la classificazione di testo.

**Definizione.** Una feed-forward network (multi-layer perceptron) è fatta di unità,  $z = \mathbf{w} \cdot \mathbf{x} + b$  e  $a = f(z)$  con  $f$  sigmoide, tanh o ReLU [slide Reti neurali pp. 3–7], in strati fully connected senza cicli; l'input non si conta: con un hidden layer la rete è «a due strati» [pp. 13–16]. **Idea:** l'hidden layer impara una rappresentazione  $\mathbf{h}$  del testo e l'output layer fa una regressione logistica su  $\mathbf{h}$ . **Formule e dimensioni.**  $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$ ,  $\mathbf{z} = U\mathbf{h}$ ,  $\hat{y} = \sigma(z)$  per due classi oppure  $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$  per  $K$  classi [pp. 16–19];  $\mathbf{x} \in \mathbb{R}^{n_0}$ ,  $W \in \mathbb{R}^{d_h \times n_0}$ ,  $\mathbf{b}, \mathbf{h} \in \mathbb{R}^{d_h}$ ,  $U \in \mathbb{R}^{1 \times d_h}$  (oppure  $K \times d_h$ );  $\hat{y} = P(\text{classe} \mid \text{testo})$ . **Perché l'hidden layer.** Una sola unità separa con una retta; XOR non è linearly separable, ma con  $\mathbf{h} = \text{ReLU}(W\mathbf{x} + \mathbf{b})$ ,  $W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ ,  $\mathbf{b} = (0, -1)$  e  $y = \text{ReLU}(h_1 - 2h_2)$  i positivi (0, 1) e (1, 0) collasano in  $\mathbf{h} = (1, 0)$  e una retta li separa [pp. 8–11]; senza non linearità più strati valgono uno solo. **Input.** (a) Le sei feature a mano della regressione logistica (lessici, «no», pronomi, «!», log della lunghezza): l'hidden layer aggiunge interazioni non lineari [pp. 24–26]. (b) *Representation learning*: word embedding dalla matrice  $E$  ( $|V| \times d$ ), concatenati (tre parole:  $3d \times 1$ ,  $W$  è  $d_h \times 3d$ ) [pp. 27–28] oppure, per testi di lunghezza variabile, padding o *pooling* in un solo vettore di dimensione  $d$  (media, o massimo per componente) [p. 29]. *Esempio:* «The dessert is...»  $\rightarrow \hat{y} = P(\text{positivo})$ . **Addestramento.** Cross-entropy  $L_{\text{CE}} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ , gradient descent  $w \leftarrow w - \eta \partial L / \partial w$ , gradienti con la backpropagation sul computation graph [pp. 38–52]. **Limiti e collegamenti.** Il pooling perde l'ordine, la concatenazione fissa la finestra; la stessa rete con softmax su  $|V|$  è il feed-forward language model; oggi transformer pre-addestrati (BERT) con fine-tuning. *Nella guida:* §6.3, p. 36.

#### 6. Esplorare il ruolo dei Transformer nell'ambito dei modelli neurali di linguaggio. Illustra il funzionamento del self-attention layer in un blocco transformer.

**Ruolo.** Prima del 2017 i modelli di sequenza erano reti ricorrenti (RNN, LSTM); «Attention is all you need» (Vaswani et al., 2017) toglie la ricorrenza e tiene solo l'attention: calcolo parallelo, addestramento più rapido, machine translation migliore; nel 2018 nasce BERT e i language model crescono di quasi quattro ordini di grandezza, da ELMo (94M parametri, 2018) a GPT-3 (175B) e Megatron-Turing NLG (530B, 2021) [slide Transformer pp. 2–4]. Perché l'attention: concatenare gli embedding per una feed-forward network non scala (512 parole di dimensione 100: 51 200 ingressi, circa 2.6 miliardi di pesi nell'hidden layer) e aggregarli perde le relazioni fra le parole [p. 6]; il transformer tiene gli embedding e aggiunge layer che mettono ogni parola in relazione alle altre e imparano su quali concentrarsi [p. 7]. Come *language model*, sopra la pila di block un linear layer +

softmax sul vocabolario predice il token successivo, con cross-entropy  $-\log \hat{\mathbf{y}}_y$  mediata sulle posizioni e teacher forcing [pp. 21–24]; genera per campionamento, greedy o beam search [pp. 25–28]. È la base dei modelli auto-regressivi (GPT), auto-encoding (BERT) e seq2seq (BART) [p. 3]. **Self-attention layer.** Ogni token  $x_i$  produce query  $q_i = x_i \mathbf{W}^Q$ , key  $k_i = x_i \mathbf{W}^K$  e value  $v_i = x_i \mathbf{W}^V$ ; il punteggio  $q_i \cdot k_j / \sqrt{d_k}$  dice quanto  $j$  serve a  $i$ ; nel LM le coppie  $j > i$  sono poste a  $-\infty$  (maschera causale); il softmax riga per riga dà i pesi  $\alpha_{ij}$  e  $y_i = \sum_{j \leq i} \alpha_{ij} v_j$ ; in forma matriciale  $\mathbf{Y} = \text{softmax}(\mathbf{QK}^T / \sqrt{d_k}) \mathbf{V}$  [pp. 8–11]. Le tre matrici sono condivise fra le posizioni: i parametri non dipendono dalla lunghezza ( $3 \cdot 100^2 = 30\,000$ ) e l'attenzione è leggibile [p. 12]; più *head* in parallelo, concatenate e riproiettate con  $\mathbf{W}^O$ , catturano relazioni diverse [p. 16]. **Nel blocco.**  $\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x}))$ ,  $\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z}))$  [p. 13]: residual connection, layer normalization, feed-forward per token (due strati con ReLU) [pp. 14–15]; i block si impilano [p. 17]; i positional embedding, sommati all'ingresso, danno l'ordine che il self-attention ignora [pp. 18–20]. **Esempio** (sez. 7.1; 3 token,  $d = 2$ ):  $\alpha_3 = (0.284, 0.140, 0.576)$ ,  $y_3 = \sum_j \alpha_{3j} v_j = (1.720, 0.716)$ : il terzo token guarda soprattutto se stesso [non nelle fonti del corso]. **Limiti e collegamenti.** Costo quadratico nel numero di token; il greedy non dà la frase più probabile («yes yes» 0.20 contro «ok ok» 0.28). Encoder (BERT), decoder (GPT), encoder-decoder con cross-attention (machine translation). *Nella guida:* §7.1, pp. 41–43.

## 7. Confrontare i modelli generativi GPT con quelli bidirezionali come BERT, evidenziando i diversi ambiti di applicazione.

**Radice comune.** Entrambi sono pile di transformer block pre-addestrate su testo non annotato e poi adattate al compito; GPT tiene il *decoder* (auto-regressivo), BERT l'*encoder* (auto-encoding) [slide Transformer p. 3]. **Architettura.** GPT: self-attention con maschera causale, ogni token vede solo i precedenti, niente cross-attention, 12 block [slide GPT p. 4]:  $h_0 = UW_e + W_p$ ,  $h_l = \text{transformer\_block}(h_{l-1})$ ,  $P(u) = \text{softmax}(h_n W_e^T)$  [p. 5]. BERT: bidirezionale, ogni token vede tutta la frase. **Pre-training.** GPT massimizza  $L_1 = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1})$ , il token successivo [p. 6]; BERT non può (la risposta sarebbe in ingresso): masked LM sul 15% dei token più next sentence prediction [slide Transformer pp. 33–37]. **Uscita e adattamento.** GPT dà la distribuzione del token successivo e *genera* (temperature, top- $k$ , top- $p$  [slide GPT p. 31]); si adatta con fine-tuning e input trasformato in sequenza ( $W_y$ ,  $L_3 = L_2 + \lambda L_1$ ) [pp. 7–8], poi, con GPT-2 e GPT-3, col solo prompt, zero/few-shot a pesi fermi (in-context learning) [pp. 11–15], e con InstructGPT tramite il RLHF (*Reinforcement Learning from Human Feedback*, apprendimento per rinforzo dal feedback umano [dal libro, p. 227; non nelle slide]): SFT, reward model e PPO [pp. 22–28]. BERT dà embedding contestuali: una testa su [CLS] classifica, una per token fa tagging. **Ambiti.** GPT per *produrre* testo: dialogo, summarization, machine translation, QA generativo. BERT per *capire* un testo dato: classificazione, NER e POS tagging, natural language inference (NLI, domanda 4), similarità, QA estrattivo (span), retrieval con bi-encoder; pochi dati etichettati e costi bassi. **Spirito critico.** GPT (da 117M a 175B parametri) allucina, costa, va allineato e pone problemi etici (bias, privacy, disinformazione [slide Intro p. 11]); ma con la scala il few-shot compete col fine-tuning [pp. 15, 19] e InstructGPT da 1.3B è preferito dai giudici umani a GPT-3 da 175B [p. 29]. BERT è piccolo e preciso ma non genera. RAG li combina: un encoder cerca i passaggi, un LLM genera la risposta. *Nella guida:* §11.5, p. 66 (tab. 11.2).

## 13.3 Altre domande probabili, per argomento

Le domande d'esame «potranno divergere, anche in modo significativo» dalle sette [programma del corso]: per ogni capitolo le domande aperte ovvie e la sezione che risponde.

| Capitolo                        | Domande aperte ovvie (sezione con la risposta)                                                                                                                                                     |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. NLP, testo e normalizzazione | Che cos'è il NLP, applicazioni e sfide (§1.1); la pipeline di normalizzazione del testo (§1.4); tokenization subword e BPE con un esempio (§1.4.2); confrontare stemming e lemmatization (§1.4.3). |
| 2. Modelli n-gram               | Language model n-gram: chain rule, assunzione di Markov, MLE (§2.1); che cos'è la perplexity e come si valuta un LM (§2.3); Kneser-Ney e la continuation probability (§2.4.5).                     |
| 3. Naive Bayes                  | Descrivere il classificatore Naive Bayes, con la derivazione dalla regola di Bayes e l'add-1 (§3.2); accuracy, precision, recall, $F_1$ , macro e micro average (§3.3).                            |

|                          |                                                                                                                                                                                                                                    |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4. Regressione logistica | Descrivere la regressione logistica: sigmoide, cross-entropy loss, SGD (§4.2–4.4); confrontare classificatori generativi e discriminativi (§4.1); softmax e caso multinomiale (§4.6).                                              |
| 5. Semantica vettoriale  | Ipotesi distribuzionale e matrici di co-occorrenza (§5.2); confrontare tf-idf e word2vec (§5.4, §5.5); analogie col parallelogramma e bias degli embedding (§5.6).                                                                 |
| 6. Reti neurali          | Perché serve un hidden layer: il problema XOR (§6.2); il feed-forward language model contro gli n-gram (§6.4); come si addestra una rete: loss, gradiente, backpropagation (§6.5).                                                 |
| 7. Transformer           | Il transformer block: residui, layer norm, multi-head, positional embedding (§7.2); generare testo: campionamento, greedy, beam search (§7.3); architetture encoder-decoder e cross-attention (§7.5).                              |
| 8. POS tagging e NER     | POS tagging e NER come sequence labeling, schema BIO (§8.1, §8.2); HMM e Viterbi contro il tagger BERT (§8.1); entity linking: bi-encoder e cross-encoder di BLINK (§8.3).                                                         |
| 9. Parser sintattici     | CFG, forma normale di Chomsky e algoritmo CKY con la tabella (§9.2); confrontare costituenti e dipendenze (§9.3); il transition-based parser e il suo oracle (§9.4).                                                               |
| 10. Parser semantici     | Meaning representation, desiderata e logica del primo ordine (§10.1, §10.2); confrontare PropBank e FrameNet; che cos'è il SRL e come si realizza (§10.3); AMR e il parser SPRING (§10.4, §10.5).                                  |
| 11. GPT e LLM            | Pre-training e fine-tuning di GPT (§11.1); zero/few-shot, in-context learning, che cosa sono le emergent abilities (§11.2); InstructGPT e RLHF: SFT, reward model, RL (§11.3); decoding: temperature, top- $k$ , top- $p$ (§11.4). |
| 12. Applicazioni         | Information retrieval (sparso: tf-idf, BM25; denso: bi-encoder) e RAG; QA open domain: retriever, reader a span (§12.1); machine translation e summarization: valutare con BLEU, ROUGE, BERTScore (§12.2).                         |

---

Tabella 13.1: Altre domande aperte per capitolo e dove sta la risposta nella guida.